



Uncertainty Quantification for Surrogate Models in Flow Simulations

Master's Thesis presented by

Sabrina Ebert

Sabrina.ebert@studium.uni-hamburg.de

Matricule nr: 7828210

For the Degree of
M.Sc. Atmospheric Science

At the Meteorologic Institute
University Hamburg
Jointly with AG Informatics
DESY Hamburg
Germany

Submission date: 19. December 2025

First reviewer: Prof. Dr. Philipp Neumann

Second reviewer: Prof. Dr. Juan Pedro Mellado

Supervisors: Dr. Yannis Schumann and MSc. Olga Catalan Aragall

Contents

List of Abbreviations	10
Acknowledgements	11
1 Introduction and Motivation	12
2 Theoretical Background	15
2.1 Uncertainty Quantification in Neural Networks	15
2.1.1 Historical Overview	15
2.1.2 Aleatoric and Epistemic Uncertainty	17
2.1.3 Aleatoric (Data) Uncertainty	20
2.1.4 Epistemic (Model) Uncertainty	24
2.1.5 Method Selection and Expected Performance	37
2.2 Flow Scenario Data: Two Idealized Flows	41
2.2.1 Plane Couette Flow	41
2.2.2 Kármán Vortex Street (KVS)	42
2.2.3 Analysis of Extracted Flow Parameters	45
2.3 Infrastructure: Base Network and Flow Data Generation Software”	50
2.3.1 Convolutional Recurrent Autoencoder for Flow Prediction	50
2.3.2 Flow Data Simulation Software	53
3 Simulation Setup, Datasets, and Training	55
3.1 Domain Geometry and Physical Setup	55
3.2 MD Box Placement Strategy	56
3.3 Simulation Procedure and Initialization	58
3.4 Datasets for Training and Out-of-Distribution Evaluation	58
3.5 Training Procedures and Hyperparameter Optimization	59
3.5.1 Reproduction of Baseline Architecture	59
3.5.2 Training and Hyperparameter Optimization of New Architectures	59
3.6 Evaluation Metrics	60
4 Results	63
4.1 Aleatoric Uncertainty Results	63
4.1.1 Aleatoric Uncertainty Distribution and Fit	64

4.1.2	Aleatoric Operational Performance	67
4.1.3	Final Model Comparison	68
4.2	Hyperparameter Tuning of Epistemic Methods	69
4.2.1	Sensitivity of Gaussian Process Kernels	69
4.2.2	Gaussian Process Hyperparameter Tuning	71
4.2.3	MC Dropout Hyperparameter Tuning	73
4.2.4	Deep Ensemble Hyperparameter Tuning	74
4.2.5	Evidential Network Hyperparameter Tuning	76
4.2.6	Laplace Approximation Hyperparameter Tuning	77
4.3	Epistemic Uncertainty Results	78
4.3.1	Epistemic Uncertainty Distribution	78
4.3.2	Operational, Error, and OOD Performance	84
4.4	AE-RNN Uncertainty Propagation and OOD Detection	91
4.5	Epistemic Uncertainty Propagation in the AE-RNN Pipeline for Coupled De- terministic Models	95
5	Conclusion	99
5.1	Summary of Findings	99
5.2	Discussion	101
5.3	Future Work	101
	Appendix	104
	Introductory Machine Learning and Neural Networks	104
	Further theory details relevant for the Methods	108
	Sparse Laplace Approximation:	108
	Sparse Gaussian Process Formulations	109
	Simulation Software Details	110
	Spatial Domain Decomposition and Macroscopic Cell Structure	110
	Simulation physics	111
	Boundary Conditions and Thermostat	115
	Simulation Settings and Data Preprocessing	116
	Simulation Initialization	116
	Training Details and Hyperparameters	117
	Reproduction of AE-RNN Results	117
	Hyperparameter Optimization and Training for Epistemic Methods	120
	Methodology Workflow Sketch	122
	Gaussian Process Kernel Functions	122
	Gaussian Process Kernel Comparison Results	126
	Separation Metrics Results	128
	Evidential Propagation Bounds	130

Histogram for Random OOD Dataset for AE vs AE-RNN	131
5.4 Code Availability: GitHub Repository	132

List of Figures

2.1	Short timeline of some uncertainty quantification methods	16
2.2	Aleatoric vs. Epistemic NN Uncertainty Explanatory Sketch	18
2.3	Model uncertainty sources in supervised learning	19
2.4	Deterministic uncertainty quantification networks	21
2.5	Parametric vs. non-parametric uncertainty quantification	21
2.6	Sources of variation in deep ensembles	25
2.7	Bagging and boosting ensemble strategies	26
2.8	BNN sketch	28
2.9	Monte Carlo Dropout inference pipeline	30
2.10	Sketch showing GPs example used on a Toy case	32
2.11	Gaussian Process deep learning architectures	33
2.12	Laplace approximation behaviour of posterior distribution	35
2.13	Hierarchical uncertainty in evidential deep learning	37
2.14	Loss landscape coverage of different NN UQ methods	40
2.15	Couette flow basic sketch	42
2.16	KVS flow basic sketch	43
2.17	Satellite image of KVS behind Guadalupe island	44
2.18	Different urban canyon main flow connection modes	45
2.19	Full Sketch of the AE-RNN pipeline for flow prediction	51
3.1	Data extraction visualization from the full domain for the KVS simulation case	56
3.2	Data extraction visualization from the full domain for the Couette simulation case	57
3.3	OOD-Dataset flow scenarios comparison sketch	59
4.1	Example Time series of NLL, Evidential, Quantile and PI aleatoric result coverage of target noise	65
4.2	Aleatoric interval width comparions	66
4.3	Computational Cost and Prediction influence of Aleatoric methods	67
4.4	PCA of training and inference data in latent space	70
4.5	Histogram of euclidian distance for PCA dimensions	71
4.6	Sketch of rbf kernel hyperparameter fine-tuning 1	72
4.7	GP RBF Kernel hyperparameter fine-tuning LS and SV	73
4.8	Fine-tuning of MCDropout method	74

4.9	Finetune of Ensemble member numbers	75
4.10	Ensemble width with training	76
4.11	Finetune of evidential offsets	77
4.12	Finetuning of Laplace	77
4.13	Example time series behavior of epistemic results for all methods	79
4.14	Histogram of epistemic uncertainty results for all methods for Reference vs TrueRandom	81
4.15	Normalised Boxplots of the Referenz vs. OOD Datasets for all epistemic methods	82
4.16	Normalised CDFS of the Referenz vs. OOD Datasets for all epistemic methods	84
4.17	Operational metrics for AE inference	85
4.18	Error correlation metrics heatmap	86
4.19	OOD metric results heatmap	87
4.20	Table summary uncertainty pipelines	90
4.21	AE vs RNN epistemic uncertainty magnitudes	92
4.22	Error correlation heatmap full pipeline	93
4.23	OOD heatmaps full pipeline	93
4.24	Example merging process sketch for epistemic uncertainty propagation in de- terministic methods in coupled AE-RNN	96
4.25	Full test propagation sketch for deterministic evidential interdependent models	97
5.1	Sketch of a simple inverse problem	105
5.2	Simple neural network sketch	106
5.3	Explanatory sketch of loss landscape and convergence algorithm behavior be- hind basic NN weights training functionality.	107
5.4	Dataset split for neural network training	108
5.5	Nested domains of the data simulation software	110
5.6	Transfer regions inside the smaller MD domain in the simulation software . . .	113
5.7	Sketch of the Micro $\leftrightarrow$ Macro Parameter transfer, by Smaller MD cell region .	113
5.8	Raw simulation result showing the flow convergence.	118
5.9	Reproduction simulation test	119
5.10	Grid search of training hyperparameters loss results for epistemic models boxplot	121
5.11	Overall workflow sketch after AE-RNN reproduction	122
5.12	Main compared Gp kernels shape and prior samples visualization	123
5.13	Influence of α on RQ kernel	124
5.14	Influence of ν on differentiability/smoothness of Matern kernel	125
5.15	Hyperparameter heatmap and calibration and separation comparison for dif- ferent kernels for GP	127
5.16	Epistemic uncertainty distribution results by dataset for different kernel types for GP	128
5.17	Barplot of propagating evidential bounds	130

5.18 Histograms of AE vs AE+RNN uncertainty quantification methods	131
--	-----

List of Tables

2.1	Model uncertainty sources in supervised learning framework (He et al., 2023)	20
2.2	KVS Flow Parameters from MD Box Data	47
2.3	Couette Flow Parameters from Configuration Files	48
3.1	Domain configuration overview	55
3.2	KVS MD box positions	57
3.3	Couette MD box positions	57
3.4	Out-of-distribution test scenarios	58
4.1	Epistemic Method AE Results: Original Dataset	88
4.2	Epistemic Method AE Results: ID-OOTR Dataset	88
4.3	Epistemic Method AE Results: Couette Dataset	89
4.4	Epistemic Method AE Results: TrueRandom Dataset	89
4.5	Epistemic Method AE-RNN Results: TrueRandom Dataset	94
4.6	Epistemic Method AE-RNN Results: ID-OOTR Dataset	94
4.7	Epistemic Method AE-RNN Results: Couette Dataset	95
5.1	Running Parameters for KVS and Couette Simulations	116
5.2	Boundary Conditions and Coupling Mechanisms	117
5.3	Reproduction Simulation Training Hyperparameters (from (Jarmatz et al., 2023))	118
5.4	Training Hyperparameters for Epistemic Uncertainty Methods	120
5.5	Distribution Distance Metrics: TrueRandom Dataset	129
5.6	Distribution Distance Metrics: ID-OOTR Dataset	129
5.7	Distribution Distance Metrics: Couette Dataset	129
5.8	Distribution Distance Metric Abbreviations and Interpretations	129

List of Abbreviations

NN Neural Network [9](#)

UQM Uncertainty Quantification Method [9](#)

KVS Karman Vortex Street (flow scenario) [9](#)

OOD Out-Of-Distribution [9](#)

AE Autoencoder [9](#)

RNN Recurrent neural network [9](#)

VAE Variational Neural Network [9](#)

GAN Generative Neural Network [9](#)

GP Gaussian Processes [9](#)

Acknowledgements

I would like to take this opportunity to personally acknowledge some of the persons involved in the success of this thesis and my personal growth.

First and foremost, I am extremely grateful to have had two of the best supervisors throughout my thesis journey. Yannis Schumann and Olga Catalan Aragall provided invaluable support. They struck a perfect balance between allowing me the time to learn, think, and question on my own, while also being readily available for extremely helpful guidance. Their very insightful questions and feedback, both structural and content-related, was perfect and empowered me to give 120% to my thesis and motivated me to develop further questions and interests on the topic. I want to sincerely thank them, not only for their amazing academic support but also for their contributions to my personal development.

I would also like to thank my professors, Philipp Neumann and Juan Pedro Mellado, for facilitating this collaboration and allowing me to delve deeply into this complex and interesting topic. Their prompt responses and organizational support ensured that I could progress smoothly with my thesis, and their constructive feedback was immensely helpful.

In addition to my supervisors and professors, I wish to express my gratitude to Anja, Dijana, Reza, Mohammad, Gedeon, Stella for their valuable advice e.g. on the structure of the thesis, for reading through it to enhance its clarity or listening to my presentation.

Finally, I want to extend heartfelt thanks to all my friends and family generally for their unwavering support and for being such wonderful companions throughout this journey.

1

Introduction and Motivation

Machine learning, particularly deep neural networks, has revolutionized various scientific and engineering fields. In atmospheric physics, recent breakthroughs demonstrate significant potential in numerical weather prediction (NWP). Notably, Google DeepMind’s GraphCast and GenCast ([Lam et al., 2023](#); [Price et al., 2025](#)) have achieved state-of-the-art results in medium-range deterministic and probabilistic weather forecasting. They utilize significantly fewer computational resources than traditional physics-based models while delivering results that are comparable to, or even more accurate than, those of conventional models for most variables. Additionally, these models even offered more precise estimations of extreme weather events. Other NWP models, such as FourCastNet ([Pathak et al., 2022](#)) and Pangu-Weather ([Bi et al., 2023](#)), have also demonstrated similar improvements in accuracy and reductions in computational time.

Beyond NWP, machine learning has contributed to several specialized atmospheric physics applications, including sea ice prediction ([Chi and Kim, 2021](#)), observational data interpolation ([Kadow et al., 2020](#)), and the detection of specific extreme weather events ([Rasp et al., 2020](#)). The rapid development of machine learning applications has also impacted many other fields beyond atmospheric physics. For example, it has made significant strides in medical diagnosis ([Esteva et al., 2017](#); [Rajkomar et al., 2018](#); [Topol, 2019](#)) and autonomous systems ([Wang et al., 2025](#)).

Predictions made by neural networks are beginning to inform critical decisions across multiple domains, significantly influencing human safety, economic stability, and scientific understanding.

Traditional neural networks often provide only point predictions, without offering estimates of confidence or reliability that account for noise, model uncertainties, or variations in training data. This "black-box" nature can create challenges and risks, particularly when it comes to trust and explainability, especially in high-stakes situations where model outputs influence critical decisions. For example, inaccuracies in hurricane predictions can result in billions of dollars in damages or, at the very least, lead to unnecessary evacuation costs ([Rappaport, 2014](#)). Additionally, in other fields, overconfident diagnostic models in medical settings can lead to misdiagnoses ([Kompa et al., 2021](#)). Autonomous systems also

require uncertainty estimation to operate safely (Osband et al., 2016).

Regulatory frameworks, such as the European Union’s AI Act (European Commission, 2021), demand transparency and risk assessment for high-risk AI applications, and uncertainty quantification plays a vital role in this. Similarly, the World Meteorological Organization (WMO) emphasizes that decision-makers not only need accurate predictions but also reliable estimates of prediction confidence, reinforcing the importance of transparency and uncertainty assessment in operational forecasting (WMO, 2021).

While uncertainty quantification (UQ) has been extensively studied for various neural network tasks, particularly in image classification Gal (2016); Lakshminarayanan et al. (2017); He et al. (2023), its application to neural networks for flow prediction remains relatively underexplored. Recent studies have begun to address UQ in specific contexts, such as turbulence closures Iskhakov et al. (2025), reduced-order models Sperrer (2025), and architectures like physics-informed neural networks (PINNs) Sun et al. (2020); Flores et al. (2025); Shih et al. (2025), as well as recurrent neural networks (RNNs) McDermott and Wikle (2019); Pollithy et al. (2020). However, compared to other fields, these examples are limited, and there remains a significant gap in systematic analysis and comparison of UQ methods.

Flow prediction datasets and models presents unique challenges, such as partial differential equation (PDE)-constrained dynamics with complex nonlinear interactions, strong spatial and temporal correlations across multiple time scales Sun et al. (2020); Valero and Meldi (2025), and localized areas of extreme sensitivity. Gao et al. (2025); Chu and Qian (2025) These characteristics complicate the direct application of UQ methods and findings from other neural network tasks, like image classification further.

The main goal of this thesis is to implement and evaluate various uncertainty quantification (UQ) methods within an existing reproduced autoencoder-recurrent neural network (AE-RNN) architecture for flow prediction. This thesis therefore aims to contribute to the gap in the field, as well as contribute valuable insights for uncertainty quantification methods in the working group’s previous neural network architecture.

Particular emphasis will be placed on the effectiveness of these UQ methods in identifying model errors and detecting out-of-distribution (OOD) data, thus enhancing the reliability assessment of the neural network. The specific objectives include the practical integration of multiple UQ methods into the AE-RNN architecture and the analysis and comparison of their capabilities for OOD detection.

Furthermore the key research questions to be explored are: How effective are the UQ methods when applied to the AE-RNN architecture designed for flow prediction? What are the behavioral characteristics of the various UQ methods in relation to OOD data? Additionally, due to the nature of the architecture, how does UQ propagate through the interconnected layers of the neural network?

With the goals and objectives set forth, the structure of the thesis is organized as follows. First, for readers who may not have experience with machine learning and neural networks, a brief appendix provides an introduction to key terms and concepts (see Appendix 5.3). The theoretical background in Chapter 2 reviews the history of uncertainty quantification methods and introduces key distinctions. It then provides a detailed discussion of the functionality and mathematical theory behind the methods used, categorized into aleatoric and epistemic uncertainty approaches (Section 2.1).

For the training data, an idealized Kármán vortex street (KVS) domain is employed. For out-of-distribution (OOD) testing, furthermore besides KVS data, idealized Couette flow and artificially generated random data are utilized.(Section 3.4) The flow scenarios, their rationale for evaluating uncertainty quantification, meteorological relevance, and data extraction processes are detailed in Section 2.2. These flow cases are generated using computational fluid dynamics (CFD) based on a lattice Boltzmann method (LBM) solver, coupled with a molecular dynamics (MD) solver via the MaMiCo software (Neumann et al., 2016) (Section 2.3).

The base architecture consists of a convolutional autoencoder for spatial feature extraction, combined with a recurrent neural network for temporal dynamics. This approach replicates previous work by (Jarmatz et al., 2023) and serves as background for the results discussed in Section 2.3.1.

Evaluation proceeds in three stages, as outlined in Chapter 4. First, we assess the method’s sensitivity to hyperparameter fine-tuning and the intrinsic behavior of epistemic uncertainty distributions (Section 4.2). Second, we evaluate the performance of uncertainty quantification methods on the autoencoder component, comparing methods for error correlation, OOD detection, and operational behavior in relation to the research questions posed (Sections 4.1 and 4.3). Finally, we select promising methods for a comprehensive evaluation of the full AE-RNN pipeline, implementing these methods into the RNN and, where necessary, propagating uncertainty through dependent models (Section 4.5).

The model weights, run scripts, configuration files, and visualization scripts will be shared on GitHub (Appendix 5.4).

2

Theoretical Background

2.1 Uncertainty Quantification in Neural Networks

This section provides a comprehensive overview of uncertainty quantification methods for neural networks and offers a detailed examination of the methods used. This is the main theoretical part of this thesis, which is essential for understanding the results. It is organized to guide the reader through a very brief historical timeline, which will help in understanding the distribution in existing methods better and the emergence of some of the used methods timeframe (see Section 2.1.1). Following this, there is a short introduction to key theoretical distinctions relevant for characterizing these methods (see Section 2.1.2). The discussion then moves to specific methods, including theoretical analyses and implementations relevant for understanding the results of the methods (see Section 2.1.3), (see Section 2.1.4). Finally, the section concludes with a summary and expectation analysis of these methods (see Section 2.1.5).

2.1.1 Historical Overview

Traditional neural networks provide point estimates for regression or classification tasks without confidence or reliability measures. The foundation for uncertainty quantification in neural networks began in the late 1980s with Bayesian approaches (Denker et al., 1987), (Moody and Darken, 1989), (Neal, 1993). Bayesian Neural Networks (BNNs) model network weights as probability distributions, capturing model uncertainty through these learned distributions.

To make Bayesian approaches practical for larger models, approximate inference methods like variational Bayesian networks emerged (MacKay, 1992), (Neal, 1993), (Hinton and Van Camp, 1993), (Graves, 2011). Gaussian Processes (GPs), another Bayesian method for uncertainty quantification (Rasmussen and Williams, 2006), assess similarities between data points using distance-based kernels.

Monte Carlo Dropout has been introduced as an approximation for BNNs, effectively serving as an approximate Bayesian inference method (Gal and Ghahramani, 2016). Deep ensembles,

where multiple independent networks are trained and their predictions aggregated, are also used to estimate uncertainty (Lakshminarayanan et al., 2017).

Calibration techniques, like temperature scaling, adjust overconfident predictions so model outputs align more closely with true likelihoods (Guo et al., 2017). Generative models, including Variational Autoencoders (VAEs) (Kingma and Welling, 2014) and Generative Adversarial Networks (GANs) (Goodfellow et al., 2014), enhance uncertainty quantification by modeling variability in training data.

Recent advancements in reducing computational costs include evidential deep learning, which captures both aleatoric and epistemic uncertainty in a single forward pass (Sensoy et al., 2018). Additionally, conformal prediction methods offer distribution-free uncertainty sets with guaranteed coverage properties (Angelopoulos et al., 2021).

Different types of uncertainty are crucial in applications: aleatoric uncertainty arises from inherent data noise, while epistemic uncertainty stems from limited knowledge or insufficient training data (Senge et al., 2014). Understanding this distinction is essential for employing the appropriate uncertainty estimation methods. Recent research also started investigating the influence of the loss landscape on the model uncertainty captured more (Fort et al., 2019; Li et al., 2018).

In 2.1 below, a visual summary of the chronological order of the relevant uncertainty methods discussed can be found.

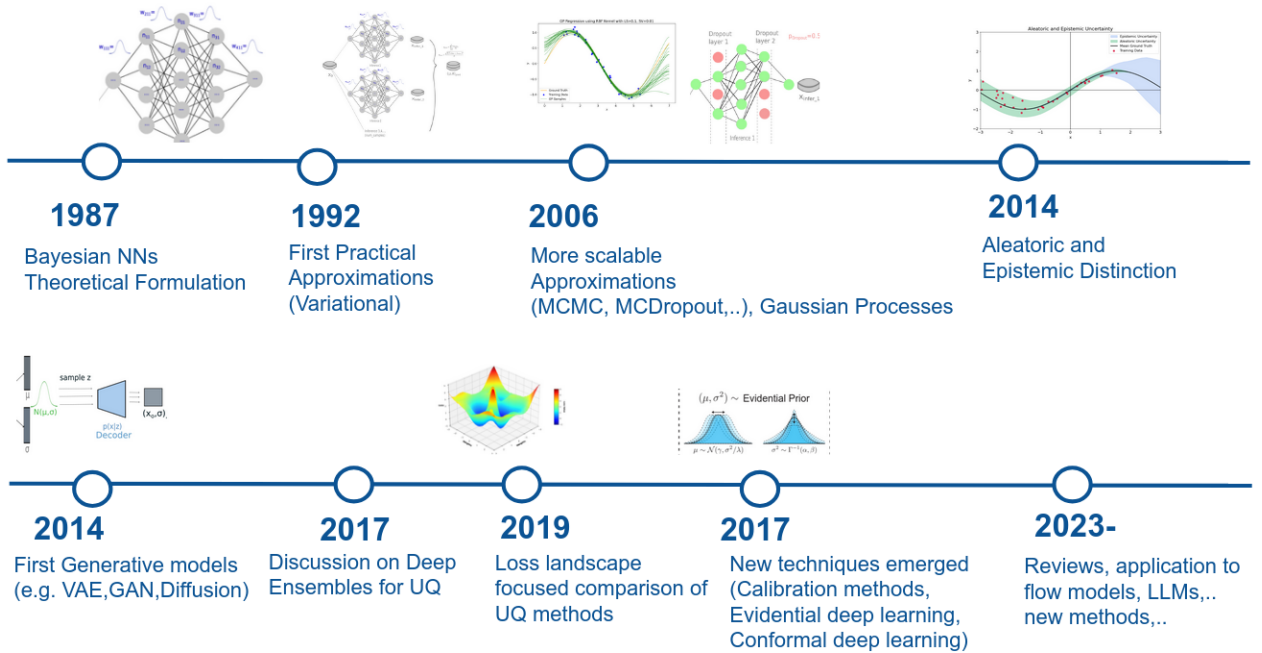


Figure 2.1: Summary timeline highlighting some of the key milestones and advancements in relevant quantification methods that will be discussed in this thesis

The following sections explore these methods in detail, starting with the formal distinction between aleatoric and epistemic uncertainty. This will be followed by specific techniques for quantifying each type, including their mathematical foundations and practical considerations.

2.1.2 Aleatoric and Epistemic Uncertainty

As discussed in the objectives 1, the focus is on quantifying uncertainty, utilizing different methods based on the type of uncertainty involved (as briefly mentioned in the historical review). This section aims to provide a deeper understanding of aleatoric and epistemic uncertainties, along with the various methods and formulations used to address them.

As introduced in the historical overview, (Senge et al., 2014) and others distinguish between two types of uncertainty: *aleatoric uncertainty* (data uncertainty), which arises from inherent noise or variability in the data, and *epistemic uncertainty* (model uncertainty), which originates from limited knowledge or insufficient training data. This distinction is beneficial, as different applications require different types of uncertainty, and the methods for estimating them can differ significantly; hence, they are usually categorized into these two groups (Het al., 2023; Gawlikowski et al., 2023).

As illustrated in Figure 2.2, aleatoric uncertainty denotes irreducible variability in the data (such as sensor noise, measurement errors, and conflicting evidence) and remains constant even with unlimited training data. In contrast, epistemic uncertainty reflects the model’s lack of knowledge about the underlying function and decreases as more training data becomes available, particularly in areas where data is sparse (Senge et al., 2014). This conceptual framework guides subsequent research and serves as the basis for categorizing uncertainty quantification methods, which will be discussed in the following sections.

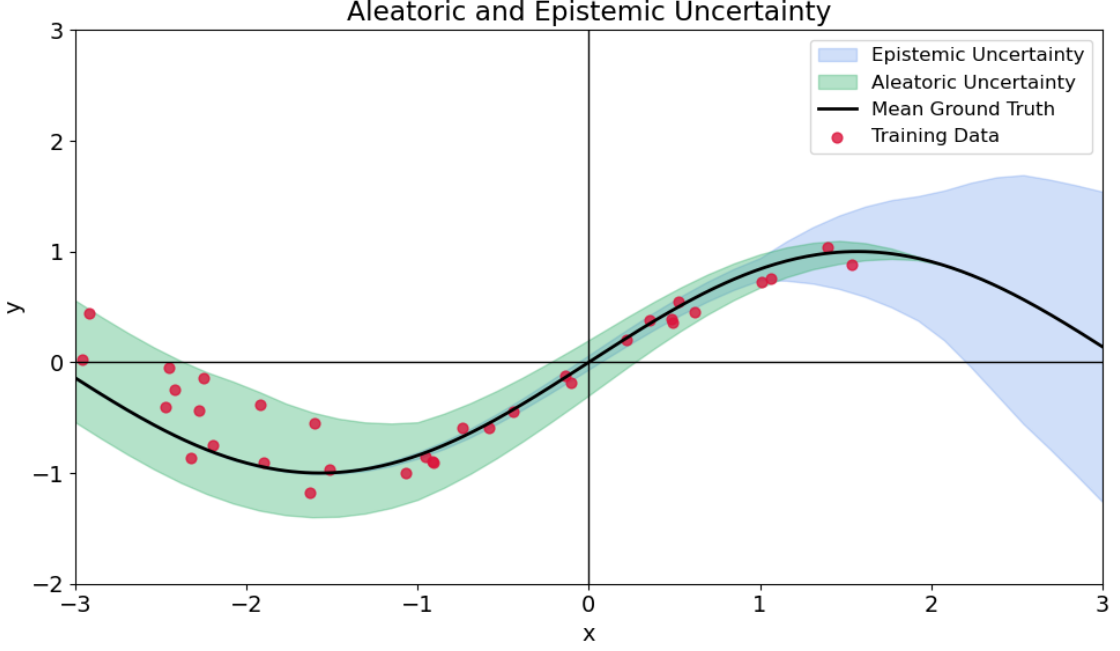


Figure 2.2: Difference between aleatoric uncertainty (green interval) and epistemic uncertainty (blue interval). Aleatoric uncertainty represents inherent data noise and remains constant, while epistemic uncertainty reflects model uncertainty and increases in regions with sparse training data.

These uncertainty types can be formalized within the supervised learning framework (He et al., 2023). Given training data $\mathcal{D}_{tr} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ drawn from an unknown distribution $p(\mathbf{x}, y)$, the learning problem seeks the optimal hypothesis h^* within a hypothesis space $\mathcal{H}$ that minimizes the expected risk:

$$h^* = \arg \min_{h \in \mathcal{H}} R(h), \quad \text{where } R(h) = \int l(y, h(\mathbf{x})) p(\mathbf{x}, y) d\mathbf{x} dy, \quad (2.1)$$

where h^* is the theoretical optimal hypothesis, $\mathcal{H}$ is the hypothesis space (set of possible models), $R(h)$ is the expected risk (average loss over the true data distribution), $l(y, h(\mathbf{x}))$ is a loss function measuring the discrepancy between true output y and prediction $h(\mathbf{x})$, and $p(\mathbf{x}, y)$ is the true joint distribution over inputs $\mathbf{x}$ and outputs y . In practice, models minimize the empirical risk over finite training data:

$$\tilde{h} = \arg \min_{h \in \mathcal{H}} R_{emp}(h), \quad \text{where } R_{emp}(h) = \frac{1}{|\mathcal{D}_{tr}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{tr}} l(y_i, h(\mathbf{x}_i)). \quad (2.2)$$

In this context, let $\tilde{h}$ represent the learned hypothesis, which may differ from the optimal hypothesis h^* due to limited data. The term $R_{emp}(h)$ denotes the empirical risk, which is the average loss calculated over a finite training set, and $|\mathcal{D}_{tr}|$ refers to the number of training samples. The summation is taken over all training pairs $(\mathbf{x}_i, y_i)$ in the training dataset $\mathcal{D}_{tr}$.

Within this framework, *aleatoric uncertainty* arises from the inherent randomness in the joint distribution $p(\mathbf{x}, y)$ itself. This type of uncertainty reflects the irreducible variability in the data generation process that cannot be removed, even with perfect models. Conversely, *epistemic uncertainty* originates from three primary sources: (1) *model hyperparameter uncertainty*, where the optimal solution h^* within a chosen hypothesis space $\mathcal{H}$ may differ from the theoretical optimum f^* in the larger hypothesis space $\mathcal{F}$; (2) *model parameter learning uncertainty*, where the learned solution $\tilde{h}$ may not align with the optimal solution h^* due to limited training data or imperfect optimization; and (3) *distribution shift uncertainty*, which occurs when the distribution of inference samples $p_2(\mathbf{x}, y)$ differs from that of the training data $p_1(\mathbf{x}, y)$, leading to differences between the theoretical optima f_1^* and f_2^* , and increasing uncertainty for out-of-distribution samples (He et al., 2023).

Figure 2.3 illustrates these relationships, depicting how f^* , h^* , and $\tilde{h}$ relate within the hypothesis spaces $\mathcal{F}$ and $\mathcal{H}$, and how various sample distributions result in different theoretical optima. These sources of uncertainty are summarized in Table 2.1.

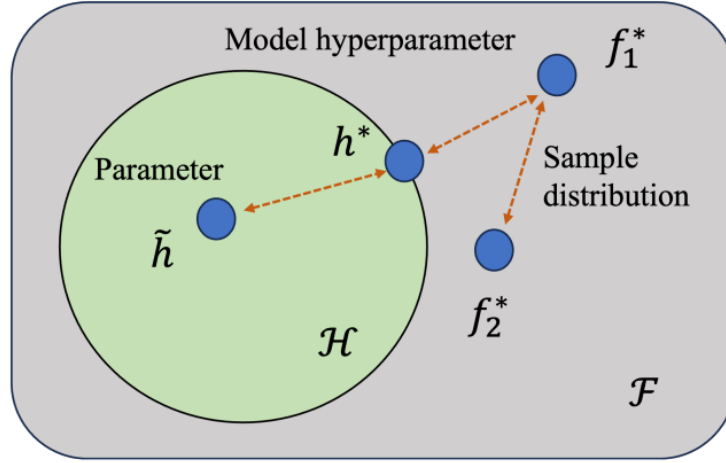


Figure 2.3: Helpful visual illustration of model uncertainty sources in supervised learning (He et al., 2023). The outer space $\mathcal{F}$ represents the full hypothesis space, while $\mathcal{H}$ (green circle) is a sub-hypothesis space for a particular model architecture. The theoretical optimal f^* in $\mathcal{F}$ may differ from h^* in $\mathcal{H}$ (hyperparameter uncertainty), and the learned solution $\tilde{h}$ may not reach h^* (parameter learning uncertainty). Different sample distributions $p_1(\mathbf{x}, y)$ and $p_2(\mathbf{x}, y)$ yield different theoretical optima f_1^* and f_2^* (distribution shift uncertainty) (He et al., 2023).

Table 2.1: Model uncertainty sources in supervised learning framework (He et al., 2023)

Model uncertainty sources	Corresponding notation in supervised learning
Choice of model hyper-parameter	Optimal solution h^* within $\mathcal{H}$ does not align with theoretical optimal f^* in $\mathcal{F}$
Model parameter learning	Learned solution $\tilde{h}$ does not align with optimal h^* in $\mathcal{H}$
Different sample distributions in learning and inference	Theoretical optimum f_1^* and f_2^* mismatch under different sample distribution $p(\mathbf{x}, y)$

The distinction between aleatoric and epistemic uncertainty is important because different applications may require varying types of uncertainty quantification, and sometimes even necessitate their separation (He et al., 2023; Gawlikowski et al., 2023). Additionally, the methods used to estimate these uncertainties can differ, as will be detailed in the methods sections below.

The following sections will explore specific methods for quantifying these types of uncertainty, starting with aleatoric uncertainty methods, which are further categorized into parametric and non-parametric approaches.

2.1.3 Aleatoric (Data) Uncertainty

As introduced in the previous section, aleatoric uncertainty (also known as data or irreducible uncertainty) arises from the inherent noise or variability in the data. This can include inherent measurement errors, intrinsic stochasticity (such as thermal noise in molecular dynamics), or unresolved subgrid processes. Unlike epistemic uncertainty, aleatoric uncertainty cannot be reduced by collecting more data, as it is an intrinsic property of the underlying process or the training data. This section explores methods for quantifying aleatoric uncertainty, organized by their approaches to modeling data variability.

Commonly used methods for addressing aleatoric uncertainty can be categorized into discriminative and generative approaches based on their functionality. Discriminative methods directly learn to output uncertainty parameters or prediction bounds, while generative methods focus on learning the underlying data distribution. As illustrated in Figure 2.4, deterministic uncertainty quantification (UQ) networks extend standard neural networks by adding output neurons that predict uncertainty parameters alongside the mean. This enables uncertainty estimation in a single forward pass. This section primarily focuses on discriminative approaches, as they are utilized in this work; generative methods are briefly discussed for theoretical completeness in Section 2.1.3.

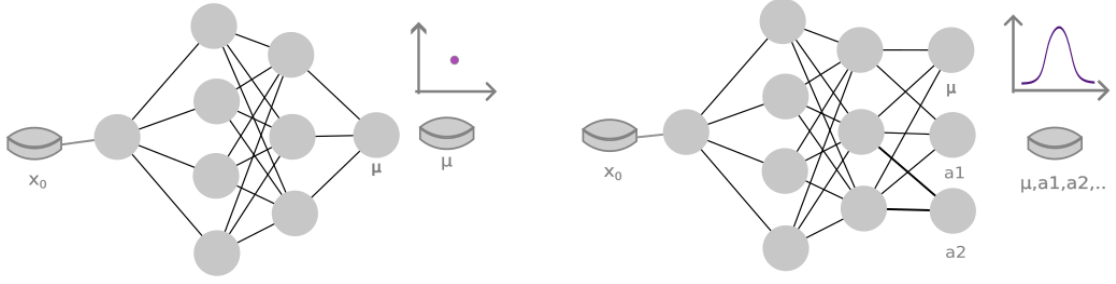


Figure 2.4: Comparison of typical neural networks and deterministic uncertainty quantification networks. Standard networks (Left) output a single point prediction (μ) with no information about confidence or variability. Discriminative Uncertainty quantification networks (Right) extend the output layer with additional neurons and loss terms, predicting uncertainty parameters (μ , a_1 , a_2 , ...) alongside the mean. During training, the network learns to predict both mean and uncertainty parameters (e.g., variance or distribution parameters) for each input, enabling uncertainty estimation in a single forward pass.

Discriminative models can be further divided into parametric and non-parametric approaches. Parametric models assume a specific distributional form (e.g., Gaussian) and learn input-dependent noise levels, capturing heteroscedasticity in the data, while non-parametric models predict intervals or quantiles without assuming an explicit distributional form. The distinction is illustrated in Figure 2.5.

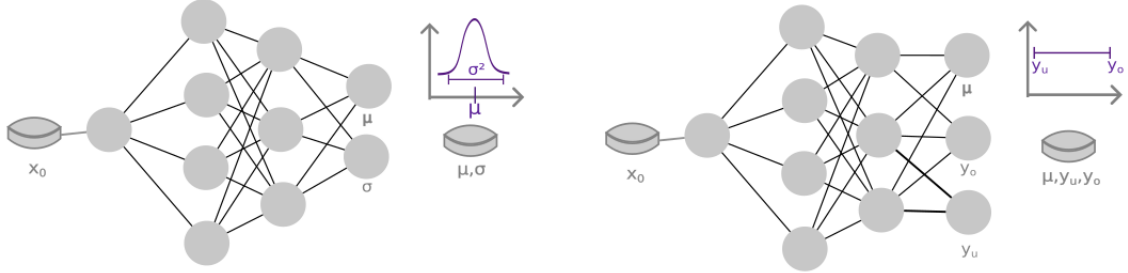


Figure 2.5: Comparison of parametric and non-parametric discriminative models for aleatoric uncertainty quantification. The network outputs distribution parameters (μ , σ) assuming a specific distributional form (e.g., Gaussian), where σ^2 represents input-dependent variance capturing heteroscedasticity (Left). The network directly outputs prediction bounds (y_u , y_o) without assuming an explicit distributional form, providing prediction intervals that capture uncertainty without distributional assumptions. (Right)

In contrast to parametric approaches, non-parametric methods do not assume explicit

distributional forms, directly predicting bounds or quantiles. This flexibility makes them valuable when distributional assumptions are uncertain or distributions are complex.

The following subsections examine specific well known methods for quantifying aleatoric uncertainty, organized by investigating two parametric and two non-parametric approaches.

Negative Log-Likelihood (NLL)

The most well known parametric approach is probably using a gaussian as output distribution shape, the Gaussian likelihood (Bishop, 2006). Heteroscedastic regression models are generally expressed as $y_i = f_\theta(\mathbf{x}_i) + \epsilon(\mathbf{x}_i)$, where $\epsilon(\mathbf{x}_i)$ here represents independent and heterogeneous Gaussian noise. This noise captures sample-dependent aleatoric uncertainty (Kendall and Gal, 2017). Such a spatial heteroscedastic model can help especially for applications in meteorology and fluid dynamics, where uncertainty levels can vary across different input regions (for example, there is often higher uncertainty in less stable atmospheric conditions).

Maximum likelihood estimation employs the negative log-likelihood function to estimate parameters effectively. (NLL) loss (Bishop, 2006; Kendall and Gal, 2017):

$$\mathcal{L}_{\text{NLL}} = \frac{1}{N} \sum_{i=1}^N \left[\frac{(y_i - \mu_i)^2}{2\sigma_i^2} + \frac{1}{2} \log \sigma_i^2 \right] \quad (2.3)$$

where N is the number of training samples, y_i is the ground truth, $\mu_i = f_\theta(\mathbf{x}_i)$ is the predicted mean, and σ_i^2 is the predicted variance. The first term is scaled squared error decreasing with better predictions, while the second term $\frac{1}{2} \log \sigma_i^2$ acts as an implicit regularizer preventing arbitrarily small variances. This balance prevents artificial uncertainty reduction to near-zero values. Variances are parameterized as $\sigma^2 = \exp(\hat{\sigma})$ or $\sigma = \text{softplus}(\hat{\sigma})$ to ensure positivity. (Bishop, 2006)

Gaussian Mixture Models (GMM)

Gaussian assumptions can be insufficient for more multimodal distributions. Gaussian Mixture Models (GMMs) extend the parametric framework to accommodate multiple prediction modes, also possibly relevant for meteorological and fluid dynamics where different conditions lead to fundamentally different outcomes. A GMM represents the conditional output distribution as a weighted sum of K Gaussian components (Bishop, 2006):

$$p(y|\mathbf{x}, \theta) = \sum_{k=1}^K \pi_k(\mathbf{x}; \theta) \mathcal{N}(y; \mu_k(\mathbf{x}; \theta), \sigma_k^2(\mathbf{x}; \theta)) \quad (2.4)$$

where K is the number of components, $\pi_k(\mathbf{x}; \theta)$ are mixture weights satisfying $\sum_{k=1}^K \pi_k = 1$ and $\pi_k \geq 0$, $\mu_k(\mathbf{x}; \theta)$ and $\sigma_k^2(\mathbf{x}; \theta)$ are component means and variances. E.g. Mixture Density Networks (MDN) integrate GMMs into neural architectures. (Bishop, 2006).

The NLL loss for MDN is (Bishop, 2006):

$$\mathcal{L}_{\text{MDN}} = -\frac{1}{N} \sum_{i=1}^N \log \left[\sum_{k=1}^K \pi_{k,i} \mathcal{N}(y_i; \mu_{k,i}, \sigma_{k,i}^2) \right] \quad (2.5)$$

where $\pi_{k,i} = \pi_k(\mathbf{x}_i; \theta)$, $\mu_{k,i} = \mu_k(\mathbf{x}_i; \theta)$, $\sigma_{k,i}^2 = \sigma_k^2(\mathbf{x}_i; \theta)$. GMMs capture aleatoric uncertainty through within-component variances (observational noise) and mixture structure (ambiguity in mode selection), though optimization is more challenging than unimodal Gaussians due to the log-sum structure (Bishop, 2006).

Quantile Regression

One of the most easiest example of a non-parametric is Quantile regression, it directly estimates prediction intervals by learning conditional quantiles of the target distribution. Traditional two-step approaches (point estimate then variance estimation) minimize prediction error rather than interval quality. Lower and Upper Bound Estimation (LUBE) directly optimizes interval width and coverage probability (Khosravi et al., 2011a), balancing coverage and efficiency. However, LUBE’s cost function is non-differentiable, requiring expensive global optimization.(Khosravi et al., 2011a). Quantile regression provides a simpler alternative by directly learning specific quantiles (e.g., 5th and 95th percentiles) without requiring complex optimization procedures.

Prediction Interval Coverage Probability (PICP)

Coverage width-based loss functions overcome LUBE’s non-differentiability through differentiable objectives (Khosravi et al., 2011b), optimizing Mean Prediction Interval Width (MPIW) and Prediction Interval Coverage Probability (PICP):

$$\mathcal{L}_{\text{PICP}} = \text{MPIW} + \lambda \cdot \max(0, (1 - \alpha) - \text{PICP})^2 \quad (2.6)$$

The metric $\text{MPIW} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_{u,i} - \hat{y}_{l,i}|$ measures the average width of prediction intervals, while $\text{PICP} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i \in [\hat{y}_{l,i}, \hat{y}_{u,i}]]$ assesses the empirical coverage fraction. The parameter λ controls the trade-off between these two metrics, and α represents the significance level. This encourages the development of narrow intervals while penalizing insufficient coverage(Khosravi et al., 2011b). The method directly optimizes the balance between interval width and coverage probability, without needing explicit distributional assumptions. However, it does come with the drawback of a complicated loss function.

Short summary on discriminative methods

Discriminative methods are generally straightforward to implement through adding the distribution/boundary parameters and further penalization terms to the loss. Depending on

the underlying data distribution and its consistency, one can choose between non-parametric and parametric approaches.

Non-parametric methods, such as quantile regression and Predictive Interval Coverage Probability (PICP), are valuable when making assumptions about the distributional form is undesirable. This is particularly true when the true distribution may be complex, multimodal, or poorly defined. Additionally, non-parametric methods offer intuitive and interpretable outputs, allowing practitioners to easily understand what it means for a prediction to fall within a given interval.

However, non-parametric methods also provide more information by producing explicit probability distributions, which simplifies certain probabilistic calculations, such as marginalizing over multiple variables. They typically also require smaller sample sizes to achieve good coverage properties.(He et al., 2023) That said, the width of the intervals can vary significantly based on optimization choices and hyperparameters, potentially leading to inconsistent uncertainty estimates across different regions of the input space.(He et al., 2023; Gawlikowski et al., 2023)

Ultimately, both parametric and non-parametric methods focus on fitting the data output variance to a specific distribution or bounds, but they may not always capture the inherent behavior of the data or the underlying system.

Generative Approaches

Generative models, in contrast to discriminative models, learn a representation of the prediction distribution itself, enabling sampling to generate new similar data. Well-known examples include variational autoencoders (VAE) (Kingma and Welling, 2014), generative adversarial networks (GAN) (Goodfellow et al., 2014), and diffusion models (Ho et al., 2020). These methods model aleatoric uncertainty by capturing variability in the training data through learned data distributions. However, due to constraints of scope, complexity, and convergence challenges, generative methods are not included in the experimental comparison of this work. They are mentioned here for theoretical completeness, as they represent an important class of aleatoric uncertainty quantification methods that may be explored in future work (see (Section 5.3)).

2.1.4 Epistemic (Model) Uncertainty

We will now briefly explore the theory mostly of the incorporated Epistemic Uncertainty methods below. Summarizing on the previous discussion in 2.1.2. Epistemic Uncertainty, also known as model uncertainty, typically arises from limited knowledge about model parameters. As discussed this limitation was shown to often be due to insufficient or unrepresentative training data or inadequate training. In some cases, it may also stem from the inherent complexity of the data or the architecture, which can result in a complicated

loss landscape or the shape of local minima. (Li et al., 2018)

The loss landscape can affect the sensitivity of the results to small parameter perturbations and the overall measured epistemic uncertainty of the method. Furthermore, different methods may respond differently to these functional aspects, as will be discussed at the end of the Section 2.1.5.

Unlike aleatoric uncertainty, epistemic uncertainty can, in principle, be reduced by collecting more data or improving the model. 2.1.2

Deep Ensemble methods

Ensemble methods provide a practical alternative for epistemic uncertainty estimation. By training multiple models with different architectures, hyperparameters, weight initializations, or data splits, prediction diversity estimates uncertainty. Ensemble members explore different regions of the hypothesis space, as illustrated in Figure 2.6.

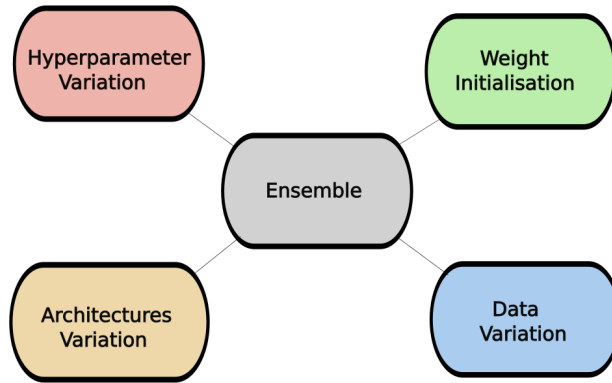


Figure 2.6: Four main sources of variation in deep ensembles are usually divided (He et al., 2023): **Hyperparameter Variation** (different learning rates, regularization strengths, or architectural hyperparameters), **Weight Initialisation** (varying weight initialization values leading to different local minima), **Data Variation** (different data splits, bootstrapping, or data augmentation strategies), and **Architecture Variation** (different network architectures or layer configurations). These variations ensure ensemble members explore different regions of the hypothesis space, providing complementary predictions that capture epistemic uncertainty through their disagreement.

The ensemble predictive mean and variance are given by (Lakshminarayanan et al., 2017):

$$\mathbb{E}[y^*] \approx \frac{1}{M} \sum_{m=1}^M f_m(x^*) \quad (2.7)$$

$$\text{Var}[y^*] \approx \frac{1}{M} \sum_{m=1}^M (f_m(x^*) - \mathbb{E}[y^*])^2 \quad (2.8)$$

In this context, let M represent the number of ensemble members, while f_m denotes the prediction function of the m -th ensemble member. The term x^* refers to a test input, and y^* is the corresponding target output. The ensemble mean prediction is denoted as $\mathbb{E}[y^*]$, and the ensemble variance, which captures epistemic uncertainty through prediction disagreement, is represented as $\text{Var}[y^*]$.

As visualized in sketch 2.6 above, Ensemble methods can be constructed in various ways, including randomizing the initial weights of each model, varying model architectures, altering hyperparameters, or training each member on different subsets of the data (through techniques like bootstrapping or bagging). Often, random initialization and data shuffling are sufficient to create diversity among ensemble members. It has been demonstrated that epistemic uncertainty quantification can be reliable even with as few as five ensemble members (Lakshminarayanan et al., 2017). The two primary data-based strategies for constructing ensembles, bagging and boosting, are compared in Figure 2.7.

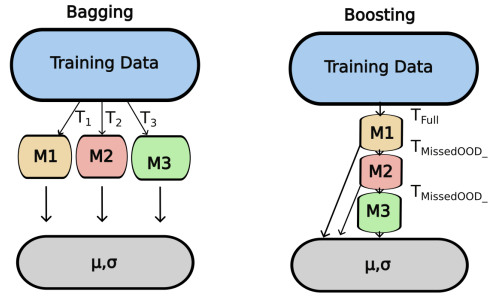


Figure 2.7: Comparison of bagging and boosting ensemble strategies. **Left (Bagging):** Training data is split into independent bootstrapped subsets (T_1, T_2, T_3), and models (M_1, M_2, M_3) are trained in parallel on these subsets. The final prediction (μ, σ) is obtained by aggregating (e.g., averaging) the independent model outputs. **Right (Boosting):** Models are trained sequentially on the full dataset, with each subsequent model (M_2, M_3) focusing on samples that previous models misclassified ($T_MissedOOD_1, T_MissedOOD_2$). The final prediction (μ, σ) is obtained through weighted aggregation, where models trained on harder examples may contribute more.

Bagging involves sampling with replacement from the training data to create separate bootstrapped subsets, on which multiple models are trained in parallel. In contrast, boosting

trains models sequentially and focuses on the samples that previous models misclassified. Various approaches to data manipulation have often been discussed in the literature, suggesting that they may lead to decreased prediction accuracy due to the removal of training data.([He et al., 2023](#); [Gawlikowski et al., 2023](#))

Ensemble methods typically outperform or match Bayesian approaches, especially when dealing with out-of-distribution or shifted data ([Lakshminarayanan et al., 2017](#); [Ovadia et al., 2019](#); [Fort et al., 2019](#)). Ensemble members tend to converge to different local minima, exploring multiple modes in function space, while variational Bayesian methods generally concentrate on a single mode ([Fort et al., 2019](#)). This exploration of multiple modes allows ensembles to capture a broader range of model uncertainty, which is the current explanation for their empirical performance ([Ovadia et al., 2019](#); [Fort et al., 2019](#); [He et al., 2023](#); [Gawlikowski et al., 2023](#)). However, ensemble methods require more computational resources, as multiple models need to be trained and stored. They also have less rigorous theoretical foundations compared to Bayesian methods, and proper calibration may necessitate additional post-processing.([He et al., 2023](#); [Gawlikowski et al., 2023](#))

Bayesian Neural Networks

Bayesian Neural Networks (BNNs) model epistemic uncertainty by representing network weights as probability distributions rather than scalar values, also in bayesian terms called, placing priors (in form of the choosen distribution, mostly gaussian) over the network weights, over the enabling principled Bayesian inference ([Denker et al., 1987](#); [Neal, 1993](#); [He et al., 2023](#)), as visualized in figure 2.8 below.

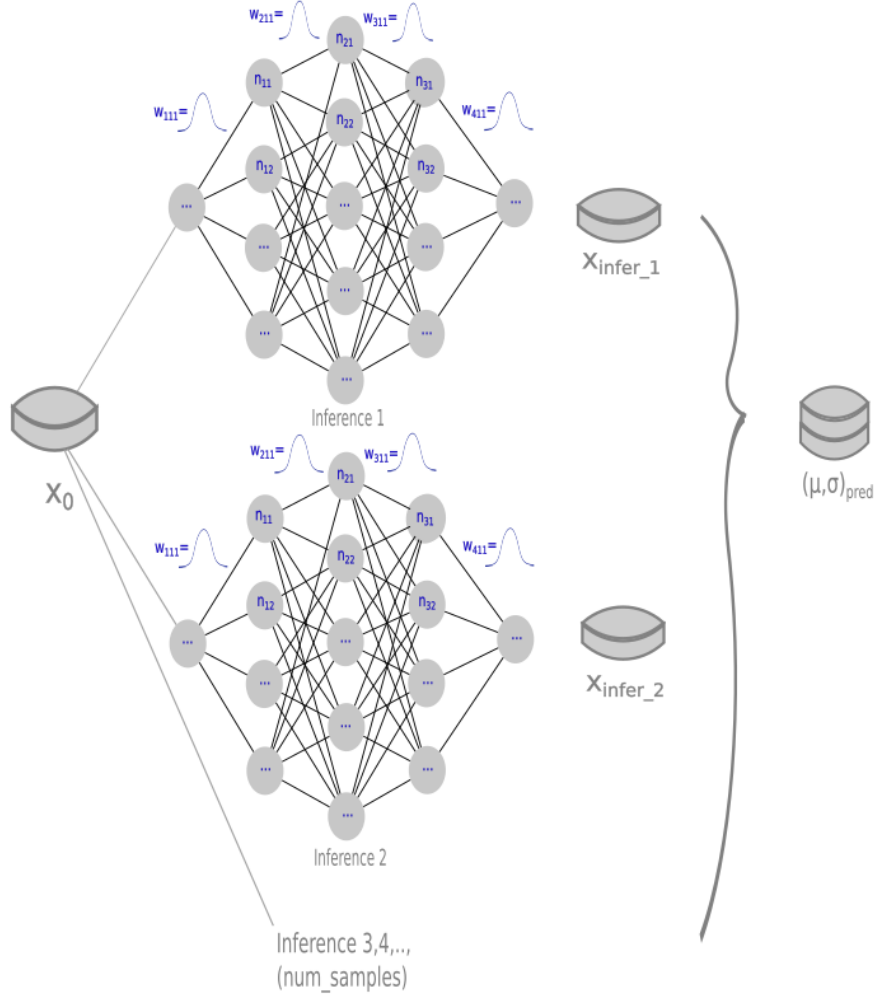


Figure 2.8: Illustration sketch of BNN, the posterior distribution is inferred through placing distributions/priors over the network weights instead of scalar.

A prior distribution is placed over network weights, and the posterior is then inferred over multiple inference passes, using Bayes' rule (Bishop, 2006):

$$p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta) \quad (2.9)$$

where θ represents network weights, $p(\theta)$ is the prior, $p(\mathcal{D}|\theta)$ is the likelihood, and $p(\theta|\mathcal{D})$ is the posterior. The predictive distribution marginalizes over the posterior by sampling weight configurations and averaging predictions. However, exact Bayesian inference is intractable for deep networks, motivating efficient approximations such as Monte Carlo Dropout (Gal and Ghahramani, 2016) and Laplace approximation (discussed in Sections 2.1.4 and 2.1.4), which are used in this work instead of full BNN inference.

Monte Carlo Dropout

Monte Carlo (MC) dropout offers a computationally efficient alternative to full BNNs (Gal and Ghahramani, 2016). During training, dropout layers are added to the network architecture, where neurons are randomly deactivated with probability p to prevent overfitting. During inference, dropout is kept active, and multiple forward passes are performed with different dropout masks (different random neuron deactivations), effectively sampling from an approximate posterior distribution over network configurations. The dropout mechanism scales active neurons by $\frac{1}{1-p}$ to maintain expected activations: (He et al., 2023)

$$h' = \begin{cases} 0, & \text{with probability } p \\ \frac{h}{1-p}, & \text{otherwise} \end{cases} \quad (2.10)$$

where h is the original neuron activation before dropout, h' is the activation after dropout application, and p is the dropout probability. When a neuron is deactivated (with probability p), its output is set to zero, while active neurons are scaled by $\frac{1}{1-p}$ to preserve the expected activation magnitude across training and inference.

This process is illustrated in Figure 2.9, which shows how multiple inference passes with different dropout masks produce a distribution of predictions that can be aggregated to estimate both the mean prediction and epistemic uncertainty. The method gained popularity when (Gal and Ghahramani, 2016) showed mathematically that dropout can be interpreted as approximate Bayesian inference, where the dropout objective minimizes the Kullback–Leibler divergence between an approximate distribution and the posterior of a deep Gaussian process. The dropout Bernoulli variational distribution over weights can thus be seen as a variational approximation to a Bayesian neural network, providing a computationally efficient way to estimate epistemic uncertainty without requiring full Bayesian inference (Gal and Ghahramani, 2016).

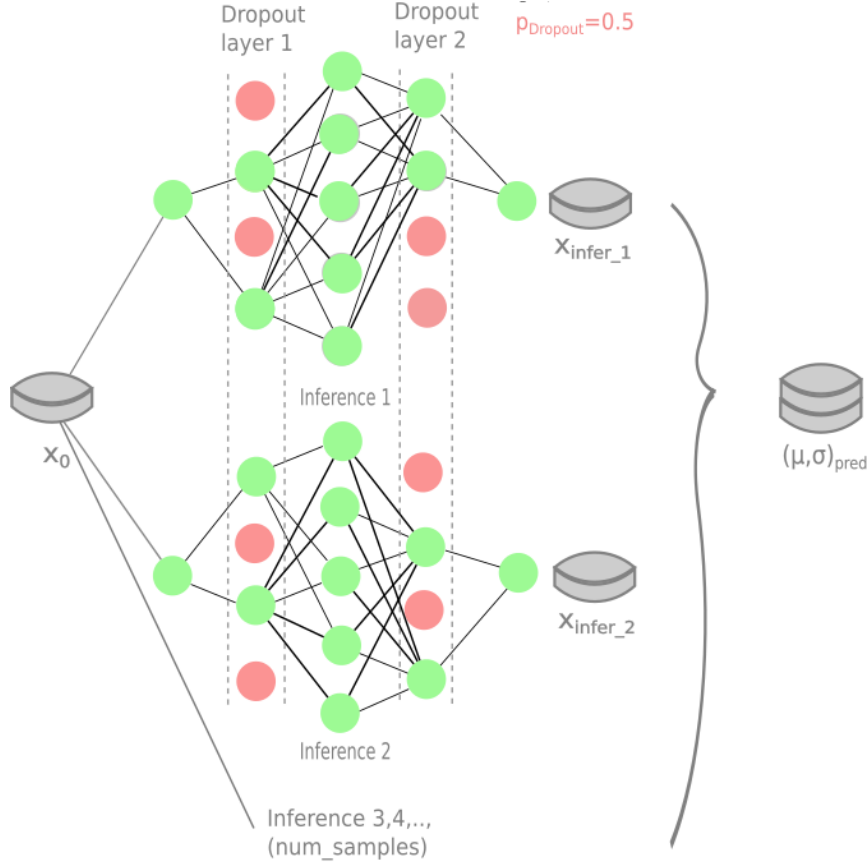


Figure 2.9: Monte Carlo Dropout inference pipeline for uncertainty quantification. Input x_0 is processed through multiple inference passes (Inference 1, 2, ..., N), where dropout layers are kept active during inference. Each inference pass uses a different dropout mask, randomly deactivating neurons (shown in red) with probability p_{dropout} while scaling active neurons (shown in green) by $\frac{1}{1-p}$. This creates different network configurations for each pass, producing varied predictions. The final prediction and uncertainty are obtained by aggregating these predictions: the mean and standard deviation across passes. This process approximates sampling from the posterior distribution over network weights, enabling epistemic uncertainty estimation through the variance of predictions across different dropout masks.

Dropout layers were originally developed for regularization, reducing overfitting by forcing neurons to learn generally helpful features across diverse contexts (Hinton et al., 2012). MC dropout benefits include minimal architectural changes (only adding dropout layers) compared to evidential deep learning, and requiring only one model compared to ensembles. However, multiple forward passes are needed at inference, and MC dropout tends to provide less calibrated uncertainty estimates than ensembles or direct Bayesian approaches, likely due to different loss landscape variance inference mechanisms. Additionally, dropout layer

addition can negatively influence prediction accuracy by changing the loss landscape, affecting convergence minimum and time, with neuron scaling $\frac{1}{1-p}$ further complicating convergence (Gal and Ghahramani, 2016). MC dropout offers a practical trade-off: scalable but approximate uncertainty quantification, capturing epistemic uncertainty through variability in network parameters.

Gaussian Process Hybrid Neural Networks

A Gaussian process (GP) is a stochastic process where any finite collection of random variables follows a multivariate Gaussian distribution, defining and placing a prior, therefore being a Bayesian method as well, but instead of placing over network weights, as for BNNs, over functions $f : \mathcal{X} \rightarrow \mathbb{R}$ (Rasmussen and Williams, 2006):

$$p(y_1, \dots, y_n) = \mathcal{N}(\mu(x), \Sigma(x)),$$

where $\mu(x)$ is the mean function vector, $\Sigma(x)$ is the $n \times n$ covariance matrix with entries $\Sigma_{ij} = \kappa(x_i, x_j)$ from positive definite kernel κ measuring similarity and controlling smoothness (Rasmussen and Williams, 2006). Kernel/prior choice determines GP properties: RBF for smooth stationary processes, Matern for rougher behavior, linear for linear relationships (Rasmussen and Williams, 2006). For deep kernel learning, kernels operate on learned latent representations $g(x_i; w)$ rather than raw inputs, enabling complex non-linear relationships through neural network feature learning combined with GP uncertainty quantification. Detailed kernel formulations are provided in (Appendix 5.3).

Given training data $\mathcal{D}_{tr} = \{(x_i, y_i)\}$, the posterior predictive distribution at a new test point x^* is Gaussian (Rasmussen and Williams, 2006):

$$p(y^* | x^*, \mathcal{D}_{tr}) = \mathcal{N} \left(K_x^\top K_n^{-1} y, K_{x^*} - K_x^\top K_n^{-1} K_x \right),$$

where K_n is the $n \times n$ covariance matrix with entries $[K_n]_{ij} = \kappa(x_i, x_j)$, K_x is the $n \times 1$ vector with entries $[K_x]_i = \kappa(x^*, x_i)$, $K_{x^*} = \kappa(x^*, x^*)$ is the prior variance, and y is the training target vector. The mean prediction is $K_x^\top K_n^{-1} y$, and the variance $K_{x^*} - K_x^\top K_n^{-1} K_x$ captures epistemic uncertainty, producing lower uncertainty in regions dense with training data and higher uncertainty where samples are sparse, as visualized in Figure 2.10 (Rasmussen and Williams, 2006; He et al., 2023).

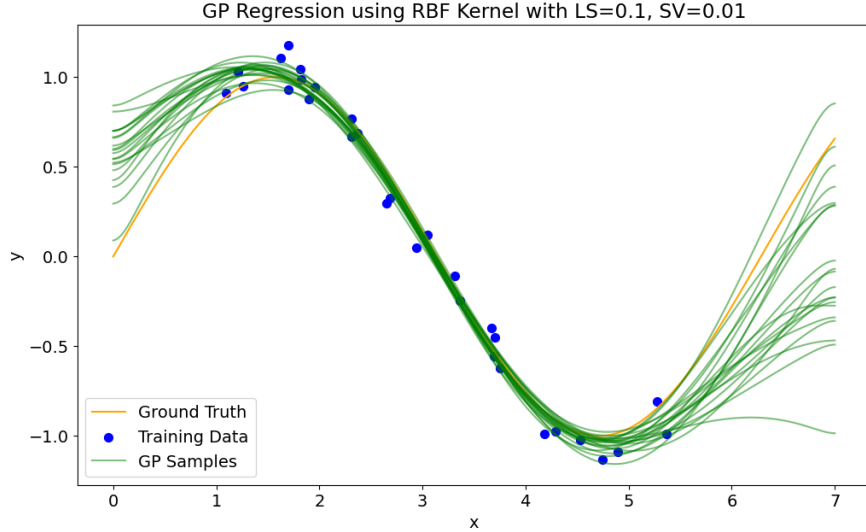


Figure 2.10: Gaussian Process regression with RBF kernel demonstrating uncertainty quantification. The ground truth function (orange line) represents the underlying true relationship, while training data points (blue dots) show noisy observations. Multiple GP samples (green lines) drawn from the kernel/prior fitted posterior distribution illustrate the model’s uncertainty. In regions with dense training data (e.g., $x \in [2, 4]$), the GP samples are tightly clustered, indicating low epistemic uncertainty. In regions with sparse or no training data (e.g., $x < 1$ or $x > 5.5$), the GP samples diverge significantly, reflecting increased epistemic uncertainty. This demonstrates the fundamental property of GPs: uncertainty naturally increases with distance from training data, making them well-suited for detecting out-of-distribution samples and quantifying model confidence. The kernel hyperparameters (length scale $LS=0.1$, signal variance $SV=0.01$) control the smoothness and amplitude of the function, influencing both the mean prediction and uncertainty estimates.

Since inversion of K_n scales cubically with the number of samples n , often the number of samples over which the variance is taken has to be reduced, furthermore for very large datasets, sparse Gaussian processes can be used, e.g. diagonal approximations further reduce complexity by approximating the full covariance matrix with its diagonal elements. Detailed mathematical formulations for the example of diagonal sparse GP method, are provided in [\(Appendix 5.3\)](#).

The choice of the prior/kernel function fundamentally determines GP properties, controlling smoothness and function characteristics. Common kernels include RBF (infinitely differentiable, smooth processes), Matern (rougher behavior, controlled by smoothness parameter ν), and linear (equivalent to Bayesian linear regression) ([Rasmussen and Williams, 2006](#)). For deep kernel learning, kernels operate on learned latent representations $g(x_i; w)$ rather

than raw inputs, enabling complex non-linear relationships through neural network feature learning combined with GP uncertainty quantification. Kernel hyperparameters are optimized via maximum likelihood estimation, balancing data fit with model complexity. Detailed kernel formulations are provided in (Appendix 5.3).

For actually fitting the Gaussian processes, besides the used kernels, there can also be multiple ways in which the processes are fitted and how many are fitted, two example approaches, often highlighted in literature are deep kernel learning and deep gaussian processes, while deep as illustrated in Figure 2.11 below.

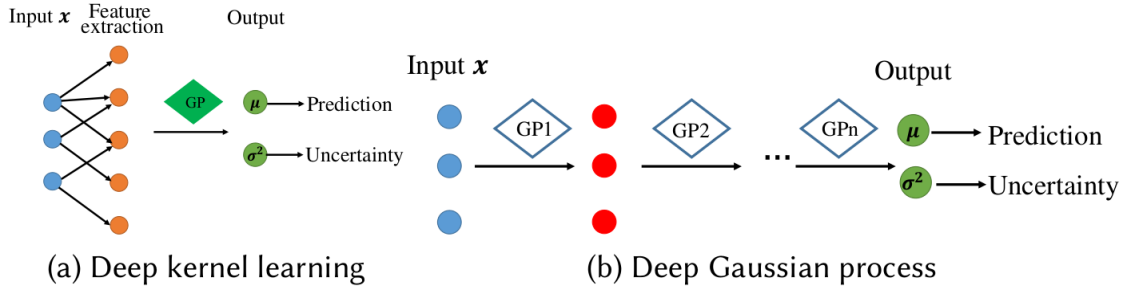


Figure 2.11: Sketch by (He et al., 2023), showing two different architectural approaches to fit Gaussian processes. Deep kernel learning (Left(a)), where the input $\mathbf{x}$ is after processing and first feature extraction directly fed into a single Gaussian Process that performs regression sampling and therefore uncertainty quantification on it, outputting both prediction (μ) and uncertainty (σ^2). And Deep Gaussian process (Right(b)), where multiple GP layers (GP1, GP2, ..., GPn) are stacked sequentially, where each layer transforms the representation from the previous layer, inspired from deep neural networks, accumulating and refining more the prediction (μ) and uncertainty (σ^2) predictions.

Deep kernel learning (see Figure 2.11a) is a straightforward approach where, after feature extraction, a single Gaussian Process (GP) operates on these learned features using one kernel, $K_\theta(g(x_i; w), g(x_j; w))$. This method approximates uncertainty and predictions based on the samples. In contrast, Deep Gaussian Processes (Figure 2.11b), inspired by neural networks, involve multiple stacked GP layers. Each layer transforms the representation and propagates uncertainty to the next layer, creating hierarchical compositions that enhance expressiveness and uncertainty propagation (Damianou and Lawrence, 2013; He et al., 2023).

In recent years, several more advanced approaches have emerged, such as spectral normalized GPs (SNGPs) (Liu et al., 2022). While GPs provide sound theoretical uncertainty quantification that takes into account sample density and distance, their reliance on Gaussian distribution assumptions and fixed kernels can limit their modeling capacity, especially in scenarios involving high-dimensional, structured data.

Laplace Approximation

The Laplace approximation offers a practical post hoc method to estimate epistemic uncertainty by approximating the posterior distribution over neural network weights as a Gaussian centered at the maximum a posteriori (MAP) estimate.

Formally, the posterior is approximated by (Bishop, 2006):

$$p(\theta|\mathbf{X}, \mathbf{Y}) \approx \mathcal{N}(\hat{\theta}, \mathbf{H}^{-1}) \quad (2.11)$$

where θ represents the network weights, $\mathbf{X}$ and $\mathbf{Y}$ are the training inputs and targets, $\hat{\theta}$ is the maximum a posteriori (MAP) estimate of the weights (the mode of the posterior), $\mathbf{H}$ is the Hessian matrix of the negative log posterior evaluated at $\hat{\theta}$, and $\mathbf{H}^{-1}$ is the inverse Hessian that determines the covariance of the Gaussian approximation (Bishop, 2006). The Hessian matrix entries are given by:

$$\mathbf{H}_{ij} = -\frac{\partial^2}{\partial\theta_i\partial\theta_j} \log p(\mathbf{Y}|\mathbf{X}, \theta) \quad (2.12)$$

where θ_i and θ_j are individual weight parameters, and the Hessian captures the local curvature of the log-posterior around the MAP estimate, with larger curvature (larger Hessian eigenvalues) indicating lower uncertainty.

This local Gaussian approximation captures the curvature around the MAP mode, enabling uncertainty quantification by considering local variations in parameter space, as illustrated in Figure 2.12 below.

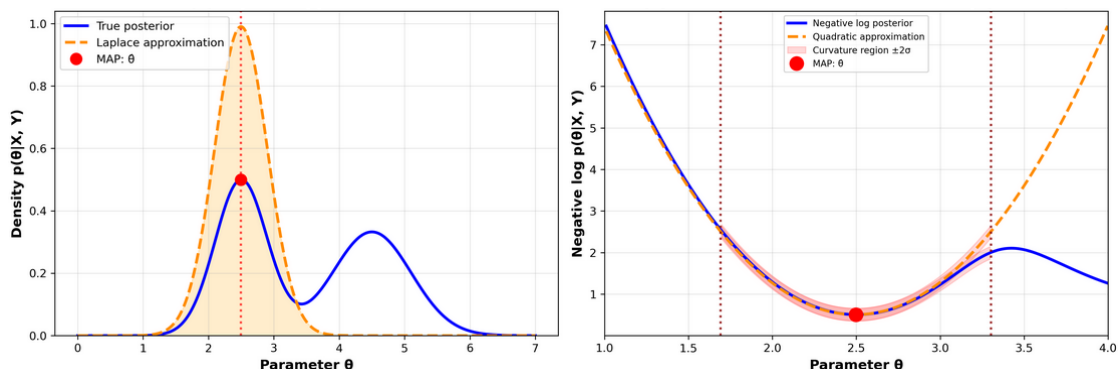


Figure 2.12: Example sketches of a Laplace approximation behaviour of a posterior distribution local minimum. In the plot we can see how the true posterior (Left) and the zoomed-in negative log posterior (right) $p(\theta|X, Y)$ (blue solid line) exhibits a bimodal distribution, but the Laplace approximation (orange dashed line) is a unimodal Gaussian centered at the MAP estimate, matching the mode and curvature of the first peak. The approximation captures the local structure around the MAP but fails to represent the second mode that could still be accessed under small perturbations, demonstrating one of the limitations of Laplace approximation even locally for multi-modal distributions. (Bishop, 2006).

The Laplace approximation matches the mode and curvature (Hessian) of the true posterior at the MAP estimate as gaussian approximated capturing local epistemic uncertainty (Bishop, 2006), but as is already visible, the approximation can even for the local minimum leave out complexity due to the gaussian approximation needed.

Unlike variational inference, Laplace approximation does not require modifying the training process and can be applied post training, which is a benefit regarding the prediction accuracy and not needed retraining and refine-tuning. However, computing and inverting the full Hessian for modern deep networks with millions of parameters is computationally costly to unfeasible. Therefore sparse Laplace approximations address this by using structured approximations of the Hessian matrix, as well as reducing the amount of parameters/dimensions included from the gradients, with studies demonstrating that even less than 1% of parameters can be sufficient to capture the loss landscape curvature (Daxberger et al., 2021). Detailed mathematical formulations of sparse Laplace methods are provided in (Appendix 5.3).

While Laplace approximation simplifies posterior inference, as already stated it is only feasible with a lot additional approximations, inherent gaussian approximation, single-modality capturing etc. Despite these limitations, Laplace approximation can remain a valuable tool for epistemic uncertainty estimation, especially if the underlying model architecture is complex and has a complex loss landscape, that is hard to train/converge, providing a

computationally efficient alternative to fully Bayesian methods in some settings applicable post hoc to pre-trained models.

Evidential Deep Learning

Evidential deep learning (EDL) is a newer framework providing a more unified form of quantifying both aleatoric and epistemic uncertainty in one forward pass by learning a higher-order evidential distribution over the predictive likelihood parameters (Sensoy et al., 2018; Amini et al., 2020). In regression, the network predicts hyperparameters $(\gamma, v, \alpha, \beta)$ of a Normal-Inverse-Gamma (NIG) distribution placed over the Gaussian likelihood parameters (μ, σ^2) , where γ is the predicted mean, v controls confidence in the mean, and α, β parameterize uncertainty over the noise variance (Amini et al., 2020).

The loss function combines data fit with evidence regularization (Amini et al., 2020):

$$\mathcal{L} = \text{NLL}(y|\gamma, v, \alpha, \beta) + \lambda|y - \gamma|(2v + \alpha),$$

where the NLL term balances prediction accuracy with uncertainty calibration, and the regularizer $\lambda|y - \gamma|(2v + \alpha)$ penalizes overconfident incorrect predictions, promoting well-calibrated uncertainty that increases when predictions are inaccurate (Amini et al., 2020).

The uncertainty is then decomposed from the learned NIG distribution output, into aleatoric uncertainty through $\mathbb{E}[\sigma^2] = \frac{\beta}{\alpha-1}$ (data noise) and into epistemic uncertainty through $\text{Var}[\mu] = \frac{\beta}{v(\alpha-1)}$ (model confidence), where $\alpha > 1$ is required. Epistemic uncertainty is therefore decreasing with larger v ($/\sigma^2$ more evidence) or smaller β , while aleatoric uncertainty increases with larger β or smaller α (Amini et al., 2020), as is visualized nicely in the hierarchical structure by Amini et. al. in Figure 2.13 below.

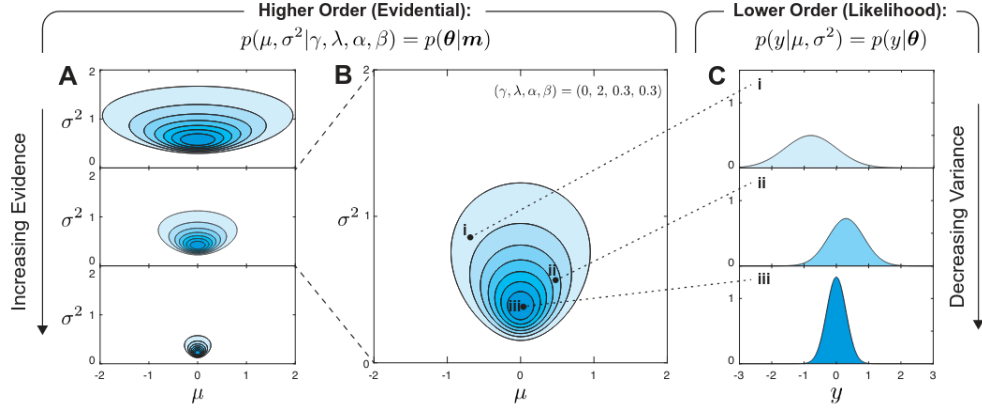


Figure 2.13: Hierarchical uncertainty behaviour in evidential deep learning sketch by sketch by (Amini et al., 2020), distinguishing between higher-order (epistemic) and lower-order (aleatoric) uncertainty. (Panels A & B) Higher order evidential: The evidential distribution $p(\mu, \sigma^2 | \gamma, \lambda, \alpha, \beta) = p(\theta | m)$ over the likelihood’s mean and variance. As evidence increases (Panel A, bottom to top), the distribution becomes more concentrated, reflecting reduced epistemic uncertainty. (Panel C) Lower order likelihood: The predictive likelihood $p(y | \mu, \sigma^2) = p(y | \theta)$ for different σ^2 values. As σ^2 decreases (top to bottom), the likelihood becomes sharper, indicating reduced aleatoric uncertainty.

The parameter ν , β and α offsets can be further fine-tuned.

The main advantage of EDL is its inference time, capturing both uncertainty types in a single deterministic model, avoiding costly multiple forward passes and also the strong combination of directly learning uncertainty by evidence correlation, as well as penalizing high prediction at the same time for the uncertainty.

However, reliance on strong conjugate prior assumptions can reduce flexibility, and EDL requires modifications to standard training procedures and careful tuning of regularization hyperparameters (Sensoy et al., 2018; Amini et al., 2020).

2.1.5 Method Selection and Expected Performance

Aleatoric methods

For the aleatoric methods, for each of the two discriminative approaches, parametric and non-parametric, two methods were chosen, along with a newer discriminative parametric method that works indirectly through evidence learning: evidential deep learning (Section 2.1.4), which combines both aleatoric and epistemic uncertainty quantification.

In the two parametric examples, the data variance is typically attributed to experimentally measurable Gaussian noise. Therefore, two Gaussian-enforcing discriminative losses were incorporated. The first is the standard Negative Log-Likelihood (NLL) (Section 2.1.3), which

enforces a Gaussian distribution and introduces penalization. The second is the Gaussian Mixture Model (Section 2.1.3), which is more complex as it enforces a combination of multiple Gaussian distributions to capture multimodal uncertainty.

For the non-parametric approach, a basic quantile approximation using quantiles of (5, 95) was selected (Section 2.1.3), along with the PICP (Prediction Interval Coverage Probability) loss (Section 2.1.3). The PICP loss is more complicated and may influence predictions significantly, but it could provide better coverage-to-width trade-offs. Both non-parametric methods are expected to perform slightly worse in capturing the noise distribution compared to parametric methods, potentially affecting prediction accuracy due to their increased complexity, though they may offer benefits in scenarios where the data distribution is not well-defined.

Lastly, the newer Evidential method works indirectly through evidence collection learning and additional prediction accuracy penalization of the evidence, as described in (Section 2.1.4). This method provides both aleatoric and epistemic uncertainty and is the fastest among all methods. However, it has the downside of employing the most complicated loss function, which could affect prediction accuracy and the convergence of training significantly.

Generative methods were also discussed theoretically (Section 2.1.3). In the literature, they are primarily categorized under aleatoric estimation since they learn the underlying distribution. However, due to constraints of scope, complexity, run-time, and convergence trade-offs, these methods will not be compared in this work. They were mentioned in the theory section and strategically excluded here because there is an ongoing doctoral project assessing several other architectures, including generative methods, to determine their capability of replacing the current AE-RNN architecture. Therefore, they are likely to be discussed in future work, as noted in the conclusion section (Section 5.3).

Epistemic methods

As discussed in the theory section, two particularly robust and theoretically rigid approaches are the full Bayesian Neural Network (BNN) and deep ensembles. Deep ensembles have repeatedly demonstrated their robustness in literature, particularly in detecting high errors and out-of-distribution (OOD) correlations through the increased variance of the ensemble members. Therefore, in this thesis, they will primarily serve as a baseline for comparing other methods.

Specifically randomly initialized starting weights will be used because they offer advantages over variations in data, architecture, and training hyperparameters. Random initialization can prevent these variations from influencing the best-found configuration or the amount of training data, thereby, while still possible changing the convergence minimum, having less negative impact on prediction accuracy (Section 2.1.4).

While the full BNN is mostly computationally intractable, many approximations exist. However, both deep ensembles and most approximated BNNs incur high computational

costs, which can be problematic, especially for our case, given the data size; hence, the inference and computational cost will be significant. Although latent space translation reduces computational expenses, this remains a concern. Therefore, while full deep ensembles or fuller BNN approximations are shown to be the most robust, they will be tested against other methods that may reduce training and inference costs.

One well-known method frequently referenced in literature for its comparatively faster inference time and ease of implementation is Monte Carlo Dropout (MCDropout) (Section 2.1.4). It will be included due to its theoretically proven connection to BNNs (Gal and Ghahramani, 2016).

Another commonly used method for uncertainty quantification is Gaussian Processes (GP). While it is also a Bayesian method, GP operates differently from BNNs and their approximations. Instead of placing a prior over the weights, it measures data similarity in the latent space and applies a kernel function to vary the posterior output. GP is besides MCDropout one of the most used uncertainty quantification methods in literature, and recent literature regarding it has more increased than decreased with additional variations. Besides previously in literature demonstrating effectiveness in OOD detection, its unique direct distance correlation and explainability is helpful, it will therefore be included in the comparison. The basic GP method is described in (Section 2.1.4), while sparse GP approximations for computational efficiency are detailed in (Appendix 5.3).

Another method worth mentioning is the Laplace method, which is an example of a post-hoc method, which does not directly impact the architecture. Although this method requires several approximations to be computational feasible, because of the hessian matrix and high-dimensional gradients, as discussed in prior literature, it has shown potential even for high parameter reduction and sparse approximations, with studies demonstrating that less than 1% of parameters can be sufficient to capture the loss landscape curvature (Daxberger et al., 2021). Detailed mathematical formulations for sparse Laplace approximations are provided in (Appendix 5.3), while the basic method is described in (Section 2.1.4).

Finally, evidential deep learning (Section 2.1.4), one of the newer methods, which was constructed to predict aleatoric and epistemic uncertainty simultaneously, will be implemented and discussed. The method works indirectly, designed to discriminatively learn a Normal-Inverse-Gamma (NIG) distribution through the learning of evidence. It approximates the uncertainty of the distribution and model through learning evidence estimates, while penalizing the evidence based on accuracy, thus providing a strong combination of directly learning uncertainty and making it dependent on accuracy.

Despite the complexities of its loss function, which could affect convergence, prediction accuracy, and the explainability of the learned evidence, evidential deep learning possesses intriguing properties. It has shown capabilities in rapidly determining both aleatoric and epistemic uncertainty quantification.

Besides the expected influences on prediction accuracy and computational cost (He et al., 2023; Gawlikowski et al., 2023), there are anticipated differences in how each method

captures uncertainty quantification, particularly in relation to loss landscape coverage. The interaction of loss landscapes with architecture and datasets has been increasingly discussed since around late 2010 due to computational costs (Li et al., 2018; Fort et al., 2019). The coverage of various uncertainty methods has also been explored, with one of the most notable studies (Fort et al., 2019) focusing on the differences in loss landscape coverage between Bayesian methods and deep ensembles. These differences are illustrated in Figure 2.14, which shows an approximation of possible intersections within the high-dimensional loss landscape.

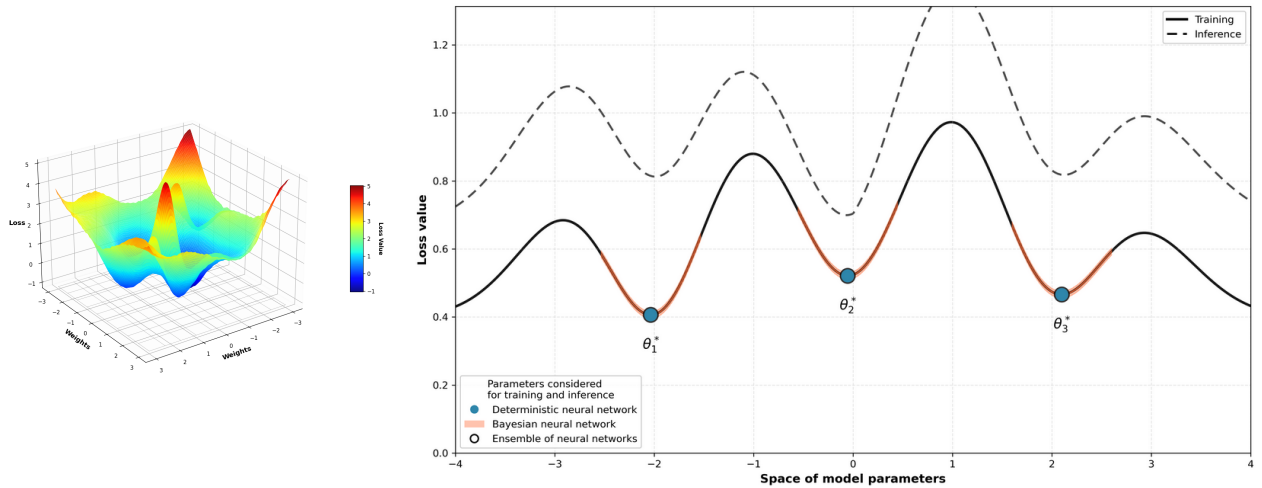


Figure 2.14: Loss landscape intersection sketch (Left). Visualization of how different uncertainty quantification methods explore the loss landscape from a intersection, adapted from (Fort et al., 2019) (Right).

Deterministic neural networks (blue dots) converge to point estimates at local minima, while Bayesian neural networks (orange shaded regions) explore distributions to capture epistemic uncertainty. Deep ensembles (purple squares) train multiple models that converge to different local minima, whereas variational Bayesian methods tend to focus more on a single local minimum (red marked area) (Fort et al., 2019).

As previously discussed, the perturbations caused by the prior over the weights generally induce small fluctuations around the local deterministic Maximum A Posteriori (MAP) convergence point in the local minimum, often preventing movement away from this local converged minimum. In contrast, deep ensembles, with various random initialization starting points, often converge to different minima in the global loss landscape, thereby capturing more of the inherent model’s epistemic uncertainty rather than solely the influence of small parameter perturbations.

Regarding Bayesian methods, both BNNs and approximations like MCDropout fall under this category. Moreover, while GP also constitutes a Bayesian method, its relationship with the local loss landscape minima is less evident, especially when considering data similarity within a single dataset. The Laplace method, which directly approximates local minimum curvature, can also be associated with its influence on local coverage, similar to BNN methods. As for evidential deep learning, it also approximates a distribution in a bayesian way but does in a more indirect and harder to directly the loss landscape connectable way.

2.2 Flow Scenario Data: Two Idealized Flows

At least two directly simulated flows are utilized to create training and testing datasets for out-of-distribution (OOD) detection. The chosen flows are Couette flow and the Kármán Vortex Street (KVS). These flows have been already developed for the selected solver within the simulation model (refer to Section 2.3).

The two flow scenarios demonstrate fundamental differences and are pertinent to atmospheric processes, as discussed in the following sections 2.2.1 and 2.2.2.

The complementary characteristics of these flows make them appropriate for testing whether uncertainty quantification methods (see Section 2.1) can effectively differentiate between fundamentally distinct flow regimes. This distinction is crucial for identifying potential erroneous predictions, which is essential for reliable deployment, particularly when one flow scenario is underrepresented in the training data. Such underrepresentation can negatively impact performance in predicting atmospheric flows and related quantities, as highlighted in the meteorological relevance sections 2.2.1 and 2.2.2.

After introducing the flow scenarios, the data extraction process will be determined based on the calculated flow characteristics of each case. This process aims to maximize the information content while minimizing redundancy (refer to Section 2.2.3).

2.2.1 Plane Couette Flow

Plane Couette flow describes the motion of a viscous fluid between two parallel plates, where at least one plate moves tangentially relative to the other, generating flow through shear rather than pressure gradients (Couette, 1890; White, 2011). The classic problem, studied since (Couette, 1890), provides a fundamental framework for understanding shear-driven flows. In its most idealized form, the flow between plates at $y = 0$ and $y = h$ is assumed to be laminar, incompressible, and fully developed, leading to the linear velocity profile (White, 2011; Batchelor, 2000):

$$u(y) = U \frac{y}{h} \quad (2.13)$$

where U is the velocity of the moving plate, h is the distance between the plates, and the fluid next to each plate adopts the plate's velocity through the no-slip boundary condition

(White, 2011).

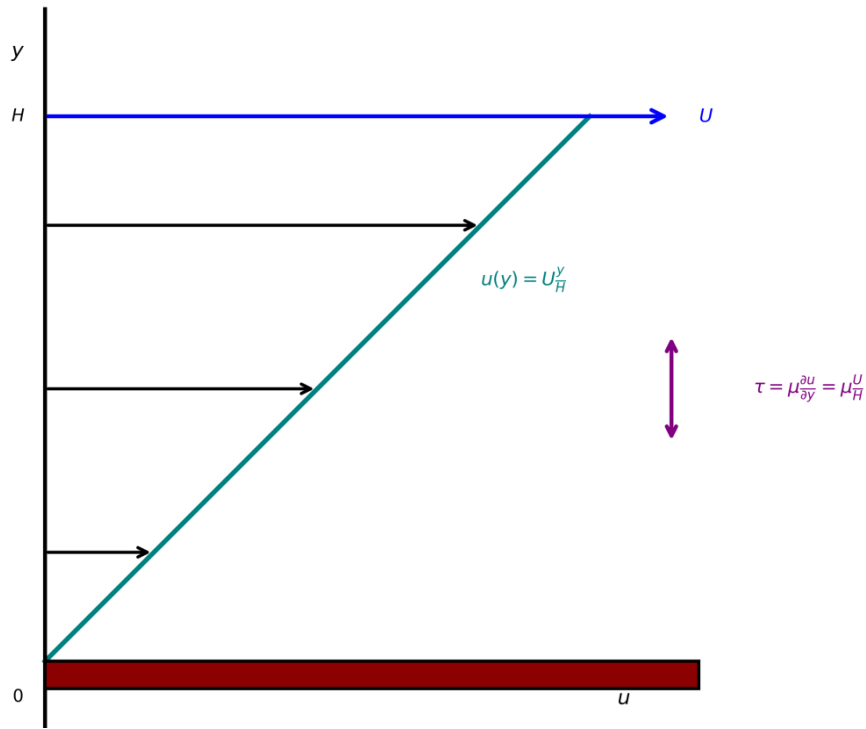


Figure 2.15: Basic sketch of the theoretical Couette flow profile.

Meteorological Relevance

Wind profiles near the ground often display Couette-like linear behavior in the surface layer (SL), the lowest part of the atmospheric boundary layer (ABL), where the ground acts as a stationary boundary and moving air above creates shear flow. Under stable nocturnal conditions, this linear behavior is enhanced due to suppressed vertical mixing from stable thermal stratification (Mahrt et al., 1998). Tower measurements from sites such as Cabauw (Netherlands) and Lindenberg (Germany) have consistently shown strong agreement with the Couette model under stable, low-turbulence conditions ($R^2 > 0.9$ when the Richardson number is low and the surface is smooth) near the surface. Furthermore the simple Couette flow analogy is utilized in wind energy, air quality modeling.

2.2.2 Kármán Vortex Street (KVS)

Kármán vortex streets (KVSs) are characterized by the periodic shedding of alternating, counter-rotating vortices in the wake of a bluff body in steady flow. In the early 1900s, Von Kármán (1911–1912) established stability conditions that show the spacing ratio between the two rows of vortices determines the stability of the vortex street (von Kármán, Theodor, 1911, 1912). This pattern is influenced by inertial and viscous forces, which are quantified

by the Reynolds number and the Strouhal number.

As fluid approaches the object, it splits and flows around both sides, with flow separation from the object's surface playing a crucial role. After separation occurs, fluid layers roll up due to variations in boundary layer velocity, forming vortices that are shed alternately and rhythmically from either side. This process creates a distinct pattern downstream ([Chopra and Hubert, 1965](#)).

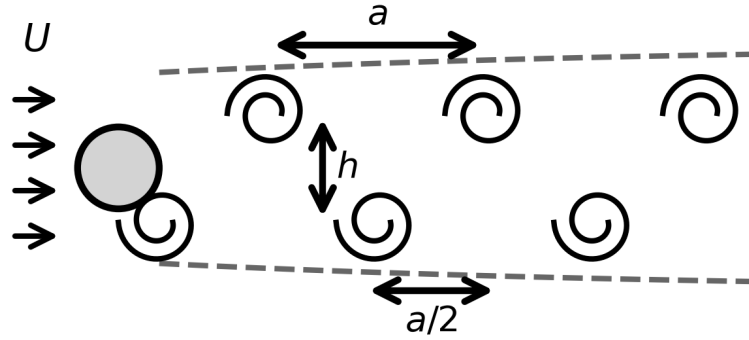


Figure 2.16: Theoretical Kármán Vortex Street for building wake scenario. Left: Flow field showing vortex shedding pattern downstream of building obstacle, with MD box positioned in the wake region. Right: Velocity deficit along the centerline and relevant flow parameters including Reynolds and Strouhal numbers.

The Strouhal number $St = \frac{fD}{U}$ relates the shedding frequency f to the flow speed U and the obstacle size D , providing a scaling law across different Reynolds numbers ([Roshko, 1954](#)). The onset and stability of the vortex street are governed by the Reynolds number $Re = \frac{UD}{\nu}$, which expresses the competition between inertial and viscous forces ([White, 2011](#); [Batchelor, 2000](#)). When the Reynolds number exceeds a critical threshold (typically $Re > 40$), instabilities can develop, leading to the formation of a regular vortex street ([Williamson, 1989](#)).

Meteorological Relevance of Atmospheric Kármán Vortex Streets

In the atmosphere, Kármán vortex streets form when steady winds encounter isolated obstacles such as mountains, islands, or tall buildings, creating a double row of counter-rotating vortices downstream that can be visible from space ([Horváth and Deneke, 2020](#)). These atmospheric vortex streets (AVSs) influence cloud formation, wind patterns, and precipitation. Formation requires specific conditions: a well-mixed boundary layer capped by a strong temperature inversion, steady upstream winds, Reynolds number $Re > 40$, and Froude number $Fr < 0.4$ (where $Fr = U/(N H)$ characterizes inertial vs. buoyancy forces) ([Horváth and Deneke, 2020](#)). When $Fr < 0.4$, the flow is subcritical, allowing stable vortex streets, whereas higher Froude numbers can lead to wave-like disturbances that disrupt the organized vortex pattern ([Horváth and Deneke, 2020](#)).

Atmospheric Kármán vortex streets have been observed in the wakes of isolated islands and mountains. Notable examples include Guadalupe Island off Baja California, where vortex streets are visible in satellite imagery, and the Tibetan Plateau, which produces large-scale AVSs affecting downstream precipitation patterns (Horváth and Deneke, 2020). High-resolution satellite wind data from GOES-16 and MODIS have enabled detailed analyses of these mesoscale vortex streets, revealing their evolution, shedding frequencies, and asymmetric vorticity decay patterns (Horváth and Deneke, 2020).

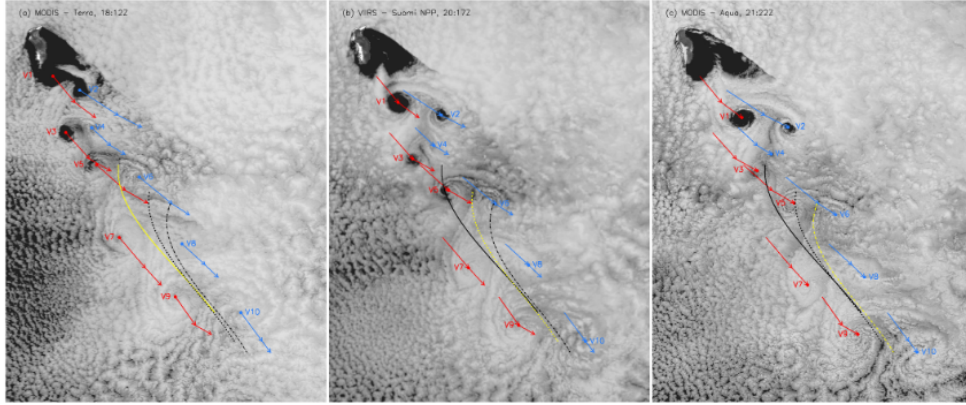


Figure 2.17: Satellite view of a Kármán vortex street in the stratocumulus-capped wake of Guadalupe Island (9 May 2018). (Horváth and Deneke, 2020) Cyclonic and anticyclonic eddies are visible as alternating cloud swirls downstream of the island.

In urban environments, building wakes can exhibit periodic vortex shedding patterns, especially for isolated tall structures under steady wind conditions (Oke, 1988; Blocken et al., 2012). Street canyon geometry and building height-to-width ratios influence recirculating vortices, affecting pollutant dispersion, ventilation rates, and pedestrian-level wind conditions (Stathopoulos, 2006; Li et al., 2020). (Blocken et al., 2012) showed that wake flows can create wind speed fluctuations at the pedestrian level, potentially exceeding comfort thresholds and compromising pedestrian safety. (Stathopoulos, 2006) found that wake flows may trap pollutants in recirculation zones, creating localized air quality hotspots, while (Li et al., 2020) demonstrated how vortex dynamics affect ventilation rates and heat flux removal, with variations in flow conditions resulting in different urban microclimate outcomes.

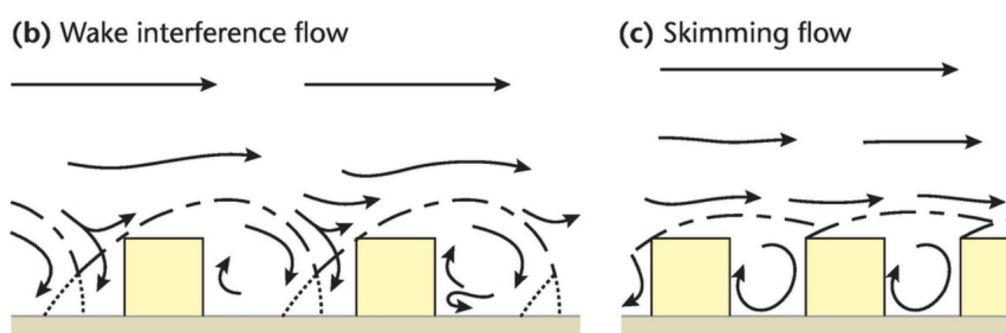


Figure 2.18: Sketch by (Oke et al., 2017) of different mean flow to urban canyon flow connection modes. Wake interference (left), where the urban canyon flow is impacted from the main flow, and skimming flow (right), which is more cut off from the upper main flow. The connection mode is usually analysed in dependency to the obstacle e.g. building distance.

These examples demonstrate that beyond urban geometry differences, the flow scenario itself can significantly impact pollutant dispersion, ventilation rates, and local wind conditions, as different flow regimes fundamentally alter wake structures and their effects on air quality and pedestrian safety (Stathopoulos, 2006; Li et al., 2020). This illustrates why it is crucial to be notified when a flow scenario differs from the training data, as different flow regimes can possibly alter wake structures, pollutant transport, and local environmental conditions, potentially influence predictions and inadequate safety assessments.

2.2.3 Analysis of Extracted Flow Parameters

To further analyze the simulated flow and its properties, dimensionless flow parameters are derived from localized measurements within the larger molecular dynamics (MD) simulation box. These parameters provide insights into the fundamental physics of the flow regime. For instance, they can help determine whether the Reynolds number is sufficiently low for 2D symmetry to still be applicable based on the obstacle structure. They also aid in distinguishing between Couette-like and KVS-like flow scenarios, which have fundamentally different dynamics, even if their mean velocities or spatial scales appear similar.

KVS Flow Parameters

KVS flow parameters are calculated using the following formulas: the Reynolds number is given by

$$Re = \frac{U_{\infty} D}{\nu} \quad (2.14)$$

(White, 2011; Batchelor, 2000), where U_{∞} is the free-stream velocity (preferably measured from the upstream MD box to avoid wake contamination), $D = 0.1$ m is the diameter of the

obstacle, and $\nu = 1.0 \times 10^{-3} \text{ m}^2/\text{s}$ is the kinematic viscosity. The Strouhal number is defined as

$$St = \frac{fD}{U_\infty} \quad (2.15)$$

(Roshko, 1954; Williamson, 1989), where f is the vortex shedding frequency (in Hz) obtained from the cross-stream velocity component (v) using Fast Fourier Transform (FFT), which is preferred due to the presence of stronger periodic oscillations. The velocity deficit is calculated as

$$\Delta U = \frac{U_\infty - U_{\text{wake}}}{U_\infty} \quad (2.16)$$

(Williamson, 1996), where $U_{\text{wake}} = \langle u \rangle_{x,y,z,t}$ is the spatially and temporally averaged wake velocity.

Velocity Scaling

The MD velocities are converted to physical units (m/s) using a scale factor $\alpha = \frac{U_{\text{physical}}}{U_{\text{MD}}}$, where $U_{\text{physical}} = 2.25 \text{ m/s}$ (as per LB solver configuration) and $U_{\text{MD}} \approx 4.88 \text{ m/s}$ (from MD data), resulting in $\alpha \approx 0.46$. This scaling ensures that the Reynolds and Strouhal numbers are dimensionless and comparable to values found in the literature.

Couette Flow Parameters

Parameters are drawn from configuration files. The Reynolds number is calculated as $Re = \frac{U_{\text{wall}}H}{\nu}$ (White, 2011; Batchelor, 2000), where U_{wall} is the velocity of the wall, H is the channel height, and ν is the kinematic viscosity. The velocity profile in this case is linear:

$$u(z) = U_{\text{wall}} \cdot \frac{z}{H} \quad (2.17)$$

(Couette, 1890; White, 2011).

Extracted Flow Parameters: Results

KVS Flow Parameters:

The KVS flow parameters were extracted from multiple MD box files to provide statistical robustness. Analysis was performed on all available simulation files from the training dataset, and the results are presented as mean $\pm$ standard deviation. The results are listed in the table 2.2 below.

Table 2.2: KVS Flow Parameters from MD Box Data

Parameter	Value	Units
Free stream velocity U_∞	2.2500 ± 0.0000	m/s
Mean velocity U_{mean}	2.0662 ± 0.1664	m/s
Reynolds number Re	225.00 ± 0.00	-
Shedding frequency f	4.3973 ± 0.3977	Hz
Strouhal number St	0.1954 ± 0.0177	-
Velocity deficit ΔU	0.0820 ± 0.0740	-
Vorticity $ \omega $	5.628572 ± 0.930402	s^{-1}

Using the proper scaling from the configuration (velocity scale factor $\alpha = 0.46$ and viscosity $\nu = 1.0 \times 10^{-3} \text{ m}^2/\text{s}$), the corrected Reynolds number is:

$$Re = \frac{U_\infty D}{\nu} = \frac{2.25 \times 0.1}{1.0 \times 10^{-3}} = 225 \quad (2.18)$$

The Reynolds number, $Re \approx 225$, places the flow within the transition regime ($190 < Re < 260$), where vortex shedding transitions from a laminar state to a three-dimensional (3D) state as complex streamwise and transverse vorticity structures develop alongside the primary spanwise vortices (Zhang et al., 2019). This transition regime is characterized by predominantly 2D-like coherent vortex structures, while spanwise instabilities begin to emerge. The calculated Strouhal number of $St \approx 0.244 \pm 0.061$ falls within the expected range of 0.15 to 0.25 for circular cylinders (Roshko, 1954; Williamson, 1989; Horváth and Deneke, 2020; Zhang et al., 2019), and the velocity deficit $\Delta U = 0.082 \pm 0.074$ is within the typical range of 0.05 to 0.15, confirming appropriate dimensionless scaling.

Couette Flow Parameters:

Couette flow parameters were extracted from configuration files corresponding to different wall velocities. The wall velocity U_{wall} varies from 0.5 to 5.0 m/s across seven configurations (0–6), and the channel height is constant at $H = 100 \text{ m}$, as well as the kinematic viscosity $\nu = 2.632106 \frac{\text{m}^2}{\text{s}}$ for all configurations. The parameters are calculated directly from the simulation setup and noted in the table 2.3, below.

Table 2.3: Couette Flow Parameters from Configuration Files

Config	U_{wall} [m/s]	Re [-]	$\frac{du}{dz}$ [s ⁻¹]	τ [Pa]
0	0.5000	19.00	0.005000	0.010700
1	1.0000	37.99	0.010000	0.021400
2	1.5000	56.99	0.015000	0.032100
3	2.0000	75.98	0.020000	0.042800
4	3.0000	113.98	0.030000	0.064200
5	4.0000	151.97	0.040000	0.085600
6	5.0000	189.96	0.050000	0.107000

As Couette flow exhibits a simple linear velocity profile that can be fully characterized by the wall velocity and channel geometry. The MD box placement strategy for Couette flow, which exploits the linear velocity profile to efficiently capture flow characteristics, is discussed in (Section 3.2) (Table 3.3).

Couette flow configurations span $Re \approx 8.7$ to 87.4 (after velocity scaling), all within the laminar regime. The linear relationship between wall velocity and Reynolds number is evident ($Re = U_{\text{wall}}H/\nu$). Plane Couette flow is linearly stable at all Reynolds numbers. Below $Re \approx 325$, flow remains predominantly 2D; 3D instabilities and turbulence can emerge at higher Reynolds numbers when triggered by finite-amplitude disturbances (Tillmark and Alfredsson, 1996; Halcrow et al., 2008). Current configurations are well below this critical value, ensuring laminar, predominantly 2D regime consistent with observed linear velocity profiles.

Summary and Expectancy

As discussed the ability to distinguish between different flow/for flow scenarios too far away from the training data, is important. Various flow types can exhibit distinct structures, mixing rates, momentum transfer mechanisms, and overall behaviors, which can impact predictions e.g. of pollutant dispersion, heat transfer, and wind loading.

For instance, considering the used Couette flow and KVS flow scenarios, two idealized yet meteorologically relevant scenarios as discussed before. In KVS flows, organized vortex structures create localized areas of enhanced mixing and velocity fluctuations, contrasting with the relatively uniform shear present in Couette flows. Furthermore, the periodic nature of vortex shedding in KVS introduces temporal variability, which is not observed in steady Couette flows. As a result, different modeling approaches may be required, and they may influence also other connected variables.

A machine learning model trained exclusively on Couette-like flows might struggle to capture the periodic vortex dynamics present in KVS flows, leading to potential prediction errors that should be flagged through appropriate uncertainty estimates. Conversely, a model trained

on KVS flows may incorrectly overestimate variability when applied to steady Couette-like conditions, which we will investigate here.

Thus, the ability of uncertainty quantification methods to identify these regime differences, as an increased epistemic uncertainty when encountering out-of-distribution flow scenarios or resulting model errors would be very helpful in increase the reliability detection. This capability can help narrow down error oversight, which is important for practical applications and for building trust in machine learning predictions in atmospheric flow applications.

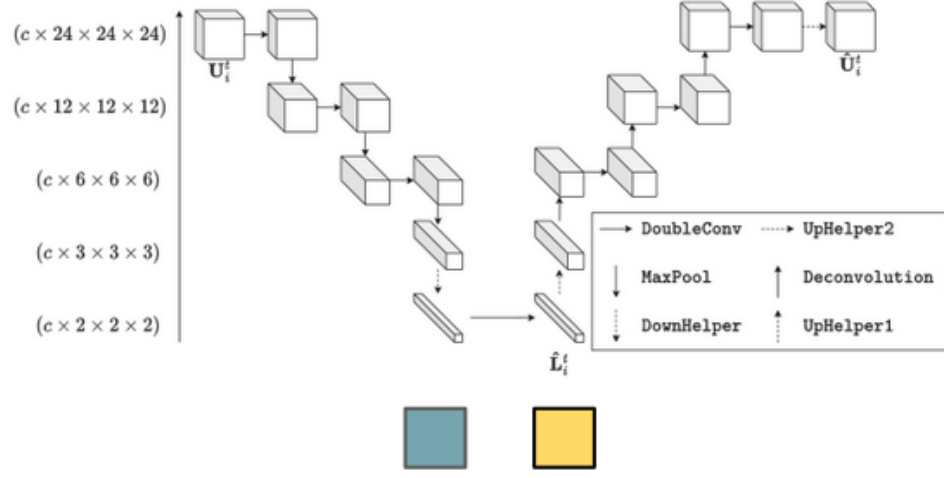
2.3 Infrastructure: Base Network and Flow Data Generation Software”

In this part of the theory section, we will briefly discuss the underlying principles of the neural network architecture used for regressive flow prediction (see Section 2.3.1), which was designed by (Jarmatz et al., 2023). Understanding the underlaying architecture principles is helpful for understanding the results of the uncertainty quantification methods later, particularly regarding the necessity of uncertainty propagation for certain methods. Additionally, we will shortly touch upon the theory behind the simulation software used to create the flow scenario data utilized for training and out-of-distribution test data (refer to Section 2.3).

2.3.1 Convolutional Recurrent Autoencoder for Flow Prediction

The basis architecture which is reproduced and the uncertainty quantification tried on methods is a convolutional recurrent autoencoder (AE-RNN) architecture previously designed and tested by (Jarmatz et al., 2023) specifically for large MaMiCo generated Flow Data. The sketch of the full architecture is shown in 2.19 below. As can be seen it is composed of two model architectures, an autoencoder[1] and a recurrent neural network[2].

[1]



[2]

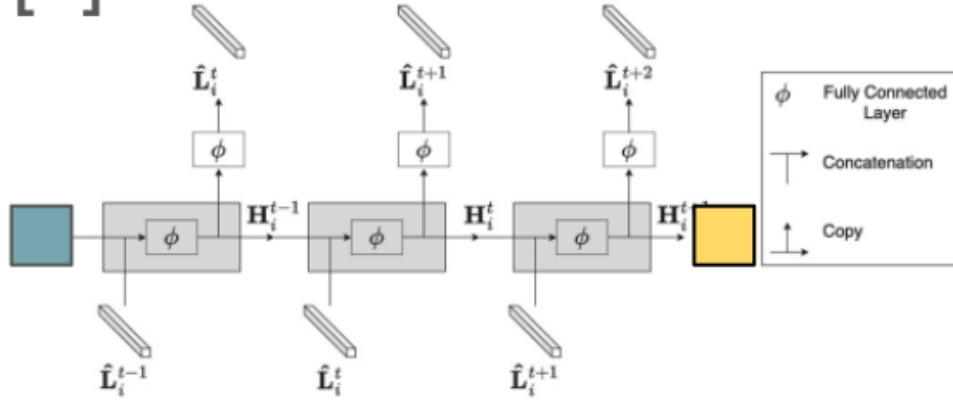


Figure 2.19: The sketches of the AE and RNN from (Jarmatz et al., 2023) that were separately trained and combined for Flow prediction. The blue and yellow boxes illustrate the data flow from the AE encoder part to the RNN back to the AE decoder, for encoding prediction and decoding back to velocity representation.

The combined AE-RNN was chosen and further designed to be able to predict temporal-spatial depending data specifically Flow Data for a large input of coupled MD-CFD simulation information, therefore besides the ability to capture and model spatial and temporal relations regressively (See Section 3.4), for this the AE and RNN inherent architectures and learnings are helpful, as will know be shortly discussed below 2.3.1, 2.3.1. It also helps in reduce the data size during the regressive prediction step through the latent space encoding via the autoencoder. (See Section 2.3). Furthermore it was designed to be denoising. (Jarmatz et al., 2023).

Convolutional Autoencoder:

The autoencoder, in our case model part 2.19[1], is a model, from its architecture good in learning to extract spatial features from an image or in our case from a flow fields.

It includes convolution and to revers deconvolution layers layers(DoubleConv, Deconvolution) for the spatial dependence logic, as well as pooling layers(MaxPool), to decrease the data size.

Convolutional layers, first systematically introduced by Yann LeCun *et al.* in 1989 (LeCun et al., 1998) for image processing tasks, learn spatial features by sliding kernels W_h across the input tensor U and computing local weighted sums:

$$\hat{L}_{i,j,k,m} = \sigma \left(\sum_{a,b,c} U_{i+a,j+b,k+c} \cdot (W_h)_{a,b,c,m} + b_{h,m} \right), \quad (2.19)$$

where the kernel size defines the receptive field capturing spatial correlations at different scales, like edges, smaller next neighbor relations or more complicated spatial relations, based on the kernel size and convolution step number. Similar as in other neural networks here, the nonlinear activation $\sigma(\cdot)$ introduces complexity beyond linear filtering.

$$\hat{U}_{i,j,k,d} = \sigma \left(\sum_{m,a,b,c} \hat{L}_{i+a,j+b,k+c,m} \cdot (W_g)_{a,b,c,m,d} + b_{g,d} \right). \quad (2.20)$$

Furthermore what is not stated in the image, but included in the original code, are skip connections between encoding and decoding layers to promote detail preservation and stable training (reducing the vanishing gradient problem). The decoder $g(\cdot)$ reconstructs spatial data approximately via transposed convolutions. The feature maps saved at each convolution step, theoretically provide more interpretable spatial filters learned to represent underlying flow structures and noise and are overgiven through the skip connections.

Recurrent Neural Network:

The recurrent neural network, in our case model part 2.19[2], is designed to extract and learn temporal features and use them to predict the next step in a temporal sequence.

It does this by learning the temporal features and saving them in hidden states, which are then used to predict the next temporal step of the sequence, regressively adding the state to the sequence and removing the first temporal step in FIFO principle, to then determine the next temporal step based on the new sequence.

The RNN was formally introduced by Rumelhart *et al.* in 1986 (Rumelhart et al., 1986) and further studied by Elman in 1990, models temporal dynamics by updating a hidden state H_t recurrently:

$$H_t = \sigma \left(\hat{L}_t W_{xh} + H_{t-1} W_{hh} + b_h \right), \quad (2.21)$$

where matrix multiplications propagate current latent input $\hat{L}_t$ and past state H_{t-1} information forward. Again a nonlinearity $\sigma(\cdot)$ (e.g. tanh or ReLU) models is introduced to model the complex temporal relations. Prediction of the next latent vector is given by

$$\hat{L}_{t+1} = H_t W_q + b_q, \quad (2.22)$$

The Hidden states H_t representing the learned dynamic behaviors over the sequence.

In their work (Jarmatz et al., 2023) also compared the RNN to the performance of so called gated cells such as LSTM and GRU instead for the architecture, to compare their benefit in addressing vanishing gradients and longer-range dependencies to the prediction skill. Because of better performance for the specific task, on the coupled molecular-CFD flow sequences for the specific hyperparameter sets tested, they concluded the RNN having the best performance for this specific task.

Historically, RNNs were generally under the first approaches for sequence tasks until the rise of transformers (“Attention Is All You Need”, (Vaswani et al., 2017)) which mitigate recency bias and enable parallel processing at higher computational costs, but can also be depending on the specific task and architecture be hard to train and converge.

2.3.2 Flow Data Simulation Software

MaMiCo Framework Overview

MaMiCo (Macro-Micro Coupling tool) is an open-source framework for hybrid molecular dynamics (MD) and continuum fluid dynamics (CFD) simulations (Neumann et al., 2016). It enables bidirectional coupling between an MD domain and a larger CFD domain, with a Cartesian grid of macroscopic cells mediating data exchange. The framework uses a domain-embedded configuration where the MD (particle-based) region is fully contained within a lattice-Boltzmann (LBM) CFD region, allowing investigation of multi-scale flow phenomena where both molecular and continuum effects are relevant.

This infrastructure generates training data for the flow scenarios described in (Section 2.2), which are used to train the AE-RNN architecture (Section 2.3.1) for flow prediction with uncertainty quantification (Section 2.1). Detailed descriptions of the MaMiCo framework,

governing equations, simulation physics, micro-to-macro transfer mechanisms, time stepping, boundary conditions, and thermostat settings are provided in ([Appendix 5.3](#)). Domain and simulation parameters are discussed in the following chapter.

3

Simulation Setup, Datasets, and Training

The simulation framework generates training data through coupled MD-CFD simulations (See Section 2.3) of two distinct flow scenarios: Kármán Vortex Street (KVS) (See Section 2.2.2) for training and Couette flow (See Section 2.2.1) for out-of-distribution testing. This section describes the domain geometry, MD box placement, and simulation procedure that collectively determine the dataset characteristics.

3.1 Domain Geometry and Physical Setup

Both scenarios employ the same MD domain configuration, a cubic region of $60\sigma \times 60\sigma \times 60\sigma$ (approximately 0.19 m^3) containing 56 molecules per direction, but differ in their macroscopic flow domains and MD box placement strategies. Table 3.1 summarizes the key domain parameters.

Table 3.1: Domain Configuration Overview

Component	KVS	Couette
<i>Macroscopic Domain (LBM)</i>		
Physical size	$2.5 \times 0.41 \times 0.41 \text{ m}$	$100 \times 100 \times 100 \text{ m}$ (channel)
Grid resolution	$(780 \times 128 \times 128)$ at $\text{cpm}=312$	Variable
<i>MD Domain</i>		
Domain size	$60\sigma \times 60\sigma \times 60\sigma$	$60\sigma \times 60\sigma \times 60\sigma$
Physical size	$\approx 0.19 \text{ m}^3$	$\approx 0.19 \text{ m}^3$
Molecules per direction	56; 56; 56	56; 56; 56
Time step	0.005	0.005
Timesteps per cycle	50	50

For KVS, the macroscopic domain is a rectangular channel ($2.5 \times 0.41 \times 0.41 \text{ m}$) containing a cylindrical obstacle positioned at $x \in [0.45, 0.55] \text{ m}$ and $y \in [0.15, 0.25] \text{ m}$, creating a wake

region downstream where vortex shedding occurs. For Couette flow, the macroscopic domain represents a channel of height $H = 100$ m between parallel plates. The MD box placement strategy for Couette flow, which exploits the linear velocity profile to efficiently capture flow characteristics, is detailed in Section 3.2.

3.2 MD Box Placement Strategy

The strategy for placing MD boxes differs between KVS and Couette scenarios and was chosen, based on their dataset role and flow characteristics, as discussed in the flow theory section 2.2. In KVS, MD boxes are positioned to gather maximum and diverse information content for training data and out-of-distribution data, whereas in Couette, the focus is on an efficient enough coverage of flow physics for out-of-distribution (OOD) testing.

KVS: Wake Region Sampling for Training Data KVS simulations generate the primary training dataset. The cylindrical obstacle creates vertical (z-direction) symmetry, and the resulting KVS vortex shedding is predominantly 2D (Section 2.2.3), enabling data extraction from a single 2D plane without losing essential physics, as illustrated in the Figure 3.1 below.

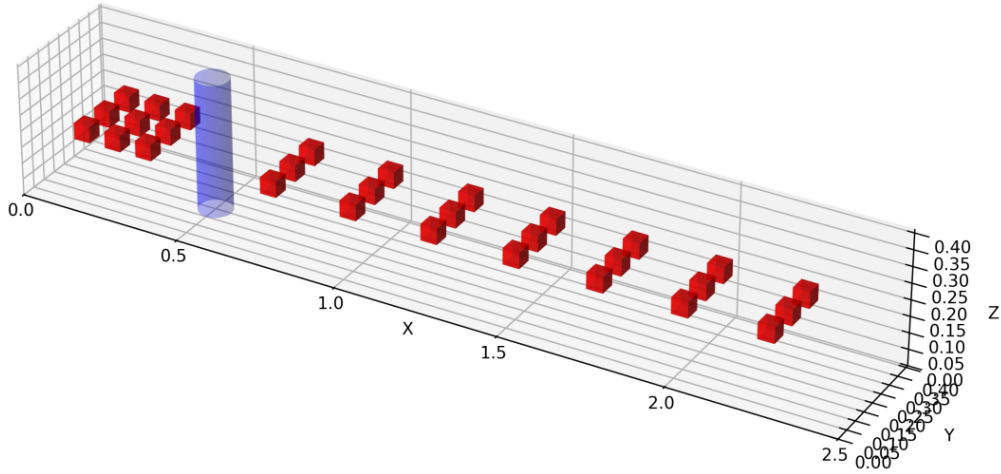


Figure 3.1: MD box positions for KVS training data. Boxes are placed in the wake region downstream of the cylinder to capture maximum turbulence intensity and velocity gradients.

MD boxes are positioned in the wake region where the velocity gradient is the strongest, (Table 3.2), helping the autoencoder to better learn the spatial relations 2.3.1, following (Jarmatz et al., 2023) for reproducibility. Additionally multiple init times are used 5.3, ensuring training data contains a good balance between temporal and spatial variations, creating high information gradient data, while also minimizing temporally redundant

information.

Table 3.2: KVS MD box positions for training data extraction.

Center Position [m]	Purpose
[1.25, 0.1, 0.205]	Training: High turbulence intensity
[1.25, 0.3, 0.205]	Training: Wake edge, strong gradients
[1.5, 0.1, 0.205]	Training: Downstream wake region
[1.5, 0.3, 0.205]	Training/OOD: Moderate turbulence

Couette Couette flow serves exclusively as an OOD test scenario, representing fundamentally different flow physics from KVS. Due to the linear velocity profile $u(z) = U_{\text{wall}} \cdot (z/H)$ characteristic of Couette flow (see Section 2.2.1), the configuration employs a minimal three-position strategy (top, middle, bottom), as listed in table 3.3 and illustrated in Figure 3.2 below, again using the flows symmetry to more efficiently capture the essential flow characteristics without excessive computational cost.

Table 3.3: Couette MD box positions for OOD testing.

Position	Domain Offset	Velocity Range
Bottom	20.0; 20.0; 0.0	$u \approx 0$ (stationary wall)
Middle	20.0; 20.0; 20.0	$u \approx U_{\text{wall}}/2$
Top	20.0; 20.0; 40.0	$u \approx U_{\text{wall}}$ (moving wall)

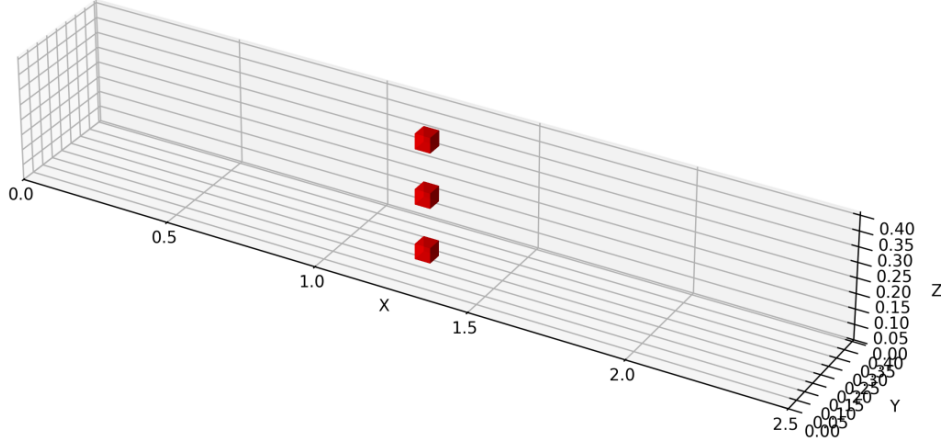


Figure 3.2: Couette flow MD box positions spanning the channel height. The three positions capture the complete linear velocity profile.

Three positions are optimal because the linear velocity profile $u(z) = U_{\text{wall}} \cdot (z/H)$ is fully characterized by these points, spanning the complete velocity range from zero to maximum.

Combined with seven wall velocities ($U_{\text{wall}} = 0.5\text{--}5.0$ m/s), this yields 21 distinct flow conditions sufficient for OOD evaluation while maintaining computational efficiency.

3.3 Simulation Procedure and Initialization

KVS simulations require extended initialization to establish periodic vortex shedding from an initially quiescent state. Multiple initialization timesteps (20000, 22000, 24000, 26000, 28000) are used, following (Jarmatz et al., 2023), 3.2, to provide temporal diversity by capturing different phases of the shedding cycle, with all phases used in training. Couette flow reaches steady state rapidly due to its linear profile, requiring no special initialization. Simulation parameter settings, data preprocessing steps, merging processes, and flow convergence requirements are detailed in the appendix (Appendix 5.3).

3.4 Datasets for Training and Out-of-Distribution Evaluation

For the Out-of-Distribution Detection Analysis, three specific datasets are selected. While more datasets were created, e.g. Rotated, Shuffled etc., these three were chosen based on their stronger characteristic different results, illustrating the spread in OOD results the best. They vary strongly by underlying physics, complexity and relationship to the training data, while the other datasets mostly fell inbetween these three extremes.

The used datasets and there inherent differences and expectancies on the resulting epistemic uncertainty are listed in the table 3.4 below. Basic flow pattern is visualized for illustration in the sketch in Figure 3.3 below.

Scenario	Physics	Distribution Shift	Expected Uncertainty
In-Domain OOD	KVS (same)	Spatial extrapolation	Moderate increase
Couette Flow	Different flow regime	Different physics	High increase
Random Data	No physical meaning	Extreme shift	Very high increase

Table 3.4: Characteristics of the three out-of-distribution test scenarios used for uncertainty quantification evaluation. The scenarios are ordered by increasing severity of distribution shift, from moderate spatial extrapolation to extreme physically invalid inputs.

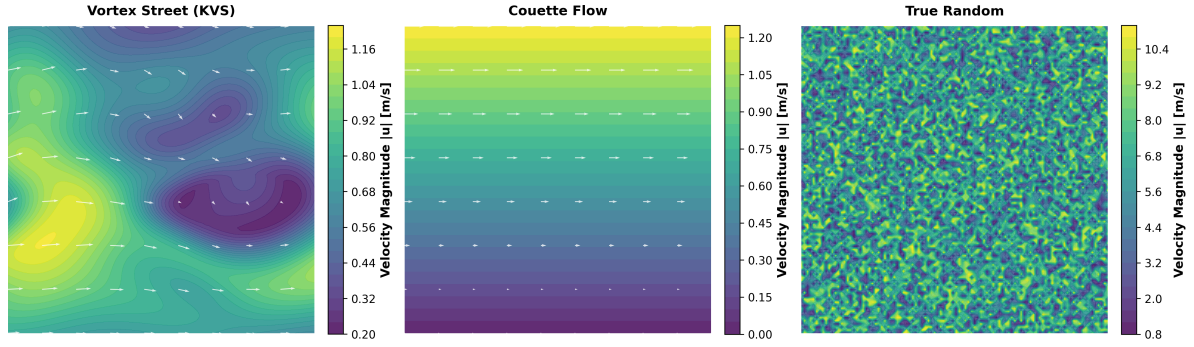


Figure 3.3: Three flow scenarios for OOD evaluation: KVS with vortex shedding (Left), Couette with linear velocity profile (Middle), randomly sampled data with no coherent structures (Right). Each represents different distribution shift levels.

First, we have in-domain out-of-training data (**ID-OOTR**), which extrapolates within the tested domain. This data is part of the original KVS simulation but comes from areas distant from the training data, showcasing less distinct velocity edges while retaining similar flow patterns.

Next is the **Couette** flow simulation data, which represents a different flow scenario. While valid, it features a linear velocity profile and steady-state behavior, contrasting with the complex periodic vortex shedding in the KVS dataset. Despite its simplicity, the model may still generalize well, making its interaction with uncertainty quantification methods interesting.

Finally, we will use randomly sampled data (**TrueRandom**) within the range of 1 to 11. This data does not correspond to any specific flow scenario, serving primarily as random noise and acting as a stress test for the uncertainty quantification methods.

3.5 Training Procedures and Hyperparameter Optimization

3.5.1 Reproduction of Baseline Architecture

Since the basis architecture and training are reproduced based on the publication [Jarmatz et al. \(2023\)](#), 2.3.1, they are included in the appendix. (See Appendix 5.3)

3.5.2 Training and Hyperparameter Optimization of New Architectures

The Training for aleatoric and epistemic uncertainty methods combined with aleatoric methods, such as evidential regression, negative log-likelihood (NLL) combined with Monte Carlo dropout, and NLL for ensemble approaches, can take a long time, and can lead to bad performance if the hyperparameters are not systematically searched. Therefore we

utilize grid search for hyperparameter optimization because it provides reproducibility and robustness, although it is also the most resource-intensive option. The found parameters are summarised in Table 5.4. Given our focus on discussing the methods and the importance of prediction accuracy and mean absolute error (MAE) for our analysis, this approach is justified. The details of the grid search analysis and results are discussed in the appendix. (See Appendix 5.3)

Additionally, the workflow design for training, executing the various uncertainty method architectures, saving results, and visualizing outcomes is outlined in the appendix. The training processes and methods can be run independently and in parallel. Results are saved in .numpy format, which can then be loaded and visualized using a separate visualization script. (See Appendix 5.3)

3.6 Evaluation Metrics

As discussed in the theory section (Section 2.1.2), epistemic uncertainty quantifies model uncertainty that can be reduced with more training data, while aleatoric uncertainty captures inherent data noise. For this evaluation, we focus on epistemic uncertainty for error correlation and out-of-distribution detection, as these tasks require identifying when the model lacks knowledge or encounters unfamiliar data. Aleatoric uncertainty, which captures inherent data variability (Section 2.1.3), is evaluated separately for its ability to approximate the target noise distribution. Epistemic uncertainty estimates must flag unreliable predictions (high-error cases) and identify out-of-distribution samples. Complementary metrics assess both discriminative ability and probabilistic calibration.

Expected Calibration Error (ECE) ECE measures calibration by binning predictions and comparing mean predicted probability with actual frequency (Guo et al., 2017) (Casella and Berger, 2002). Predictions are partitioned into M bins $B_1, \dots, B_M$ based on uncertainty values, and ECE is computed as:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (3.1)$$

where $|B_m|$ is the number of samples in bin m , $\text{acc}(B_m)$ is the actual frequency of high-error events in bin m , and $\text{conf}(B_m)$ is the mean predicted probability in bin m . Lower ECE indicates better calibration, with 0 representing perfect calibration. ECE complements Brier score by providing bin-wise calibration assessment, useful when uncertainty distributions are non-uniform.

Spearman Rank Correlation Spearman correlation (Spearman, 1904) (Casella and Berger, 2002) measures the monotonic relationship between epistemic uncertainty u_i and absolute

prediction error $e_i = |y_i - \hat{y}_i|$:

$$\rho_S = 1 - \frac{6 \sum_{i=1}^N (R(u_i) - R(e_i))^2}{N(N^2 - 1)} \quad (3.2)$$

where $R(\cdot)$ denotes rank transformation and N is the number of samples. The correlation $\rho_S \in [-1, 1]$ quantifies whether higher uncertainties correspond to higher errors: values near 1 indicate strong monotonic relationship, values near 0 indicate no relationship. This rank-based metric is robust to outliers and non-linear dependencies, suitable for non-parametric uncertainty distributions.

ROC-AUC ROC-AUC (Hanley and McNeil, 1982) (Hastie et al., 2009) evaluates discriminative ability by measuring how well uncertainty separates high-error from low-error predictions, or OOD from in-distribution samples, across all classification thresholds:

$$\text{AUC}_{\text{error}} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(t)) dt, \quad \text{AUC}_{\text{ood}} = \int_0^1 \text{TPR}_{\text{ood}}(\text{FPR}_{\text{ood}}^{-1}(t)) dt \quad (3.3)$$

where $\text{TPR}(t)$ is the true positive rate (sensitivity) at threshold t , $\text{FPR}(t)$ is the false positive rate ($1 - \text{specificity}$), and $\text{FPR}^{-1}(t)$ is the inverse function. The integral computes the area under the ROC curve, with $\text{AUC} \in [0, 1]$: values above 0.5 indicate better-than-random discrimination, 1.0 represents perfect separation. ROC-AUC depends only on rank ordering, making it insensitive to absolute scale—useful when uncertainty magnitudes vary across methods. Threshold-independence helps when optimal decision thresholds are unknown. However, ROC-AUC measures discriminative ability only, not calibration: a model can achieve perfect AUC while systematically over- or under-estimating uncertainty magnitudes.

Brier Score The Brier score (Brier, 1950) (Casella and Berger, 2002) evaluates calibration by treating uncertainty as a probability for high-error events or OOD samples. Uncertainty u_i is normalized to probabilities:

$$p_i = \frac{u_i - \min(u)}{\max(u) - \min(u) + \epsilon} \quad (3.4)$$

where ϵ prevents division by zero. The Brier score measures squared differences:

$$\text{BS}_{\text{error}} = \frac{1}{N} \sum_{i=1}^N (p_i - y_{\text{high},i})^2, \quad \text{BS}_{\text{ood}} = \frac{1}{N} \sum_{i=1}^N (p_i - y_{\text{ood},i})^2 \quad (3.5)$$

Lower scores indicate better calibration with 0 equalling a perfect. As a proper scoring rule, Brier score penalizes overconfidence (high p_i when $y_i = 0$) and underconfidence (low p_i when $y_i = 1$), needed when uncertainty guides probabilistic decisions.

Metric Usage for Error Calibration and OOD Detection: Metrics are often categorized by what they assess: rank-based metrics evaluate discriminative ability through ordering, while calibration metrics evaluate probabilistic interpretation through magnitude matching. For error calibration, Spearman correlation measures monotonic relationship between uncertainty and error without thresholds. Calibration metrics (Brier score, ECE) require thresholds to define high-error events: following [Tan et al. \(2023\)](#), we use error percentiles (20th, 50th, 75th percentile) as thresholds $\tau_e = \text{Percentile}(e, p)$, creating binary labels $y_{\text{high},i} = \mathbb{1}(e_i > \tau_e)$. Brier score evaluates how well normalized uncertainty probabilities match these labels; ECE assesses bin-wise calibration using the 20th percentile threshold. For OOD detection, ROC-AUC evaluates threshold-independent discriminative ability by comparing uncertainty distributions between Original and OOD datasets. Brier score uses uncertainty-based thresholds: percentile thresholds (25th, 50th, 75th percentile of Original uncertainty).

Rank-based metrics (Spearman, ROC-AUC) assess discriminative ability through ordering, while calibration metrics (Brier, ECE) assess probabilistic interpretation through magnitude matching. Both are needed: high discrimination without calibration means uncertainty can rank cases but not be trusted as probabilities; good calibration without discrimination means average magnitudes are correct but ranking is ineffective.

4

Results

In the following sections, now the results of the method comparison will be examined.

Starting with the results of the Autoencoder, focusing first on the comparison of aleatoric methods, which are discussed in [Section 4.1](#).

Next, we will delve with more depth, since this is the main goal of this master thesis, into the epistemic results, beginning with a section that highlights some interesting results from method hyperparameter tuning and concluding with our choices for hyperparameter settings in [Section 4.2](#). And then finally, presenting the main fine-tuned epistemic method results comparison, with operational performance, error calibration and especially OOD Detection results in [Section 4.3](#).

We will also explore the results of the AE-RNN full pipeline concerning epistemic uncertainty for the chosen methods, based on the outcomes from the Autoencoder comparison in [Section 2.3.1](#), which was discussed in the last section of the epistemic results for the Autoencoder in [Section 4.3](#).

Lastly, we will briefly introduce and discuss the propagation approach needed for the evidential method, because of the dependent AE and RNN models. While this topic is somewhat outside the scope of this thesis, it remains relevant for the AE-RNN evidential results mentioned earlier. The discussion will conclude in [Section 4.5](#) before we finalize our conclusions.

4.1 Aleatoric Uncertainty Results

As discussed in ([Section 2.1.3](#)) Aleatoric uncertainty captures the inherent noise in the data variability that often persists even with perfect models.

It can therefore be used to capture the inherent data noise or also to divide it from the epistemic uncertainty to make a narrower approximation on what is the models fault and what not (But this can be also problematic if the epistemic uncertainty methods behave differently or characterise different uncertainty sources and therefore result in different epistemic magnitudes and should be more used for relative comparison, rather than direct

approximation of the uncertainty like for aleatoric methods approximating the data noise).

For the aleatoric uncertainty methods, we compare five methods for capturing the MaMiCo generated simulation noise, as discussed in the theory section (Section 2.1.3).

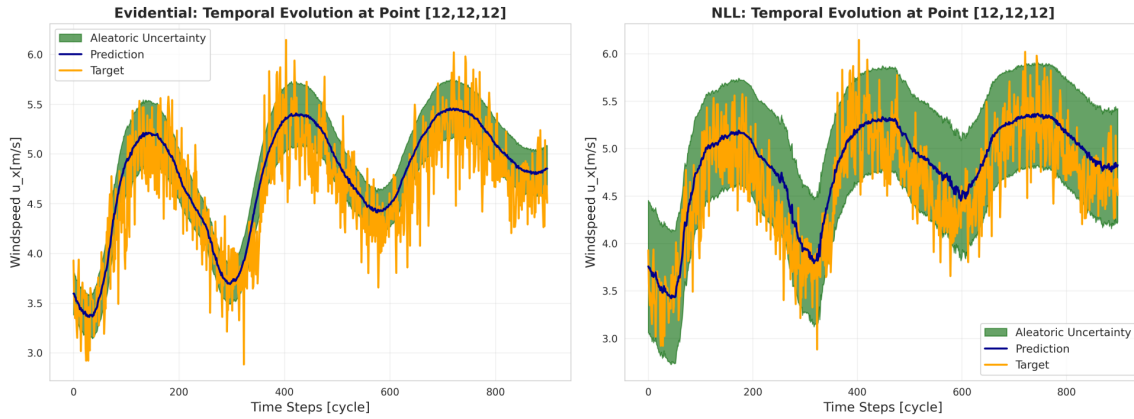
The Negative Log-Likelihood (NLL) (See Section 2.1.3), Evidential regression (See Section 2.1.4), Quantile regression, Gaussian Mixture Models (GMM) (See Section 2.1.3), and Prediction Intervals (PI) (See Section 2.1.3).

4.1.1 Aleatoric Uncertainty Distribution and Fit

In the following we will now shortly look at the resulting aleatoric interval widths of the different methods.

Starting of with a first simple illustrative point example of the results in Figure 4.1 below, and finally looking more quantitatively on the interval width and how much target noise is captured over multiple temporal steps and inference files in 4.2 below.

For all results where multiple inference files are used, due to limitations from amount of ID-OOTR samples 5 files and the standard deviation used we be used for all datasets.



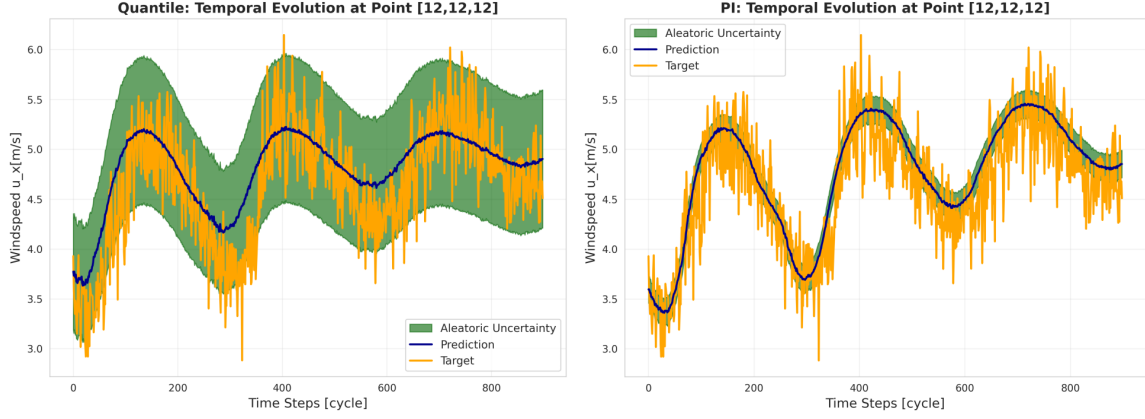


Figure 4.1: Time-series predictions with aleatoric uncertainty bands for NLL method.

Directly visible here for the example points in 4.1 and aligning with the more quantitative results for the interval width later (4.2), we see that while already all methods seem to capture the interval of target noise, which is good, their amount of target noise captured/the conservativeness of their fit is varying.

With the NLL and Quantile method having here the biggest fit capturing almost all outliers, while Evidential and PI seem to be a little bit more narrower.

More quantitatively we can see that this is also the case for more temporal steps and inference files in Figure 4.2 below, showing different percentile ranges of target noise, compared to the aleatoric methods full result.

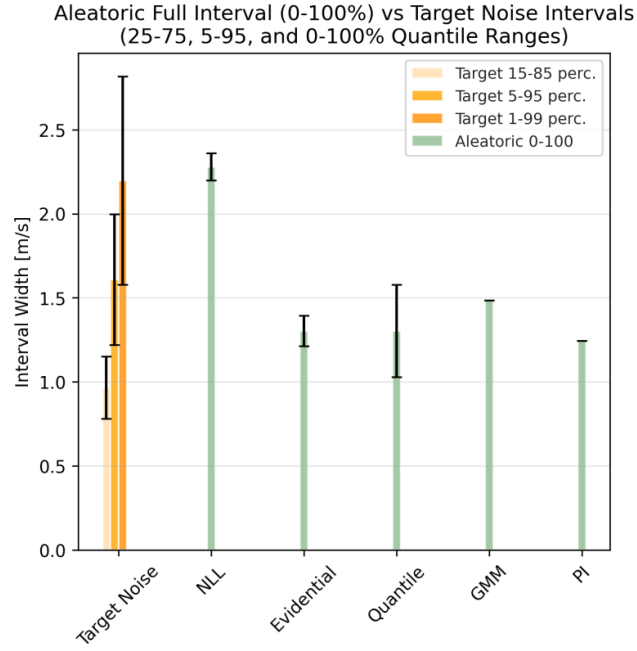


Figure 4.2: Result total interval widths of noise from means of multiple timesteps and over multiple inference files (standard bars).

When comparing the different methods, some exhibit a narrower fit to the target data, such as the Evidential and Quantile approaches, while others like NL and GMM capture more of the outliers of the target noise, as well.

This may arise from inherent underconfidence due to the Gaussian assumption in NLL and GMM, including in the loss terms involved, or just from the variance shrinking with training.

Evidential through its theoretical principle of learning to predict aleatoric and epistemic uncertainty 2.1.4, this method maybe tends to exclude more of its epistemic uncertainty from the aleatoric uncertainty and also has a penalization term for the distributions overconfident distribution prediction, which could be the reason why it has the lowest interval width predictions overall. Similarly since the Quantile method is specifically trained on enforces a percentile loss 2.1.3 (in this case actually 95%), its lower interval width makes sense, also since it is directly in the correct range of noise percentile interval region it was designed to predict, it is actually performing as expected, potentielle if more outliers should be included, this could be further adjusted.

Which method should be choosen dependents a lot on the goal of the data range assessment itself in this context. If capturing the full range of noise, including all outliers, is crucial, a method that is less confident in its narrowness is preferable, e.g. NLL. Conversely, if the goal is to approximate the noise as narrowly as possible and capturing outliers is not important,

more confident approximations like those provided by Evidential and Quantile methods are better suited, and of course it also depends on if the data distribution is known or not.

4.1.2 Aleatoric Operational Performance

Furthermore besides the data fit trade-of, also operational metrics are relevant for the decision of what method to choose.

Besides fast training convergence, for which NLL and Quantile performed the best, in contrast to PI and GMM, and finally evidential, why it worked suprisingly well here.

Further it will be looked at the inference time and prediction accuracy influence of the methods as shown in Figure 4.3 below.

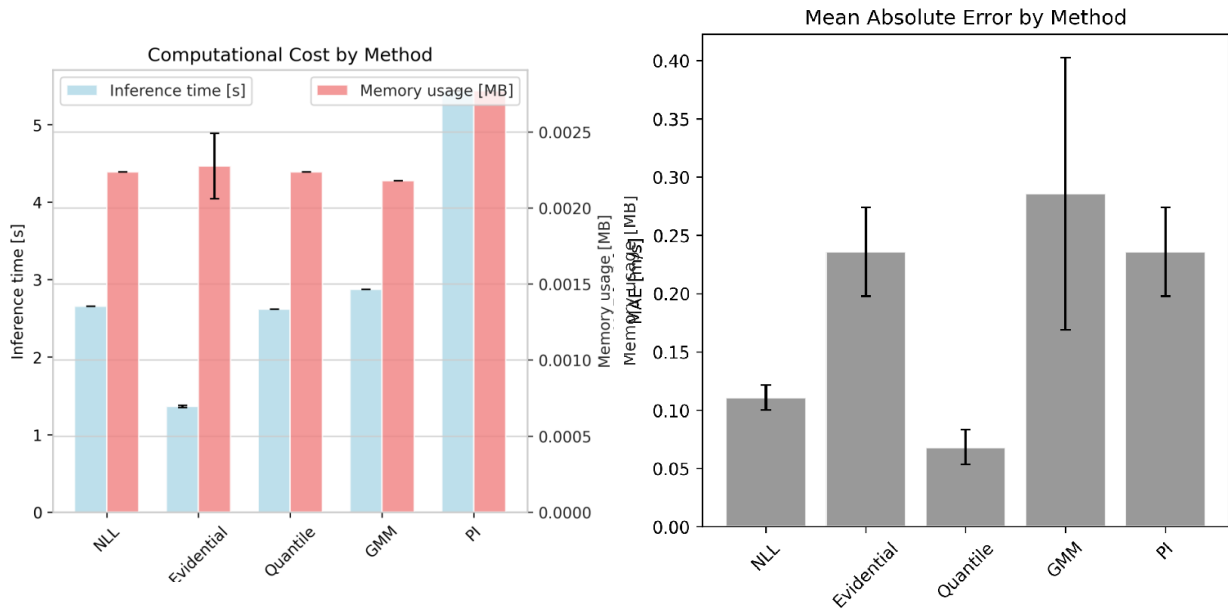


Figure 4.3: Inference time vs. MAE for different aleatoric uncertainty methods.

All methods show relatively similar inference times, with the Predictive Interval (PI) and Gaussian Mixture Model (GMM) being slightly slower and evidential being slightly faster. Memory usage difference seems to be negligible here. However, predictive accuracy varies more significantly. The Quantile and Negative Log-Likelihood (NLL) methods achieves the lowest Mean Absolute Error (MAE) In contrast, the Evidential, PICP and especially GMM methods demonstrate slightly higher errors, while still all being relatively low.

The influence on MAE appears, seems to correlate with the complexity of the introduced loss functions. While evidential seems to still to suprisingly well, besides its complicated

loss, suggesting that it may either reflect the intricate nature of its loss function that infers less into the prediction aspect during training or benefit from the loss additionally penalizing prediction accuracy decrease or also better fine-tuning and tuning schedulers of the training parameters compared to other methods.

This seems to first indicated that methods which jointly optimize for prediction and uncertainty in their loss, such as NLL and Evidential, achieve better accuracy, compared to their relative loss complexity, compared to those that primarily focus on uncertainty estimation addition, while having a less complicate loss(compared to evidential), like GMM and PI.

4.1.3 Final Model Comparison

Examining the literature, these findings align with recent comparative studies (He et al., 2023; Gawlikowski et al., 2023), which indicate that the true value of Negative Log-Likelihood (NLL) arises besides being a parametric approach from its accuracy and favorable trade-off in producing narrow and very precise data uncertainty estimates. These studies have also highlighted that evidential methods can achieve excellent calibration with suitable hyperparameter tuning, particularly with the regularization coefficient λ .

When comparing all the implemented methods based on their ability to deliver good narrow-accurate predictions and the trade-off between inference and Mean Absolute Error (MAE), the NLL, Evidential, and Quantile aleatoric losses stand out as the most recommended. This is additionally also due to their robustness in fine-tuning and convergence during training, besides the narrow-accurate aleatoric prediction, and the advantage of low inference run time while maintaining a low MAE impact on predictions.

From all the methods NLL is especially recommended when the noise in the data appears to be Gaussian distributed, which is often the case. The Evidential method is preferable when there is a need for additional epistemic uncertainty, and inference speed is significant, provided better if the loss function isn't already overly complex, while there isnt any literature on the effectiveness of Evidential compared on this. The Quantile method is suitable for situations where the data uncertainty distribution is non-parametric, often changing, or not known and requires a straightforward approach to introduce aleatoric uncertainty

Furthermore, both Quantile and NLL can be very effectively and more easily integrated into complex loss functions f.e. for additional combined epistemic uncertainty quantification or other additional predictions, due to their respective advantages. When both aleatoric and epistemic uncertainties are needed and when the loss is not too complex and model architecture does not include multiple inter-dependent models, I would recommend the Evidential method most highly, because it is one approach for both and very fast.

4.2 Hyperparameter Tuning of Epistemic Methods

The different introduced methods have different number of and type of hyperparameters and sensibilities on these.

While for a deep ensemble taking different random weight initializations can already be enough, and for sparse Laplace, we cant really change much regarding the basic functionality besides the choice of the level of sparsity of the hessian matrix and the parameters [4.2.6](#). For GP there are many different kernels we could choose and the kernel hyperparameter can have a large impact on the sensibility [4.2.2,4.2.2](#).

Because of GPs good explainability through the latent space data similarity, we will have a longer look on it, and first will visualize the data in the latent space for it and to also better understand the dataset distance in the latent space [4.2](#).

While for MCDropout the numbers of dropout layers, dropout probability and inference samples can have the most impact.[4.2.3](#)

For Evidential experimentalle the offset calibrations of the NIG parameters are varied, to calibrate the sensibility of the evidence to epistemic uncertainty translation.[4.2.5](#)

4.2.1 Sensitivity of Gaussian Process Kernels

As discussed in the theory section on Gaussian Processes ([see Section 2.1.4](#)), specific functions called kernels are used to calculate the distance between training and inference data in latent space. Different kernels are beneficial depending on the underlying data, the number of dimensions, desired smoothness, and other factors. The uncertainty estimates and smoothness of the Gaussian Processes can significantly depend on the chosen kernels. Moreover, the sensitivity and smoothness can also critically rely on the tuning of the hyperparameters inherent to the kernel function ([see Appendix 5.3](#)).

Given the substantial impact of these factors on estimation, we will briefly focus on kernel selection and hyperparameter analysis. This will allow for a deeper understanding of the method's functionality and fine-tuning, particularly because it offers better explainability compared to other methods. This clarity will also aid in explaining the results later.

Latent Space Distance Analysis To better understand the fundamental calculations, let's look at a PCA (Principal Component Analysis) of the latent space, including the data and inference points, as shown in Figure ([See Section 4.4](#)).

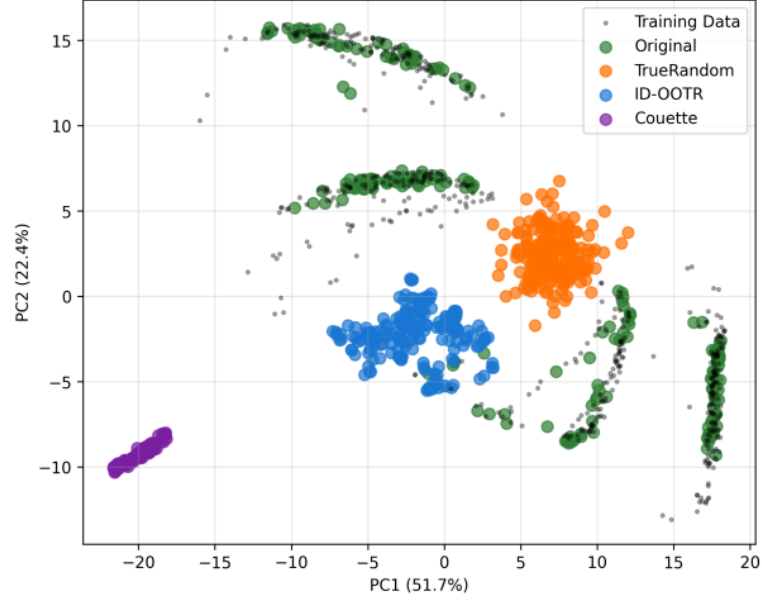


Figure 4.4: PCA visualization of latent space showing clustering of different datasets.

The PCA visualization indicates the distance of the datasets, particularly the TrueRandom dataset (in orange) and the Couette dataset (in purple), from the training and test inference data (in grey). In contrast, the validation dataset (in green) and the ID-OOTR dataset (in blue), which are still within the same domain, are strongly overlapping with the training data. Notably, the Couette case is distinctly isolated based on its primary PCA latent feature traits, which is expected due to its very different flow characteristics and symmetries (See Section 2.2.1). Since the euclidian distance in Gaussian Processes (GP) is the main translating factor directly to epistemic uncertainty, we expect it to influence the behavior towards the Couette OOD dataset (Out-of-Distribution) for OOD detection, as we will discuss in the OOD detection results later.

In Figures (See Section 4.5) below, we can quantitatively analyze the distribution of overall distances and the actual minimal and maximal separations of the training and inference points through Euclidean distance, as illustrated by the histograms, cdfs and boxplots.

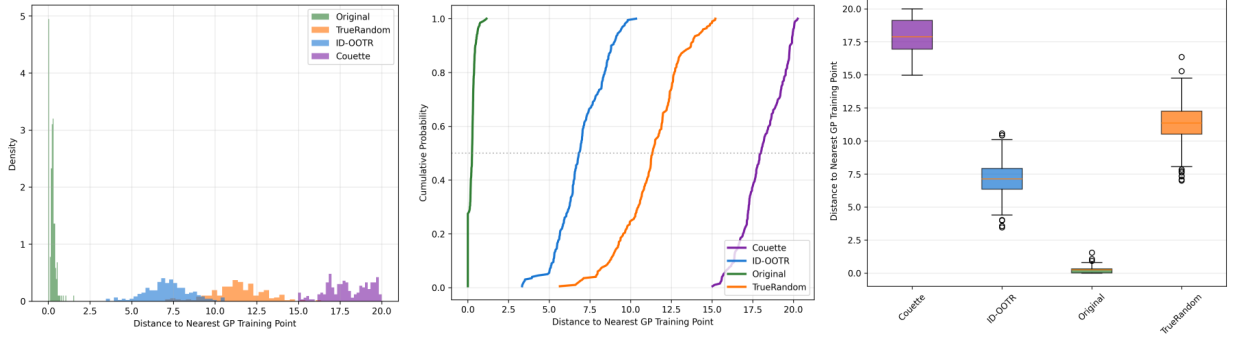


Figure 4.5: Distance-to-nearest-training-point analysis: density histogram (left) and cumulative distribution (middle) and boxplot of euclidian distance for PCA dimension (right).

The strong separation observed in the Couette case, compared to the other test cases, is quite notable. Additionally, there is a significant difference in the separation of the other out-of-distribution (OOD) test cases.

Particularly striking is the shape of the Reference/Original data and the Couette OOD data in relation to the training data. They exhibit sharper distributions in Euclidean distances compared to ID-OOT and TrueRandom. The significant difference in overall flow characteristics in the Couette case, which shows a very linear increase in amplitude, explains this observation. The calculated OOD-threshold, based on percentiles discussed later, classifies the Euclidean distances as OOD, even when the overall data falls below this threshold. This classification is reasonable based on the definitions outlined in theory. (See Section 3.4 for more details.) TrueRandom is closer to the reference test dataset than Couette because the amplitudes sampled in the TrueRandom data approximate the mean of the reference dataset, and the overall variations spatially align to some extent with vortex jumps. The random nature of the sampling explains the largest variance among all the test datasets observed in this result. It would be intriguing to delve deeper into the structures of the OOD datasets, examining their aligned features and overall differences. This exploration would be especially relevant for the randomly generated datasets, as their unexpectedly similar results compared to Couette merit further investigation, particularly concerning the changes in ID-OOT strength relative to the reference dataset.

4.2.2 Gaussian Process Hyperparameter Tuning

RBF kernel is selected to be the most relevant for the current analysis (refer to Appendix 5.3 for further discussion among different kernels).

We will now discuss here only the hyperparameter sensibility and finetuning for the final choice for the chosen RBF kernel.

The RBF kernel’s performance depends on three hyperparameters: length scale (LS), signal

variance (SV), and noise variance (α). Length scale exhibits the strongest sensitivity: for $LS < 0.1$, uncertainty saturates at maximum (1.0), while for $LS > 10$, uncertainty decreases toward zero. Small LS values create a narrow kernel assigning high uncertainty to all points except those extremely close to training data, while large LS values create a wide kernel that smooths uncertainty estimates. Signal variance shows moderate sensitivity, with uncertainty increasing gradually from near-zero at $SV < 0.1$ to high values (≈ 0.95) at $SV > 100$. While Figure Figure 4.6 visualises the different rbf hyperparameters influence on normalized epistemic uncertainty with value, Figure

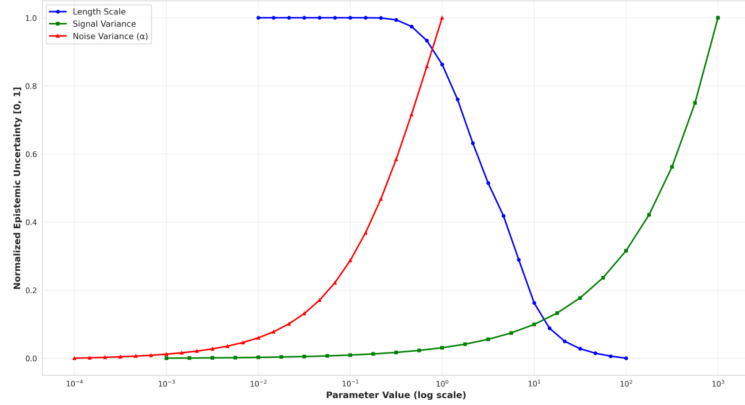


Figure 4.6: Normalized sensitivity of RBF kernel hyperparameters across several orders of magnitude.

For small LS values (0.5, 1.0), uncertainty rises steeply and quickly saturates at ≈ 0.7 , making the GP overconfident about OOD detection but losing discriminative power between different OOD types. For larger LS values (5.0, 10.0, 20.0), uncertainty increases more gradually with distance, providing better calibration that distinguishes between moderate OOD (In-Domain-OOT) and extreme OOD (Couette, TrueRandom). How saturation of epistemic uncertainty looks is also again visualised for $LS=1$ vs $LS=10$ for the RBF kernel in Figures 4.7 below through a mean bar plot and a histogram.

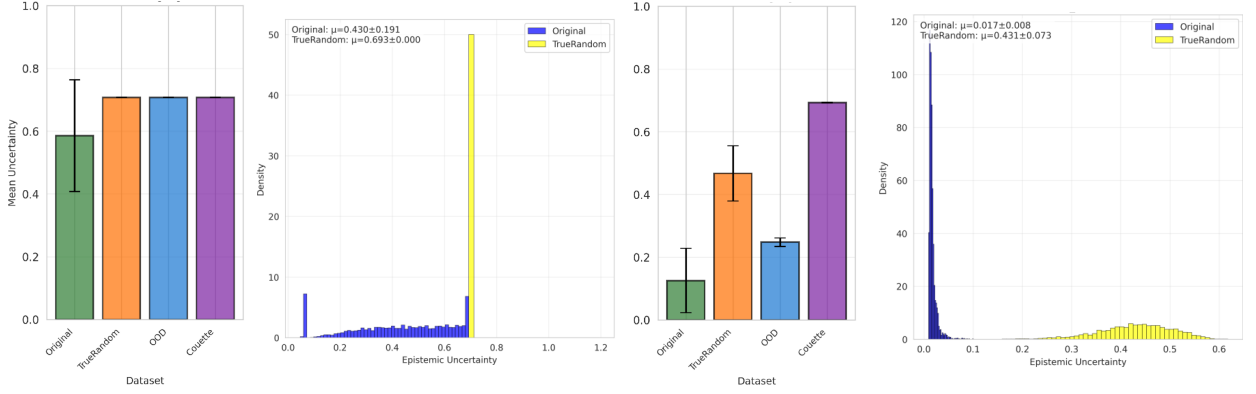


Figure 4.7: Influence of length scale saturation on epistemic uncertainty distributions, poorly calibrated and saturating (LS=1) vs. well-calibrated (LS=10) configurations shown by mean over multiple files barplot (left) an histogram for one file (right) of epistemic uncertainty with different OOD Datasets.

As can be seen from the overall LS and SV influence Figures and the lower saturation of epistemic uncertainty, here the RBF Kernel with length scale LS=10 and signal variance 1.0 seem to give a good non-saturating balance. Since we don't have the uncertainty saturating at maximum, losing the ability to distinguish between different types of distribution shift.

Finding truly optimal hyperparameters is challenging, as the loss landscape contains multiple local minima and the relationship between hyperparameters and OOD detection performance is non-linear. However, the selected configuration (LS=10, SV=1.0) also aligns with prior studies ([Rasmussen and Williams, 2006](#)), where optimal length scales typically fall in similar relative ranges providing sufficient smoothness while maintaining sensitivity to distance.

4.2.3 MC Dropout Hyperparameter Tuning

For the MCDropout model, fine-tuning is especially important because the architecture is directly influenced by the dropout layers (See Theory: [2.1.4](#)). In the worst-case scenario, this can affect the accuracy of the predictions. Therefore, we want to avoid using too many dropout layers and setting the dropout rate too high. Conversely, in order to sufficiently vary the architecture and capture its uncertainty, we also shouldn't use too few dropout layers. Thus, we need to find an optimal range for uncertainty quantification that still maintains good accuracy.

As shown in Figure [4.8](#) below on the left, this is not easy. Even for dropout rates of 0.1 or higher, the validation loss significantly worsens. This is particularly concerning given that the predictions typically fall within the range of 2-8 m/s.

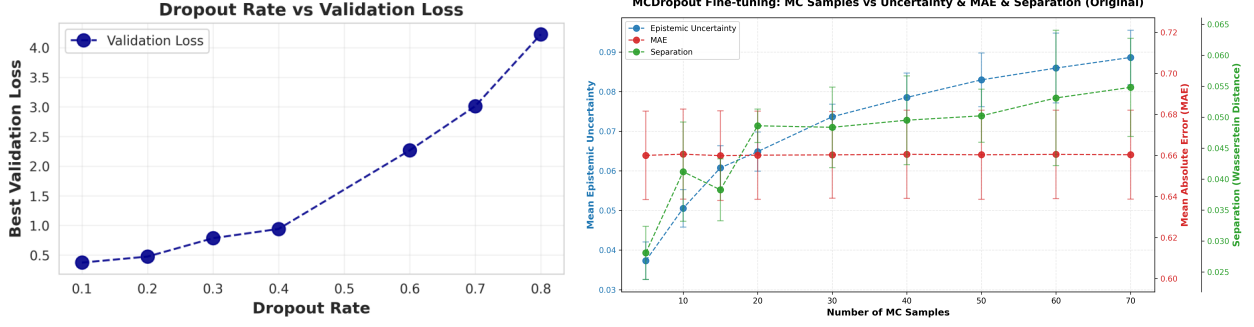


Figure 4.8: Fine tuning of MCDropout. and optimal validation loss with dropout rate during grid search. (Right) And Epistemic uncertainty, accuracy and separation between epistemic uncertainty estimates of Original and OOD Dataset with increases inference sample size (right).

For this reason, we have decided to maintain a even lower dropout rate of $p = 0.001$, based on best grid search results for minimizing the loss 5.3, to avoid excessive influence on the model, though this could influence the epistemic uncertainty.

In the future, it may be possible to finetune the dropout rate even further and assess uncertainty and separation more directly together. In this current work, we focused on fine-tuning the sampling amount through MCDropout in terms of mean absolute error (MAE) and the mean out-of-distribution (OOD) epistemic uncertainty and separation compared to the reference/original, as shown on the right in Figure 4.8 because of time constrains.

The mean epistemic results for one of the OOD scenarios and the separation from the reference appear to improve with an increasing number of samples. However, the most significant relative increase is observed with around 20-30 samples, although the increase continues beyond that point. The MAE shows a large variance across all samples, which, interestingly, does not seem to greatly affect the mean relative to the overall trends.

4.2.4 Deep Ensemble Hyperparameter Tuning

For deep ensembles a lot could be varied to try out their sensibility, we could look into different kind of deep ensembles (training hyperparameter variations, architecture variations , as discussed in the (theory: 4.2.4), because of time constrains and the sheer number of possible points to look at, we will focus on just looking at the convergence of epistemic uncertainty for ensemble member amounts, mainly to improve and validate the ensemble member optimal amount choice for the comparison as well as to maybe validate literature on this and maybe learn more regarding the loss landscape shape. 2.1.5

Variation of amount of ensemble members and model training

The number of ensemble members used can significantly affect the assessed variance of the convergence in the loss landscape minima. As discussed in the theory section on ensembles (see Section 2.1.4), we can evaluate the loss landscape and the inherent uncertainty of the models without altering the architecture or the amount of training data by employing different weight initializations. This is illustrated in Figure 4.9, which shows that variations in weight initialization can greatly impact the relative epistemic variance. However, it is important to note that this effect is also strongly influenced by the loss landscape itself, often depending on the architecture and the quality of training.

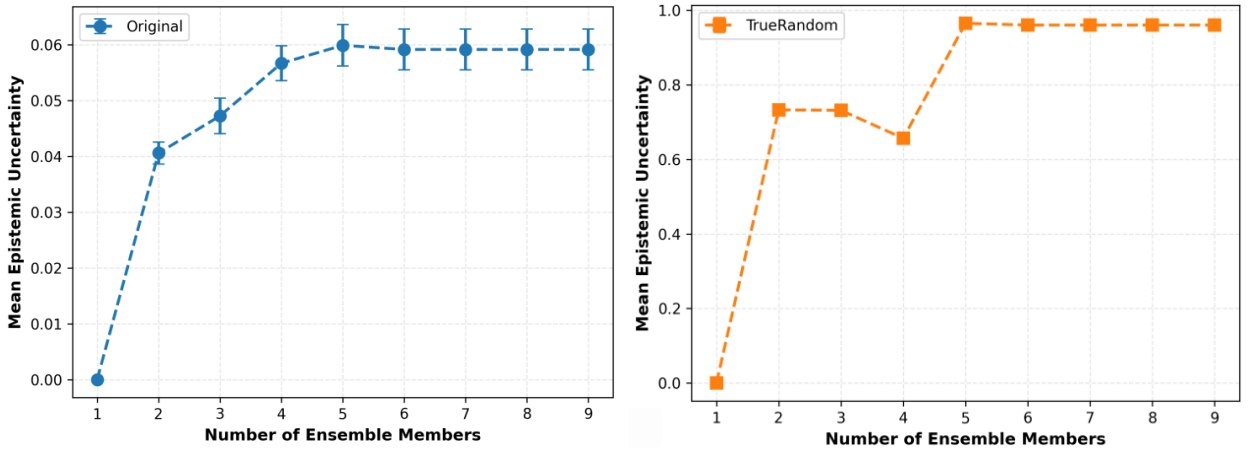


Figure 4.9: Mean epistemic uncertainty with number of ensemble members for Original/Reference vs OOD Dataset TrueRandom.

Figure 4.9 shows that, with sufficient training data, approximately five ensemble members are sufficient for the epistemic uncertainty prediction to converge and cover its most significant relative range. This reproduces similar results to those discussed by (Lakshminarayanan et al., 2017).

Effect Variation of amount of training

But additionally, it seems that this results is strongly dependent on the amount of training and training data. Further shortly illustrated here.

If we change the amount of training on the ensemble members on the uncertainty with the same architecture, Figure 4.10 presents the AE ensemble with seven ensemble members across different training epoch counts.

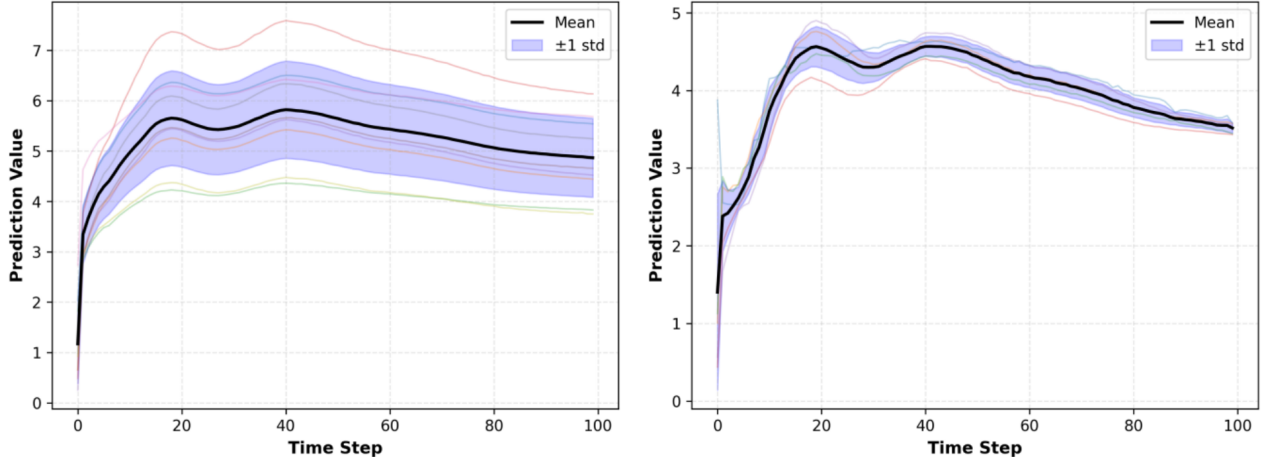


Figure 4.10: Epistemic uncertainty with training epoch of model for the example of the AE for 15 epochs (not fully converged) vs 150 epochs (well converged loss) for 5 ensemble members.

We see here clearly, that for the not enough amount of training data (left) compared to right, 5 ensemble members (marked in light blue), could possibly not enough to capture the full range of the epistemic uncertainty/variation of converged values along the global loss landscape, compared to the right longer trained ensemble members even if some ensemble members hit the target closely did diverge strongly from the approximated epistemic uncertainty through the 5 ensemble members. For both seven ensemble members were used and only using 5 for the epistemic uncertainty (See Theory: 2.1.4). For the right case this is a good enough approximation, as we also saw above in Figure 4.9. Therefore if this is in the future further explored and reproduced in other cases, this behaviour, maybe if only approximated, could be an additional indicator of/if the amount of training is enough.

4.2.5 Evidential Network Hyperparameter Tuning

For the evidential method, a regularization coefficient of $1e-4$ was determined through grid search (see Section Appendix 5.3). Additionally, during inference, calibration constants of $1e-3$ were added to the calculations of the values v , σ , and μ in the original code by (Amini et al., 2020) on GitHub. We experimented with varying these constants to assess their impact on the NIG values (shown in Figure 4.11 below) and, consequently, the epistemic uncertainty. The mean absolute error (MAE) was also measured to ensure that the predictions were not inadvertently affected.

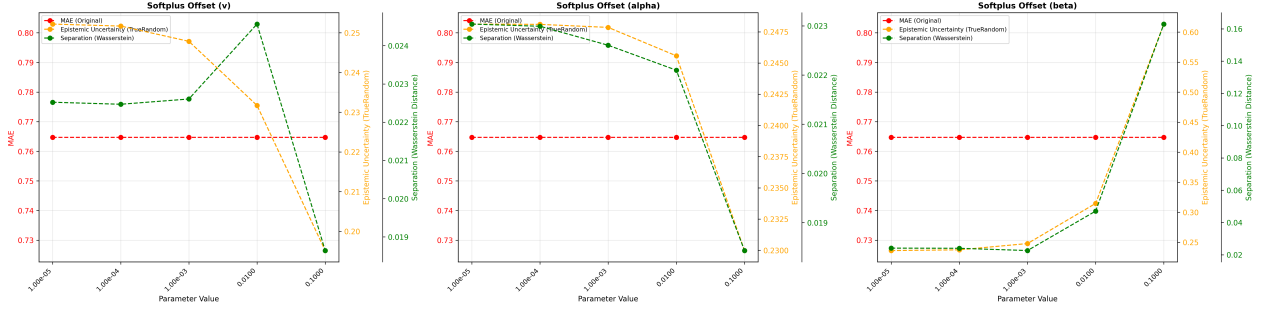


Figure 4.11: Plots of Mean epistemic uncertainty of one OOD Dataset (yellow) and Separation to Referenz (green) and MAE (red) with variations of already existing constants added to softplus function of v (left), α (middle) and β (right).

4.2.6 Laplace Approximation Hyperparameter Tuning

For the laplace method, since it is computationally not feasible to calculate the full Hessian for all parametergradients around the local minimum, we need a really high sparsity approximation, as discussed in the theory section 2.1.4 in literature even up to below $< 1\%$ of total parameters can still be a reasonable result.

This relation is shown in Figure 4.12 below on the left and right, choosing a set limited number of parameters or taking a percentage from the existing amount of parameters.

On the right we see the relation of epistemic uncertainty and separation to the number of samples taken from this hessian approximation.

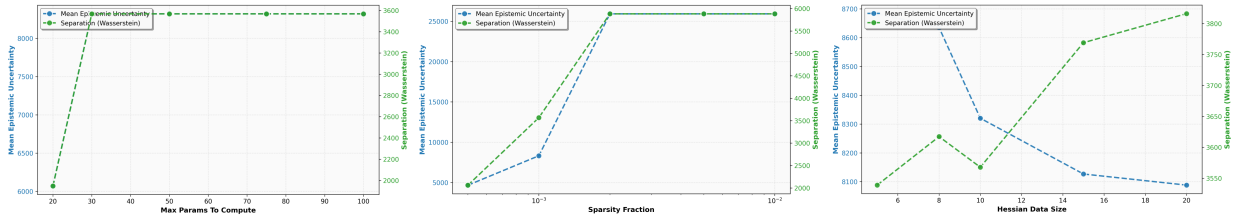


Figure 4.12: Finetuning plots for laplace, varying the max parameter or sparsity settings choosing the sparsity approximations of how many parameters should be used.(Left,Middle),as well the number of samples taking from the hessian (Right).

Interestingly we see already a saturation in increase of separation for a higher number of parameters used, though it is not clear, if we already have the most relevant parameters included or if this is only a platue.

From the resulting behaviour the values are choosen for which separation is maximised, while not already saturated.

Furthermore even more sparsity approximations are needed, taking only the diagonal of the

hessian 2.1.4, as well as only using the last layer, through this at the end we arrive at a time period of inference around 80s which is still reasonable in comparison to the other methods, while still being high.

We will see in the results later if the amount of sparsity approximations are not influencing the method too much in its uncertainty quantification capability, but else the loss on inference time compared to the ensemble would be too high.

4.3 Epistemic Uncertainty Results

We will now focus on the main comparison of epistemic uncertainty for the methods introduced in Section 2.1.4.

We will begin by exploring the initial epistemic uncertainty distribution forms for the different datasets, as illustrated in Figures 4.13, 4.14, 4.15, and 4.16. Next, we will quantitatively evaluate their model error and out-of-distribution (OOD) detection capabilities using the test datasets introduced in Section 3.4, applying the metrics discussed in Section 3.6.

As summarised in Section 2.1.5, we will compare the five epistemic methods: Deep Ensemble, which serves as our baseline (see Section 2.1.4), Monte Carlo (MC) Dropout (see Section 2.1.4), Laplace Approximation (see Section 2.1.4), Gaussian Process (GP) (see Section 2.1.4), and Evidential Regression (see Section 2.1.4).

4.3.1 Epistemic Uncertainty Distribution

Introduction

Firstly in Figure 4.13 the epistemic methods will be for one spatial points in the MD domain be visualized for illustration and to maybe provide valuable first insights into general temporal behavior and the differences in their outputs for the Referenz/Original case.

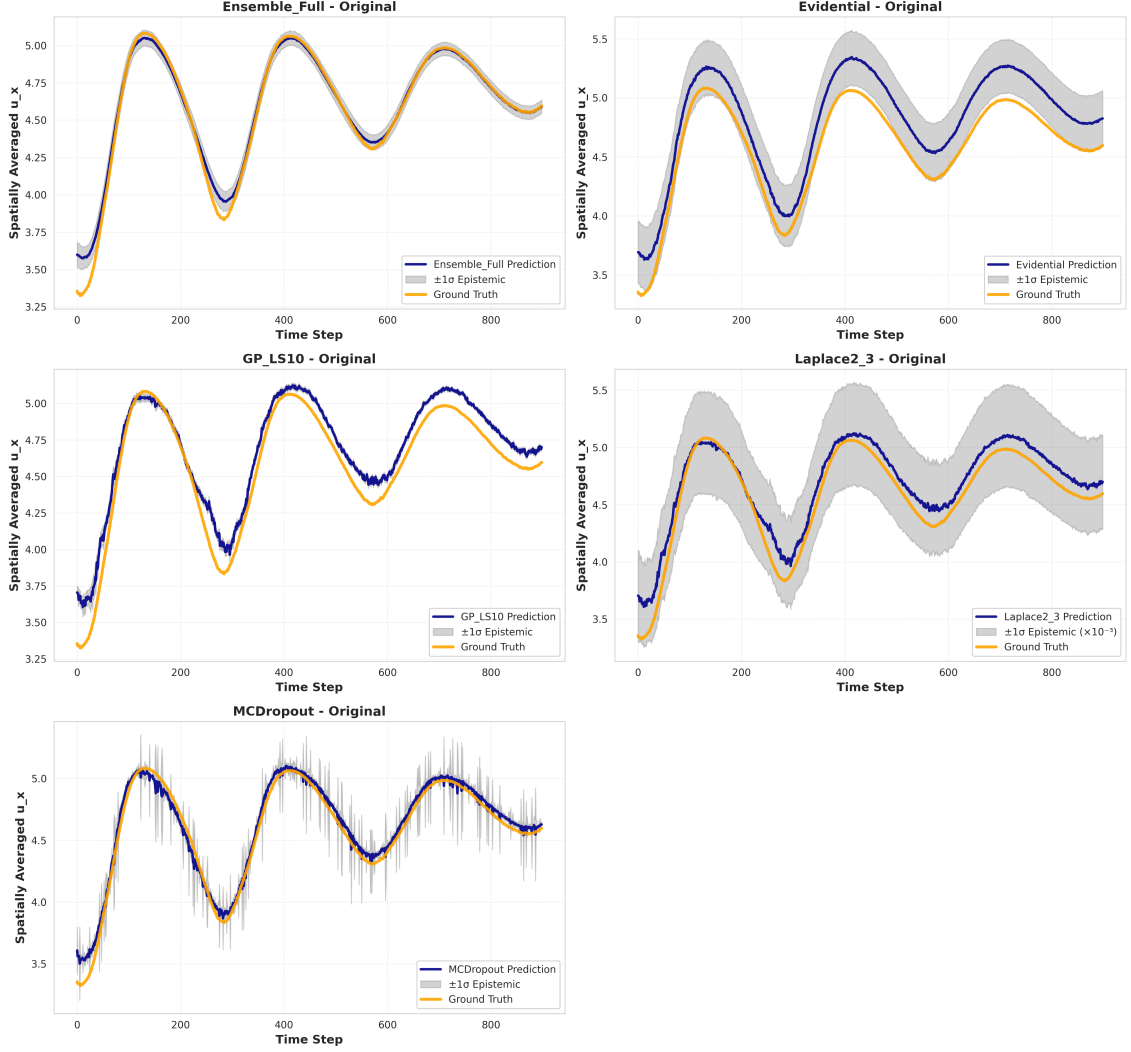


Figure 4.13: Full time-series predictions for one selected cell in the MD domain, with epistemic uncertainty bands ($\pm 1\sigma$) in grey for the different methods, showing interesting intrinsic behavior.

One direct noticeable observation, compared to the aleatoric results, is the wider variation in shape and magnitudes of epistemic uncertainty bands. These variations are inherently influenced by the distinct ways in which the methods capture epistemic uncertainty, as discussed in Section 2.1.4, particularly in relation to the loss landscape mentioned in Section 2.1.5.

Among the methods, the behavior of Gaussian Processes (GP) is the easiest to explain due to the close proximity of inference data to training data for the validation dataset, which is very limited or almost nonexistent, reproduced by (Jarmatz et al., 2023). Larger magnitudes of uncertainty are observed at single points that were less consistently represented in the training data, such as e.g. at the beginning of the dataset.

For MCDropout, the explanations are less straightforward. This method exhibits a peakier

form of uncertainty, which is strongly influenced by the probabilities of which neurons were activated or deactivated, hence maybe contributing to the strong differences.

In contrast, it is more challenging to draw conclusions for the other methods assessing epistemic uncertainty. The notably large magnitude value of 10^3 observed for the Laplace method could reflect the characteristics of the local loss minimum, suggesting higher complexity. This indicates that the Gaussian approximation may be overconfident or that there is a significant flatness in the loss landscape surrounding the converged minimum. This low curvature, interpreted as high uncertainty, is also supported by current theories discussed in Section 2.1.5. However, these assertions remain speculative and require further data and analysis for confirmation.

Furthermore, although the evidential model demonstrates the second-largest band of epistemic uncertainty, this band is relatively narrow compared to the inherent target noise. This aligns with the results of other methods and could indicate a small prediction range for the evidential model, suggesting effective and robust fine-tuning. Generally, epistemic uncertainty tends to decrease further with increasing training data.

Uncertainty Distribution Analysis While the topic is interesting, what is more relevant for this work is the relative change in epistemic uncertainty for each method when comparing the out-of-distribution (OOD) datasets to the reference dataset. To introduce to the method results, we examine in Figure 4.14 one histogram for the TrueRandom OOD stress-test Dataset below (as discussed in the Dataset section 3.4), to evaluate first, of the methods show the effect the should.

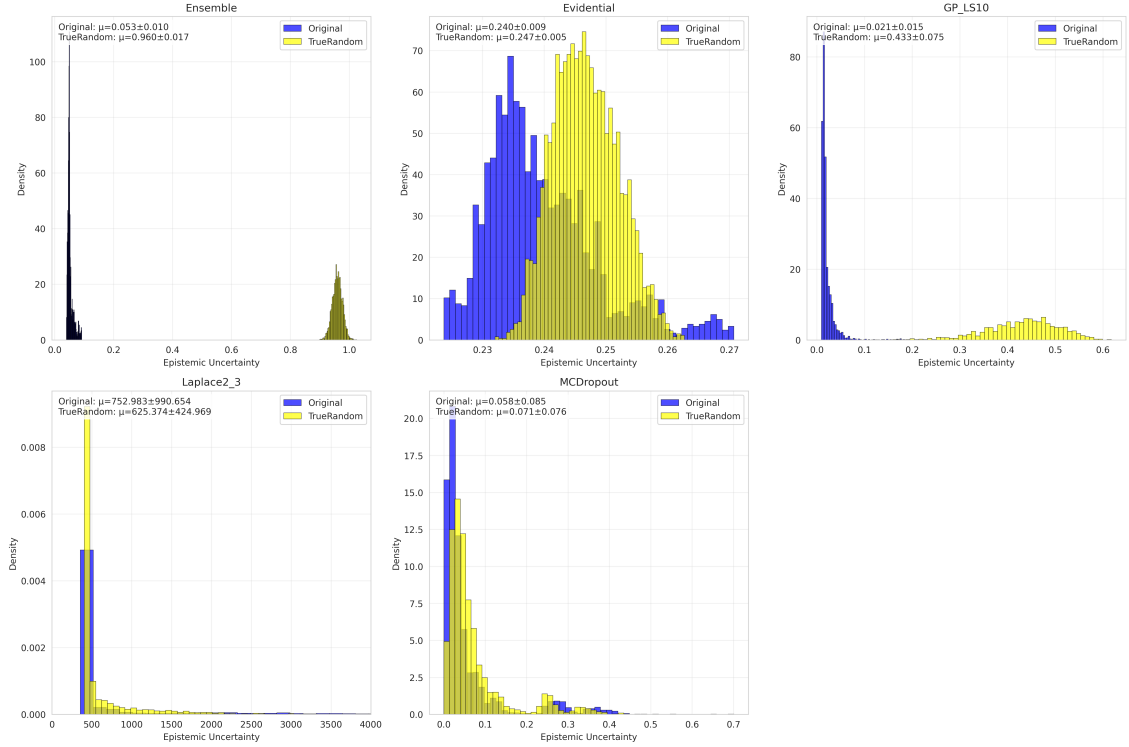


Figure 4.14: Histograms of the methods predicted distribution of epistemic uncertainty for the Reference Original Validation Dataset (Blue) vs. the TrueRandom Out-Of-Distribution Dataset (Yellow). We can clearly see a right-shift/increase in epistemic uncertainty for all of the methods, but with different relative effect and distribution shape change.

The histogram is flatten for one inference file.

First of all it is promising that all methods appear to capture the increase in model uncertainty caused by the strong OOD dataset through showing a relative increase/right-shift of the epistemic uncertainty distribution for the OOD dataset, this is already a major result, and was not for granted, especially as the GP method needed a lot of fine-tuning to have the correct sensibility 4.2.2 and Laplace needed a lot of sparsity to give results in reasonable time scope 4.2.6.

Furthermore, it is reproduced and seems reasonable to consider using the relative distribution's right-shift or increase, compared to the original/reference dataset, as a method for OOD detection, as also done in literature and discussed in the metrics section 3.6.

Additionally, it is interesting to note the differences in distribution shapes among the methods. While Ensemble and GP and afterwards show really strong shifts, not even overlapping in distribution, MCDropout and Laplace (while harder to interpretate because of its large magnitudes here), seem to be less sensible possible impacting the ood detection based on thresholds as well.

This overlap will be shortly further visualized in a more normalized manner due to the

varying total epistemic amplitudes in form of a normalised boxplots and normalised CDFs for all OOD datasets in Figure 4.15 and 4.16 below, but we will not take all inference files here into account, since this is only for visualization and the ood detection evaluations will be performed on multiple inference files 4.19. Furthermore detailed results of different separation metrics (Wasserstein, KL-Divergence, Overlap statistical value differences),etc. are included for all inference files in the appendix 5.3

The normalized boxplots and cdfs below 4.15, 4.16 clearly show again that all methods seem to somewhat capture increase in epistemic uncertainty for the OOD Datsets, although in different shapes.

In Figure 4.15 below our reference case deep ensembles we see as expected for the datasets 3.4, a relative increase in epistemic shift from ID-OOTR towards TrueRandom, while ID-OOTR also has the highest variance and more overlapping with Referenz, which also makes sense because of some samples maybe being nearer in behaviour to the Training Data. This is similar for almost all methods, while they all notice TrueRandom as OOD shifting epistemic uncertainty, some have more special/differentiating behaviour especially in how the react to Couette.

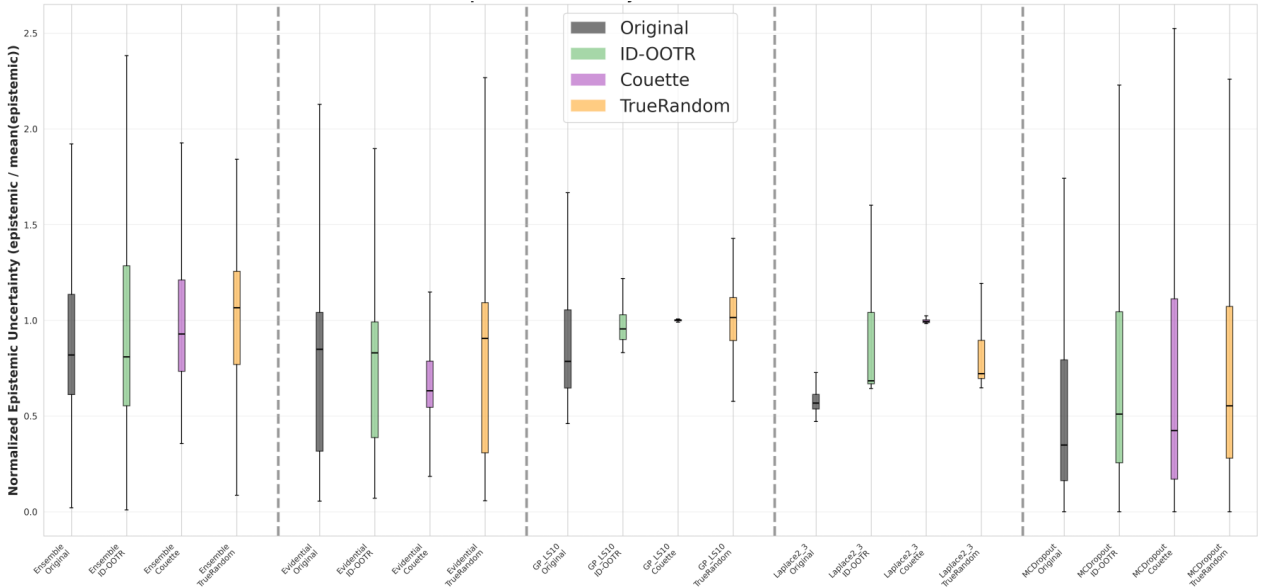


Figure 4.15: Boxplots of the normalized epistemic uncertainty distribution for all three OOD Datasets: TrueRandom , In-Domain-OOT , and Couette Datasets in (green,yellow,purple) vs Original/Reference in grey.

For certain methods, such GP and Laplace, there seems to be a significant shift and significantly reduced variance for Couette. This is likely due to their dependence on data distance (refer to Section 4.2) and local minimum curvature (see Section 2.1.4). Notably, for the Couette flow, the GP method saturates almost fully. This makes sense given the large relative

distance in latent space in this case, as observed and illustrated in the latent space PCA (Figure 4.4). Interestingly, Laplace behaves similarly to Couette concerning this saturation, in this case strong curvature change, possible hinting at the loss landscape influence of the simpler dataset; however, beyond hypothesizing, the reasons for this behavior cannot be pinpointed further without more evidence, as already explained in the introduction (Section 4.3).

In contrast, the Couette and TrueRandom cases for the other methods exhibit much greater variance and sometimes do not appear to be right-shifted or even detected as out-of-distribution (OOD) at all, especially with methods like MC Dropout and Evidential. It is also worth noting that the ensemble case did not shift Couette quite as much as TrueRandom.

This phenomenon may be explained by the model’s generalizability and the relative simplicity of the scenarios. The model could not change much in its predictions because it still works good, remaining even robust under dropout conditions. This is could also be possibly further connected to the loss landscape, as indicated by the low variance for Laplace and the deep ensemble cases for Couette.

Additionally, the separation of the distributions is more clearly represented in the cumulative distribution function (CDF) shown in Figure 4.16. Particularly for the ensemble method, the normalized separation between different OOD test cases is quite strong, again reflecting the expected relative increase between OOD datasets (refer to Section 3.4).

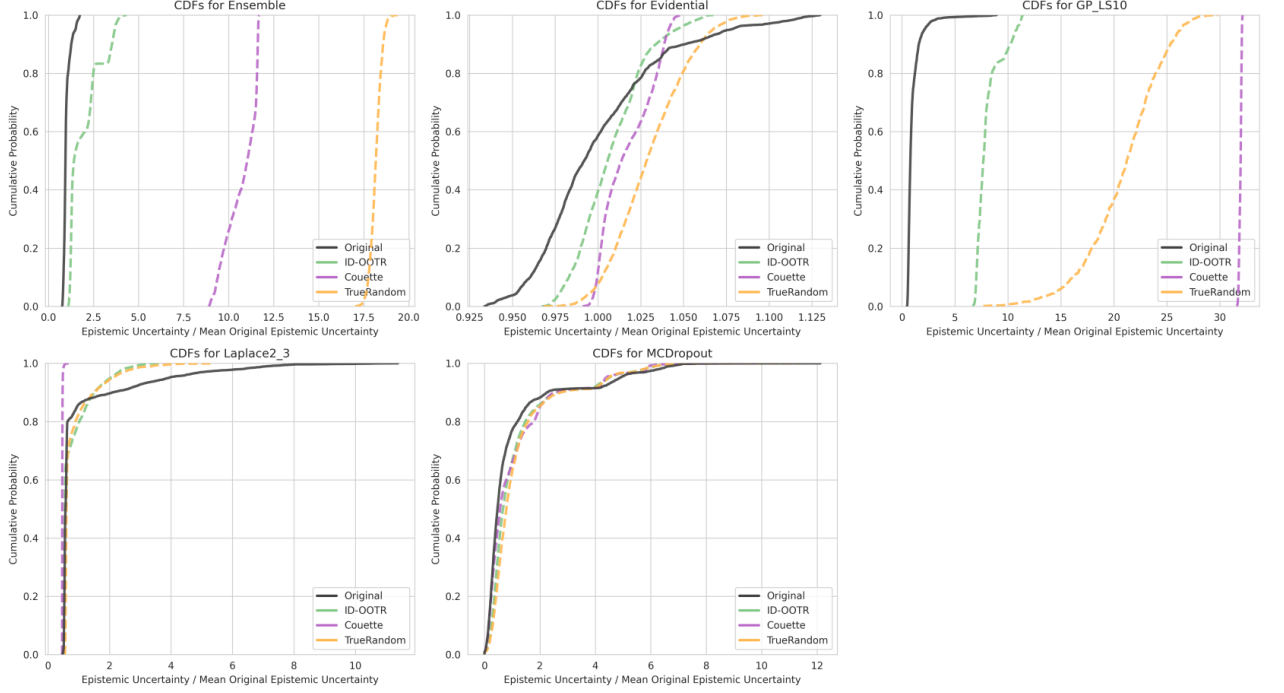


Figure 4.16: Cumulative distribution functions (CDFs) of epistemic uncertainty for the different methods: Ensemble (green), MCDropout (orange), Laplace (blue), GP (purple) and Evidential (red)

Furthermore the GP method here, probably because of its effective fine-tuning, exhibits a similar significant separation between the datasets, but it shows a swapping of TrueRandom and Couette in increase-order, similar to what is depicted in the boxplots for GP in Figure 4.15. Connectable again to the latent space distance of Couette flow scenario case, as shown in the PCA 4.4.

Following, the evidential model behaves also similarly to the deep ensemble in terms of distribution increase order, although the separations are slightly less in magnitude. This could hint at possibly later impacting the OOD-distinguishability in all sample cases from the OOD-datasets.

Finally also normalized cdfs for MCDropout and Laplace are observed, showing a relatively slight right shift with a significant overlap of the distributions again with the reference case for all OOD-datasets even after normalisation, which may significantly impact the OOD detection results later on.

4.3.2 Operational, Error, and OOD Performance

After examining the basic forms, characteristics, distributions, and shapes of the epistemic uncertainty results for the Reference/Original and Out-of-Distribution (OOD) datasets, we have observed some promising initial capabilities for OOD detection, indicated by their

separation and the increase in epistemic magnitudes. We will now after first shortly looking at the and also shortly looking at the trade-of with the operational behaviour in Figure 4.17 below, conduct a more quantitative evaluation of the OOD detection capabilities using the introduced metrics 3.6 in a calibrative and discriminative way compared in heatmaps 4.19.

We begin with operational performance, examining inference time of the methods and influence of the prediction accuracy (MAE) on the Original/Reference dataset, as shown in Figure 4.17, since even if a method is robust in detecting ood, one main goal is to make it applicable and to find other methods besides deep ensemble, since these are often very computationally expensive, due to multiple training and inference time. 2.1.5 The Figure 4.17 below shows the inference time of all methods on the left to the prediction accuracy influence on the right. Computational inference time are approximated by encapsulating the relevant method functionality in code, using the time Python package for timing.

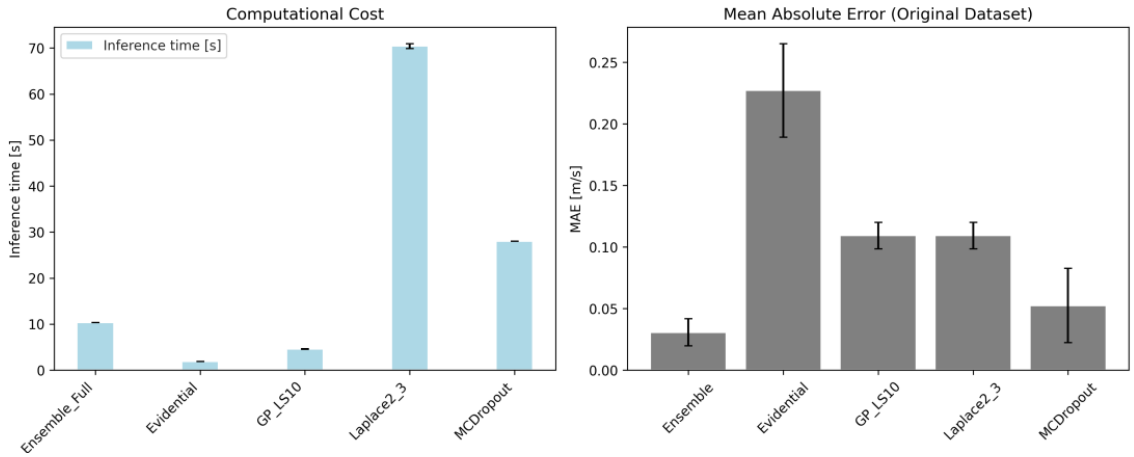


Figure 4.17: Operational metrics comparison for the different methods. Inference time (left) and MAE (right) over multiple spatially and temporally averaged Inference file variance.

In terms of inference time, the models can be ranked from lowest to highest as follows: Baseline and Evidential model, followed by Gaussian Processes (GP), Ensemble, MC-Dropout, and finally Laplace.

While the training data covariance matrices are saved before, GPs speed during the fitting process is still surprising. While evidential fastness was in contrast expected and one of its main advantages as discussed in Section 2.1.4.

Furthermore, since we use 30 samples per pass for MCDropout and still have a computationally expensive calculation for Laplace, even for decreased sparsity, both methods remain computationally expensive and perform poorly compared to the other methods, as already estimated in Section 2.1.5. It may be possible to decrease the number of MC samples further

or increase Laplace sparsity even more, but we would still need a reduction of 2/3 to 6/7 in the required computational inference time to become only comparable to 5 ensemble members deep ensemble, which seems more unrealistic without significantly impacting estimation accuracy because of the hyperparameter relations, as explored in the Finetuning Sections 4.2.3 and 4.2.6.

Regarding prediction accuracy on the Original/Reference dataset, the Ensemble model generally performs the best, which aligns with our expectations, effectively increasing accuracy, as highlighted in Section 2.1.4. Through grid search, both the Evidential model and MCDropout were finely tunable enough to compete with some of the weaker deep ensemble members. Notably, even the complex loss function of the Evidential method performed well, achieving a maximum increase in prediction accuracy of 0.15 m/s in the worst-case scenario.

Now coming to the actual error and ood quantification, the introduced metrics are compared in six heatmaps below, for each OOD-dataset one, for error correlation 4.18 and for ood-detection in Figure 4.19 using the metrics discussed in 3.6. For all heatmaps the OOD Datasets heatmaps are placed from left to right, mimicing the expected increase in epistemic uncertainty. With the methods as rows and the metrics as columns.

We will start with the error correlation heatmaps for AE in Figure 4.18 below.

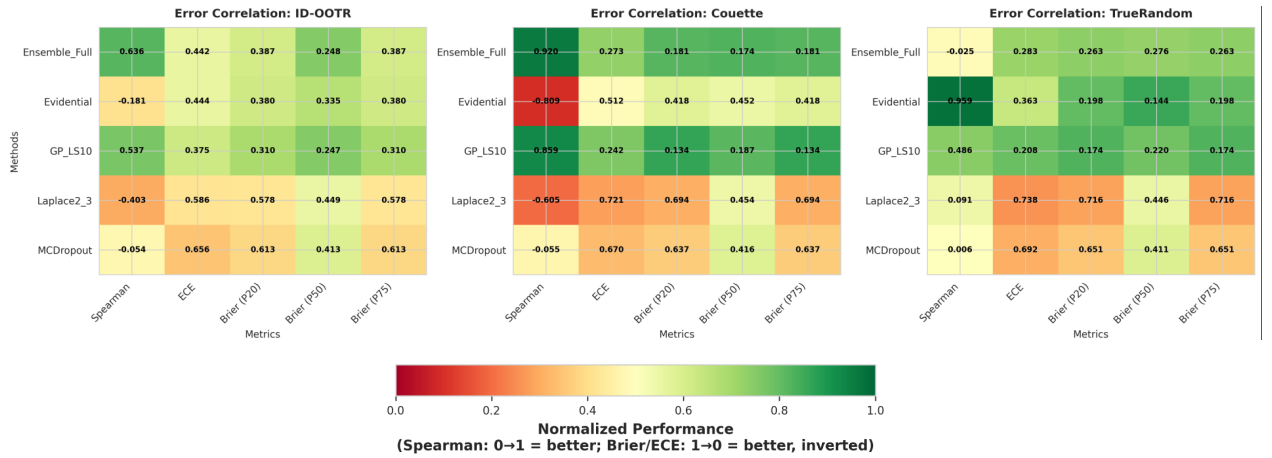


Figure 4.18: Error correlation heatmaps showing Spearman correlation, ECE, and Brier scores for all methods across OOD datasets.

We can clearly see that the Ensemble, Evidential, and GP methods outperformed Laplace and MCDropout in terms of error correlation across all metrics. For Laplace, this outcome is understandable, given its functioning (see Section 2.1.4). Its lower sensitivity to actual predictions makes it expectedly less effective for this task. In the case of MCDropout, the results are intriguing and suggest several possibilities. It

could be that the amount of dropout layers are affecting the architecture too significantly, the dropout rate might not be high enough to achieve sufficient sensitivity, since it was finetuned more focused on prediction accuracy 5.3, or otherwise fine-tuning of the MCDropout method e.g. training hyparameters could be even further improved (see Sections 2.1.4 and 4.2.3). Conversely, Ensemble, GP, and Evidential methods, aside from Spearman correlation, demonstrated good overall error correlation. While strong performance from the ensemble method is expected, it is particularly surprising for GP in this context because prediction error does not need to directly correlate with data similarity 2.1.4, but is reasonable. The robustness of its performance here is notable, although further investigation is warranted. Moreover, the Evidential method, which is specifically designed to penalize prediction errors (see Section 2.1.4), effectively learns to correlate evidence with error, which was expected. But still this performance is pleasantly surprising, especially given the method’s complexity.

Furthermore in Figure 4.19 below, we will now finally look at the OOD detection behaviour of the epistemic methods, examining the ROC-AUC scores and Brier scores, as discussed in 3.6, and the heatmap as previously introduced(4.18). Again similar to for the error correlation Ensemble and GP show the most robust capability to distinguish OOD for all datasets, with Evidential coming after.

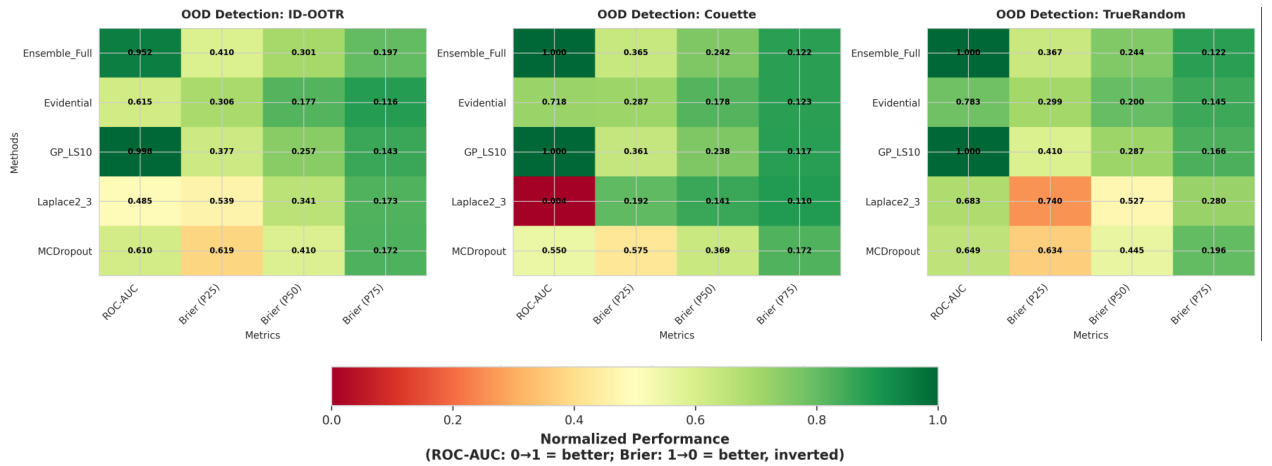


Figure 4.19: OOD detection heatmaps showing ROC-AUC and Brier scores with different thresholds for all methods across OOD datasets.

While the deep ensemble methods was expected to be robust, as discussed in the literature (see Section 4.2.4), Gaussian Processes (GP) performed admirably, tying or coming in a close second. Interestingly, evidential approaches also demonstrated significant robustness, particularly at the 25th percentile threshold. This performance may be again attributed to mainly the strong loss combination of the indirect evidence learning mechanism, as well as fine-tuning 2.1.4

From the discussion on the epistemic distribution (see Section 4.3), it was anticipated that

GP would excel here due to its fine-tuned separation of epistemic uncertainty regarding out-of-distribution (OOD) datasets (refer to Section 4.2.2). However, for the evidential approach, despite the lower separation seen in the boxplots and cumulative distribution functions (CDFs), its performance following the ensemble method did appear still very well.

Similarly, while MCDropout and Laplace showed poor separation in their initial epistemic distribution assessments, they managed to still perform better at higher percentile thresholds (notably the 75th percentile Brier score), despite expectations of underperformance, the 75th percentile seemed to be still enough for detection.

Another noteworthy observation is Laplace’s capability in predicting the Couette OOD-dataset, despite its strong sparsity reduction and post-hoc nature, which was anticipate based on the distribution analysis before and its behavior for the Couette scenario (discussed in Section 2.1.4).

The mean results and their deviations across multiple inference files for all metrics — operational, error, and out-of-distribution (OOD) — are detailed in Tables 4.1, 4.4, 4.2, and 4.3 below.

The best results for each metric are highlighted for each OOD dataset, again indicating that Ensemble, Gaussian Process (GP), and Evidential methods perform the best. Notably, GP and Evidential methods show surprising effectiveness, while Ensemble, which closely follows, still offers increased prediction accuracy, albeit with longer inference times compared to Evidential methods.

Additionally, it is worth noting the expected larger standard deviation across multiple inference files for ID-OOTR, quantitatively reflected in both the error and OOD results presented in Table 4.2.

Table 4.1: Epistemic Method AE Results: Original Dataset

Method	Ensemble	Evidential	GP_LS10	Laplace2_3	MCDropout
Accuracy [%] ↑	96.93 ± 1.10	77.28 ± 3.80	89.08 ± 1.07	89.08 ± 1.07	94.75 ± 3.02
MAE [m/s] ↓	0.0307 ± 0.0110	0.2272 ± 0.0380	0.1092 ± 0.0107	0.1092 ± 0.0107	0.0525 ± 0.0302
Inference Time [s] ↓	10.334 ± 0.000	1.888 ± 0.014	5.717 ± 0.253	70.435 ± 0.517	27.979 ± 0.000

Table 4.2: Epistemic Method AE Results: ID-OOTR Dataset

Method	Ensemble	Evidential	GP_LS10	Laplace2_3	MCDropout
MAE [m/s] ↓	0.0680 ± 0.1164	0.2010 ± 0.3438	0.3935 ± 0.4342	0.3935 ± 0.4342	0.3011 ± 0.4554
Spearman ↑	0.636 ± 0.250	-0.181 ± 0.492	0.537 ± 0.390	-0.403 ± 0.301	-0.054 ± 0.041
Brier Error ↓	0.387 ± 0.133	0.380 ± 0.085	0.310 ± 0.101	0.578 ± 0.121	0.613 ± 0.025
ECE Error ↓	0.442 ± 0.181	0.444 ± 0.102	0.375 ± 0.147	0.586 ± 0.109	0.656 ± 0.021
ROC-AUC OOD ↑	0.998 ± 0.005	0.615 ± 0.221	0.998 ± 0.005	0.485 ± 0.228	0.610 ± 0.022
Brier OOD (P25) ↓	0.410 ± 0.066	0.306 ± 0.052	0.377 ± 0.028	0.539 ± 0.167	0.619 ± 0.038
Brier OOD (P50) ↓	0.301 ± 0.061	0.177 ± 0.056	0.257 ± 0.026	0.341 ± 0.132	0.410 ± 0.045
Brier OOD (P75) ↓	0.197 ± 0.059	0.116 ± 0.025	0.143 ± 0.021	0.173 ± 0.099	0.172 ± 0.027

Table 4.3: Epistemic Method AE Results: Couette Dataset

Method	Ensemble	Evidential	GP_LS10	Laplace2_3	MCDropout
MAE [m/s] ↓	0.9994 ± 0.1661	3.2124 ± 0.2172	3.4833 ± 0.1871	3.4833 ± 0.1871	3.8265 ± 0.2021
Spearman ↑	0.920 ± 0.080	-0.809 ± 0.195	0.859 ± 0.226	-0.605 ± 0.359	-0.055 ± 0.037
Brier Error ↓	0.181 ± 0.064	0.418 ± 0.084	0.134 ± 0.031	0.694 ± 0.031	0.637 ± 0.014
ECE Error ↓	0.273 ± 0.088	0.512 ± 0.104	0.242 ± 0.055	0.721 ± 0.025	0.670 ± 0.012
ROC-AUC OOD ↑	1.000 ± 0.000	0.718 ± 0.204	1.000 ± 0.000	0.004 ± 0.008	0.550 ± 0.020
Brier OOD (P25) ↓	0.365 ± 0.010	0.287 ± 0.054	0.361 ± 0.009	0.192 ± 0.101	0.575 ± 0.039
Brier OOD (P50) ↓	0.242 ± 0.009	0.178 ± 0.052	0.238 ± 0.008	0.141 ± 0.033	0.369 ± 0.036
Brier OOD (P75) ↓	0.122 ± 0.006	0.123 ± 0.023	0.117 ± 0.006	0.110 ± 0.073	0.172 ± 0.021

Table 4.4: Epistemic Method AE Results: TrueRandom Dataset

Method	Ensemble	Evidential	GP_LS10	Laplace2_3	MCDropout
MAE [m/s] ↓	0.0498 ± 0.0009	1.0486 ± 0.0045	0.9832 ± 0.0033	0.9832 ± 0.0033	1.3790 ± 0.0064
Spearman ↑	-0.025 ± 0.000	0.959 ± 0.000	0.486 ± 0.000	0.091 ± 0.000	0.006 ± 0.000
Brier Error ↓	0.263 ± 0.000	0.198 ± 0.000	0.174 ± 0.000	0.716 ± 0.000	0.651 ± 0.000
ECE Error ↓	0.283 ± 0.000	0.363 ± 0.000	0.208 ± 0.000	0.738 ± 0.000	0.692 ± 0.000
ROC-AUC OOD ↑	1.000 ± 0.000	0.783 ± 0.146	1.000 ± 0.000	0.683 ± 0.175	0.649 ± 0.020
Brier OOD (P25) ↓	0.367 ± 0.005	0.299 ± 0.033	0.410 ± 0.010	0.740 ± 0.071	0.634 ± 0.037
Brier OOD (P50) ↓	0.244 ± 0.004	0.200 ± 0.041	0.287 ± 0.009	0.527 ± 0.170	0.445 ± 0.042
Brier OOD (P75) ↓	0.122 ± 0.003	0.145 ± 0.021	0.166 ± 0.007	0.280 ± 0.185	0.196 ± 0.030

Based on the results of the deep ensemble, the Gaussian Process (GP) and Evidential Deep Learning (EDL) methods will be further considered for the full pipeline. These three methods demonstrated the best and most robust results for the autoencoder across all out-of-distribution (OOD) datasets, considering distribution separation (refer to Section 4.3) and all metrics (see Figures 4.18 and 4.19). They also exhibited especially good to very good inference times and prediction accuracy influences. 4.17

First, the Ensemble method has already been established as a robust baseline in the literature and has been validated again through the autoencoder results here (refer to Section 3 and Section 4.3).

Next, the Gaussian Process (GP), which is frequently referenced in the literature, is recognized as a robust option here as well. The very good robustness here may specifically stem from the fine-tuning applied, but it is also noted in the literature as particularly effective for OOD detection. Notably, again it does also not alter the model architecture and has better direct explainability 2.1.4, 2.1.5.

Lastly, the Evidential Deep Learning (EDL) method has emerged as a surprisingly strong candidate, despite its complex loss, demonstrating robust results for error calibration and OOD detection (refer to Section 4.3). Additionally, to remained again, it has the strong advantage of fast single-pass inference time and aleatoric and epistemic estimation at the same time. (see Figures 4.17 and 5).

MCDropout and Laplace will be excluded from this analysis primarily due to their poorer Brier skill scores for error correlation and OOD detection, as well as their low inference speeds. While their results were still good, they will not be discussed here further, because of limited added value in comparison, atleast in this scope and for the tried architecture, especially considering the dropout-layer architecture and sparsity levels have still had to high inference time after already strong reductions. (4.2.6,4.2.3).

Consequently, Table 4.20 illustrates the full maximum combinations implemented and tested for combined approaches of aleatoric and epistemic. The Negative Log Likelihood (NLL), was used throughout because of it simplicity and minimal prediction influence and high robustness due to the further changes to loss and architecture.

For the full pipeline deep ensemble as baseline will be tested together with Gp and EDL.

Method	Uncertainty type	Scope
Baseline	-	Local(AE)
Baseline	-	Local(RNN)
Baseline	-	Global(AE+RNN)
Ensemble + NLL	Epistemic + Aleatoric	Local(AE)
Ensemble + NLL	Epistemic + Aleatoric	Local(RNN)
Ensemble + NLL	Epistemic + Aleatoric	Global(AE+RNN)
Monte Carlo Dropout + NLL	Epistemic + Aleatoric	Local(AE)
Monte Carlo Dropout + NLL	Epistemic + Aleatoric	Local(RNN)
Monte Carlo Dropout + NLL	Epistemic + Aleatoric	Global AE+RNN
GP + NLL	Epistemic + Aleatoric	Local(AE)
GP + NLL	Epistemic + Aleatoric	Local(RNN)
GP + NLL	Epistemic + Aleatoric	Global(AE+RNN)
Laplace + NLL	Epistemic + Aleatoric	Local(AE)
Laplace + NLL	Epistemic + Aleatoric	Local(RNN)
Laplace + NLL	Epistemic + Aleatoric	Global(AE+RNN)
EDL	Epistemic + Aleatoric	Local(AE)
EDL	Epistemic + Aleatoric	Local(RNN)
EDL	Epistemic + Aleatoric	Global(AE+RNN)

Figure 4.20: Full pipeline summary table providing overview of all methods, their uncertainty types, and scope. In blue Ensemble while be used as the baseline for the uncertainty quantification, while GP and EDL while be used as complementary in functionality and result OOD Detection approaches.

While it is possible for Gaussian Processes (GP) to approximate the influence of a Recurrent Neural Network (RNN) on the latent space distance of both training and inference data by shifting the fitting step after the prediction, the integration for Evidence Deep Learning (EDL) into a model inter-dependent pipeline presents more challenges due to its deterministic nature, the uncertainty being achieved only after the full pass 2.4 which was discussed as especially problematic for the autoencoder because of the RNN being in between. This issue will be discussed in section 4.5 and is a basic propagation approach.

It is experimentally but will included in the AE-RNN results for evidential in the next section.

4.4 AE-RNN Uncertainty Propagation and OOD Detection

In this section, we will now examine less into depth because of scope constrains, the error correlation and out-of-distribution (OOD) detection results for the entire pipeline incorporations. These results are more experimental and less trustworthy than those obtained with the autoencoder (AE) alone, as the recurrent neural network (RNN), even when utilizing the same hyperparameters as in the original paper, proved to be very sensitive to changes and less robust in convergence and change than the AE.

Due to the limited literature on Gaussian Processes (GP) and Evidential models, approximate approaches are employed on the methods to assess the epistemic uncertainty of the AE and RNN together. These combined approaches are not directly found in the literature, as discussed more thoroughly in the next section (4.5).

Nevertheless, we will briefly analyze the overall influence of the RNN on the metrics compared to the AE alone. Starting by shortly visualizing the first primary observation of the addition of the RNN, follwoing by analysing the results for error and ood distribution for the full pipeline for the same heatmaps again as for the epistemic autoencoder results 4.3 in Figures 4.22,4.4 below.

A Figure comparing the histograms from epistemic AE for TrueRandom for ensemble,Evidential and GP vs. for the same Dataset for the RNN variants is moved to the appendix 5.3. The section is concluded with more detailed ood and error results including the standard deviations in table 4.5,4.6,4.7.

Looking at 4.21 below, we can see that for the strongest OOD-Dataset, TrueRandom, the epistemic magnitudes overall increase for all methods when the RNN was added.

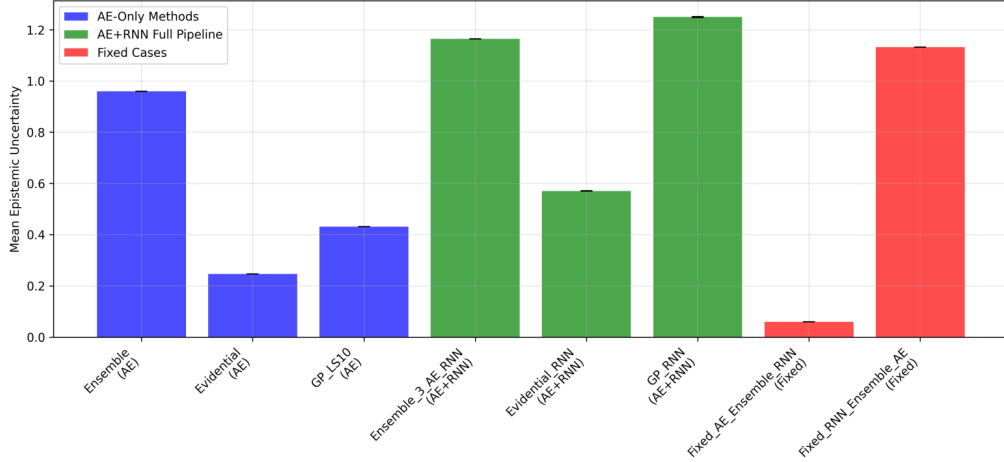


Figure 4.21: Comparison of epistemic uncertainty mean magnitudes for AE(blue) AE+RNN full UQ pipeline(green) and full pipeline but only UQ in AE or RNN (red).

For the deep ensemble approach, the epistemic magnitude was higher for the Autoencoder (AE), indicating that the Recurrent Neural Network (RNN) contributed only a slight increase. In contrast, for the Evidential and Gaussian Process (GP) models, the increase was doubled or more than doubled.

Furthermore for the deep ensemble based on the results of fixing the AE or the RNN separately red bars in 4.21, the fixed RNN showed the highest variance. This strongly suggests that the AE's transformation into the latent space significantly influences the RNN's epistemic uncertainty. The RNN from this first results seems to be particularly sensitive and non-robust when starting predictions based on a latent space transformed by a different AE than the one used for training it.

This effect is here especially detrimental, as the RNN ensemble was generated using the same AE, while different AEs were used for the complete ensemble. Unfortunately, this realization came too late; nonetheless, it represents an important finding.

For GP it seems that the RNN's prediction step further increases the distance in the latent space to TrueRandom. It would be valuable to visualize the latent space again afterwards.

Furthermore we will now look at heatmap Figures the results of the AE-RNN methods for error correlation 4.22 and ood detection 4.4 below.

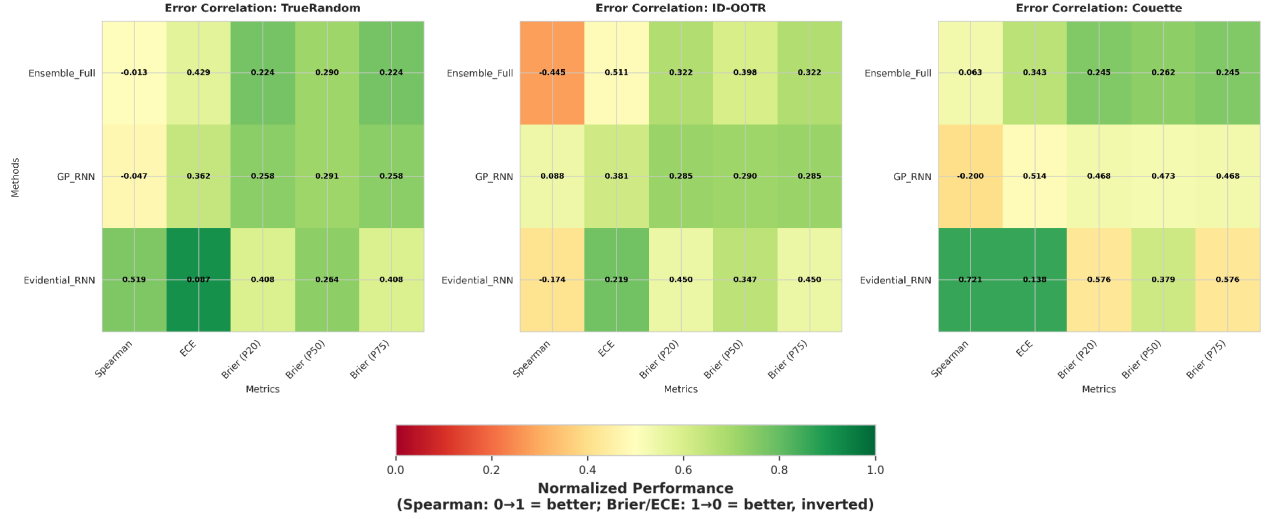


Figure 4.22: Error correlation heatmaps showing Spearman correlation, ECE, and Brier scores for all methods across OOD datasets for AE+RNN methods

For the error correlation analysis, all methods appear to perform similarly to the autoencoder (AE), although their performance is slightly worse than expected due to the increased complexity of the combined model architecture. Notably, the Gaussian Process (GP) shows a significant drop in performance compared to the AE, particularly with the Couette dataset. Additionally, all methods exhibit the best correlation at the 50th percentile threshold, which was also observed in the AE results.

The out-of-distribution (OOD) results for the AE-RNN are briefly discussed in Figure 4.4. Generally, these results replicate the findings from the AE. All methods depicted in 4.19 effectively capture the primary task of OOD detection, despite some outliers in the ensemble method. Overall, the ensemble method underperformed slightly compared to the other approaches, though not as significantly as observed in the error correlation analysis.

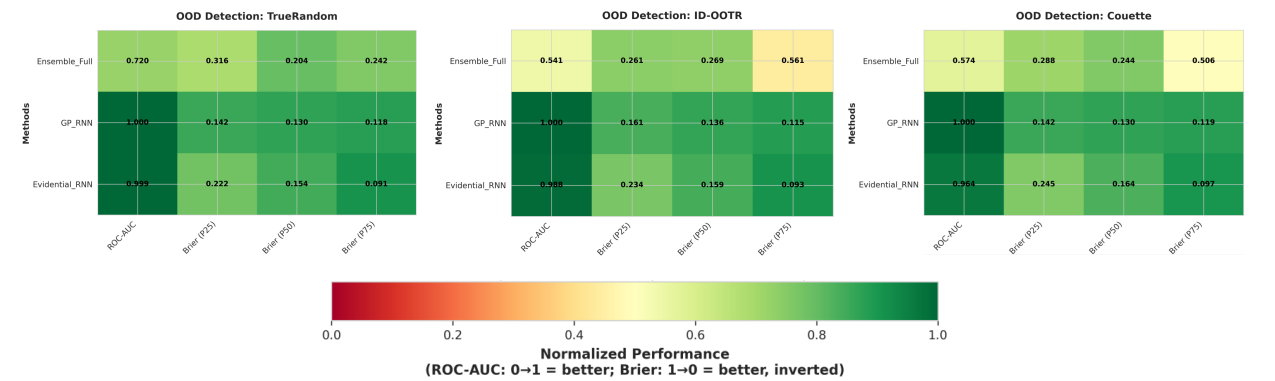


Figure 4.23: OOD detection heatmaps showing ROC-AUC and Brier scores with different thresholds for all methods across OOD datasets for AE+RNN methods

Interestingly GP and Evidential are even outperforming the ensemble here, and are surprisingly even for the RNN versions robust. Especially the propagation evidential approach, introduced in the next section 4.5 seems to work out surprisingly well.

In the following tables 4.5,4.6,4.7 we also see again that while all methods performed quite well this time GP and Evidential even outperformed the ensemble.

This could here be the case, as already discussed at the beginning of this section 4.21 because for the full ensemble the RNNs were trained only on one AE and because of the strong sensibility of the RNN towards the AE latent transformation, which was later found out.

Still the good result of the basic evidential propagation approach and the basic GP approach should be noted.

Additionally, the evidential method stands out as it appears to be the best option. With its experimental propagation approach, it is almost as robust, if not more so, than ensemble methods in distinguishing OOD samples. Notably, evidential methods perform better in terms of error correlation for the Couette dataset compared to the IT-OOTR samples.

Table 4.5: Epistemic Method AE-RNN Results: TrueRandom Dataset

Method	Ensemble_3_AE_RNN	GP_RNN	Evidential_RNN
Spearman $\uparrow$	-0.013 ± 0.050	-0.047 ± 0.027	0.519 ± 0.007
Brier Error $\downarrow$	0.224 ± 0.009	0.258 ± 0.012	0.408 ± 0.007
ECE Error $\downarrow$	0.429 ± 0.028	0.362 ± 0.035	0.087 ± 0.012
ROC-AUC OOD $\uparrow$	0.720 ± 0.000	1.000 ± 0.000	0.999 ± 0.002
Brier OOD (P25) $\downarrow$	0.316 ± 0.000	0.142 ± 0.014	0.222 ± 0.038
Brier OOD (P50) $\downarrow$	0.204 ± 0.000	0.130 ± 0.003	0.154 ± 0.020
Brier OOD (P75) $\downarrow$	0.242 ± 0.000	0.118 ± 0.007	0.091 ± 0.011

Table 4.6: Epistemic Method AE-RNN Results: ID-OOTR Dataset

Method	Ensemble_3_AE_RNN	GP_RNN	Evidential_RNN
Spearman $\uparrow$	-0.445 ± 0.319	0.088 ± 0.475	-0.174 ± 0.289
Brier Error $\downarrow$	0.322 ± 0.084	0.285 ± 0.088	0.450 ± 0.041
ECE Error $\downarrow$	0.511 ± 0.112	0.381 ± 0.026	0.219 ± 0.085
ROC-AUC OOD $\uparrow$	0.541 ± 0.000	1.000 ± 0.001	0.988 ± 0.018
Brier OOD (P25) $\downarrow$	0.261 ± 0.000	0.161 ± 0.029	0.234 ± 0.043
Brier OOD (P50) $\downarrow$	0.269 ± 0.000	0.136 ± 0.007	0.159 ± 0.023
Brier OOD (P75) $\downarrow$	0.561 ± 0.000	0.115 ± 0.011	0.093 ± 0.008

Table 4.7: Epistemic Method AE-RNN Results: Couette Dataset

Method	Ensemble_3_AE_RNN	GP_RNN	Evidential_RNN
Spearman $\uparrow$	0.063 ± 0.461	-0.200 ± 0.771	0.721 ± 0.256
Brier Error $\downarrow$	0.245 ± 0.070	0.468 ± 0.212	0.576 ± 0.055
ECE Error $\downarrow$	0.343 ± 0.076	0.514 ± 0.184	0.138 ± 0.038
ROC-AUC OOD $\uparrow$	0.574 ± 0.000	1.000 ± 0.000	0.964 ± 0.031
Brier OOD (P25) $\downarrow$	0.288 ± 0.000	0.142 ± 0.015	0.245 ± 0.046
Brier OOD (P50) $\downarrow$	0.244 ± 0.000	0.130 ± 0.003	0.164 ± 0.026
Brier OOD (P75) $\downarrow$	0.506 ± 0.000	0.119 ± 0.007	0.097 ± 0.005

4.5 Epistemic Uncertainty Propagation in the AE-RNN Pipeline for Coupled Deterministic Models

In this section, we will explore how to combine epistemic uncertainty estimates from the autoencoder (AE) and the recurrent neural network (RNN) across different models. This integration is simpler for ensemble and Gaussian Process (GP) models than for deterministic models, which require more complex propagation through the neural network (NN). Thus, we will focus on employing the fundamental propagation principle to integrate outputs from multiple deterministic epistemic models.

Although this topic is not in the scope of this master’s thesis and focuses more on the interactions between model architectures, we’ll discuss it last. This section will be presented in a general manner rather than a quantitative one, outlining basic evidential propagation, comparing principles for ensemble models, GPs, and evidential approaches, and examining the impact of integrating the RNN. Additionally, we will briefly illustrate how evidential epistemic uncertainty propagates throughout the pipeline, as depicted in Figure 5.17.

The hybrid architecture for flow predictions combines an autoencoder (AE) to compress spatial information into a latent space with an RNN that predicts how this representation evolves over time. Model uncertainty arises from the AE during encoding, the RNN during predictions, and the AE again during decoding. Importantly, these uncertainties are interdependent; the RNN’s predictions depend on the latent space influenced by the AE’s uncertainties.

Two critical points highlight this interdependence: 1. The RNN’s uncertainty is affected by the AE’s uncertainty during the latent transformation, resulting in combined prediction uncertainty. 2. The decoding process further compounds these uncertainties, necessitating a merge.

This interdependence prevents simply adding uncertainties across stages for deterministic

methods, distinguishing it from easier methods like Deep Ensembles. My investigations reveal that while recent works in fields like medical imaging address similar challenges (Feiner et al., 2023; Péron et al., 2025; Douglas et al., 2024), few up to none methods exist for dependent discriminative models UQ propagation approaches, especially not in scientific computing.

Given the limited references, the AE-RNN propagation described here is a new, basic and experimental propagation contribution over all emerging discriminative distributions through the full Evidential AE and RNN pipeline. Propagation is implemented by approximating upper and lower prediction bounds through the addition and subtraction of epistemic uncertainty. The resulting NIG distributions 2.1.4 from both after the RNN and the AE pass will be merge to assess the total propagated epistemic uncertainty, which must be repeated at least twice throughout the process, as illustrated in Figure 4.24 below, illustrating the basic principle, for just linear distributions.

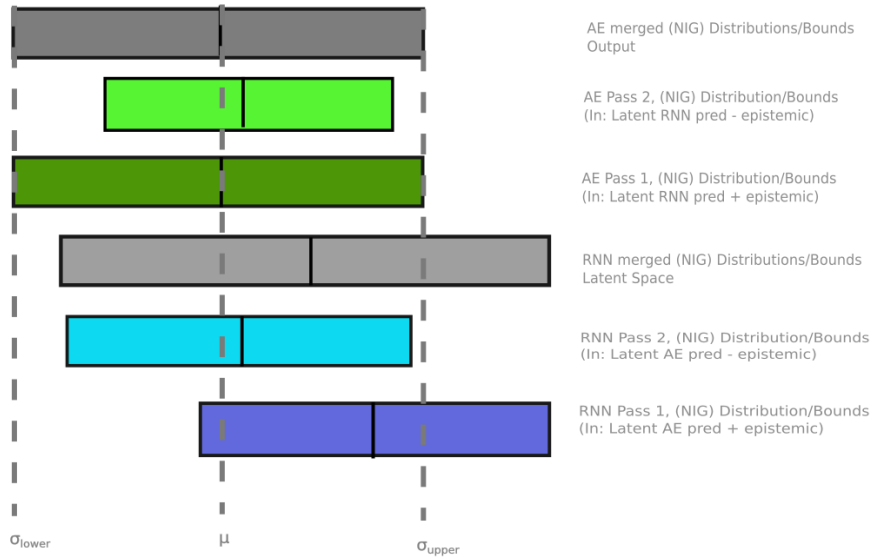


Figure 4.24: Example of how the merge processes are resulting the total propagated epistemic uncertainty. The epistemic uncertainty is propagated through the RNN (blue) and AE decoder (green), resulting in two distributions each that are again merged (grey).

Additionally our nested Autoencoder (AE) architecture, we face the challenge of receiving the complete distribution or bounds only at the end of the model pass, rather than during the latent space encoding. This complicates the introduction of uncertainty from the AE encoder.

To address this, we can either use a method that captures uncertainty quantification (UQ) directly in the latent space of the encoder and decoder separately, or choose between heuristic approaches regarding the epistemic uncertainty: performing a full AE Pass before or neglect

the initial encoder uncertainty altogether.

The first approach is to perform a complete AE pass without the Recurrent Neural Network (RNN) to obtain the AE's epistemic uncertainty, which is then encoded into the latent space for the RNN to propagate.

The second approach would be to ignore the encoder's induced uncertainty altogether and focus only on the uncertainties predicted from the AE decoder for the full AEs uncertainty, but this would not insert the AEs uncertainty into the RNN, and as was seen in 4.4 the AE can have a large effect on the RNNs epistemic uncertainty. Alternatively, we could also sample the input multiple times through the encoder, though this may be less effective than creating an AE ensemble.

Both approaches are illustrated in the propagation diagram 4.25 below.

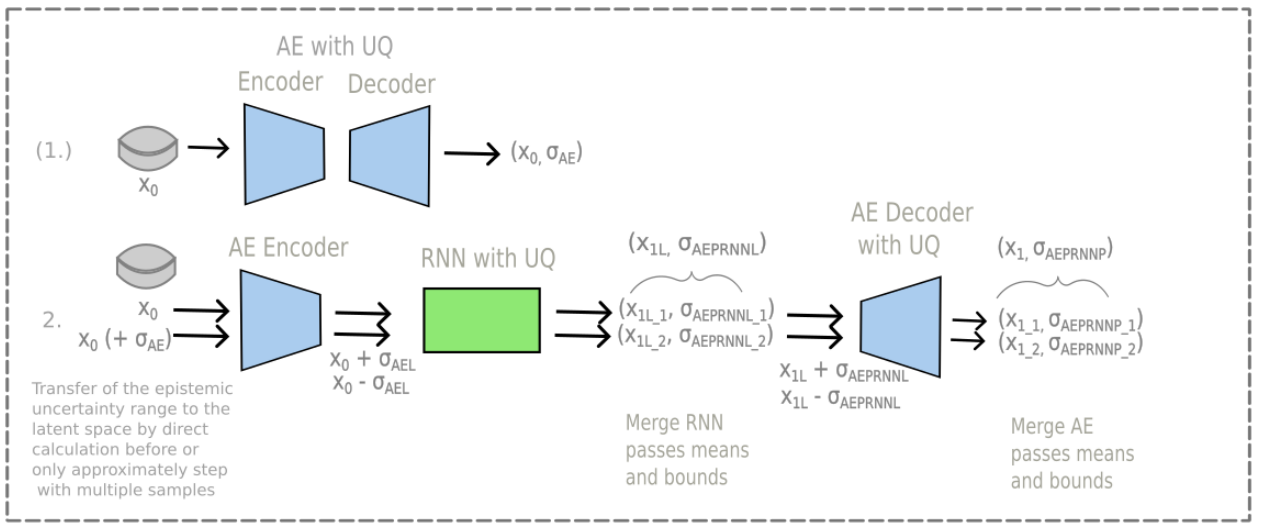


Figure 4.25: Sketch of the full uncertainty propagation approach through the pipeline. σ_{AE} and σ_{RNN} are the UQ method predicted AE and RNN epistemic uncertainty and L and P stand for latent space representation and propagated. We need to transform the σ_{AE} through the autoencoder into σ_{AEL} representation to feed to the RNN for propagation of the AE uq part σ_{AELP} .

Both approaches have strengths and weaknesses and are experimental. The Full approach captures a wider range of epistemic uncertainty by integrating Autoencoder (AE) uncertainty into the Recurrent Neural Network (RNN). However, it may overestimate uncertainty by assessing it twice for the complete AE and is also influenced by the encoding step.

Conversely, the second approach may lead to overconfident predictions, as it doesn't fully account for the AE encoder's impact on RNN predictions.

For now the first method is chosen to rather have under than overconfidence, therefore it will enable us to approximate the uncertainty propagation for the full AE-RNN evidential pipeline experimentally within this thesis.

5

Conclusion

5.1 Summary of Findings

In this master’s thesis different methods for addressing uncertainty in neural networks were explored through both theoretical discussion and practical implementation in a flow prediction model. The flow model is based on the AE-RNN from (Jarmatz et al., 2023) and was trained using simulations from the coupled MD-CFD MaMiCo software.(Neumann et al., 2016)

The architectures of the epistemic methods required adjustments to the basic model and retraining (e.g., NLL, MCDropout+NLL, Evidential). These architectures were trained, and their hyperparameters were fine-tuned using a grid search algorithm and inference testing, as discussed in Sections 4.2 and 5.3. For the Evidential approach, schedulers and value ranges were applied, as referenced in earlier publications (Amini et al., 2020). Additionally, sparse approximations for Gaussian processes and Laplace methods were used due to their high computational costs, as mentioned in Sections 2.1.4 and 2.1.4. The Gaussian Process (GP) method was particularly fine-tuned, with evaluations of different kernel functions and their hyperparameters, which significantly influenced the epistemic sensitivity, as detailed in Sections 5.3, 4.2.2, and 4.2.2.

The aleatoric methods were specifically examined for the trade-off between the narrowness of the fit and the inclusion of the most relevant distribution of ground truth data noise.4.2 We also assessed their influence on inference time and prediction accuracy. It was concluded that for our case of predominantly Gaussian noise, the negative log-likelihood (NLL) loss function provided the simplest and most robust training, yielding fast inference speeds. While the Evidential deep learning approach aimed to capture epistemic uncertainty as well, it could reduce overall inference time but might increase training complexity and slightly raise prediction errors. The Quantile method was implemented and tested as a non-parametric loss function; it performed well in terms of inference time and fit, as discussed in Section 4.1, although its convergence robustness was less good in comparison.

It is possible that newly integrating the aleatoric uncertainty component may not be needed

in future for all architectures, as current projects are exploring architectures including generative components a lot discussed in Section 2.1.3. These models, e.g. the Diffusion and Continuous Normalizing Flow (CNF) architectures, can be trained to inherently sample from the data distribution.

The epistemic methods were evaluated based on their training and convergence difficulties, inference time, memory usage, and impact on predictions, as outlined in 4.3.1. They were further analyzed in detail regarding their correlation with error and epistemic uncertainty, the separation of epistemic uncertainty from references to out-of-distribution (OOD) datasets, and OOD detection. The primary goals of the chosen neural network (NN) uncertainty quantification implementation were error correlation and OOD detection.

Each method exhibited different strengths and weaknesses. Generally, they succeeded in detecting TrueRandom as OOD 4.15 and 4.4, although some methods were more effective than others (e.g., Deep Ensemble, GP, and Evidential) compared to techniques like Laplace and particularly MCDropout. For Laplace, poor error correlation was expected due to its estimation limitations on local minima. In contrast, MCDropout’s particularly poor error correlation, despite good prediction accuracy, could also be influenced by the chosen dropout rate, as discussed in the Appendix regarding the MCDropout grid search 5.3. The Proper Brier score indicated that Deep Ensemble achieved the best calibration for OOD detection, as expected from existing literature, followed by GP and, surprisingly, Evidential, which is a newer and faster approach but requires more complex training.

Furthermore, we observed differences in the methods’ ability to detect OOD in Couette scenarios 4.15 and 4.19, which was not surprising, given the model’s good generalizability in that context. Methods assessing model variance, such as Deep Ensemble and MCDropout, often found Couette to be less strongly OOD, while GP predicted OOD in these scenarios more reliably. Particularly, both GP and Laplace exhibited strong saturation for Couette, detecting it robustly as OOD due to their fundamentally different ways of capturing epistemic uncertainty, as discussed in Section 4.3. While this approach might pose challenges for some models and scenarios—especially when the data significantly differs from the training set—in the context of flow prediction, a robust model trained on more complex scenarios could handle simpler flow scenarios effectively.

Based on the evaluation, the methods chosen for further implementation in the more complex AE-RNN pipeline include Deep Ensemble as a baseline, Gaussian Processes (GP), and, experimentally, Evidential approaches. For the Evidential method, a experimental propagation approach was used to connect the AE and RNN discriminative uncertainties in a interdependent way 4.5. This was necessary because the propagation of deterministic uncertainty quantification through a dependent model chain is less frequently discussed in the literature, to the best of my knowledge.

The AE-RNN results showed that both Gaussian Processes (GP) and Evidential methods performed surprisingly well, as noted in Section 4.4. Although Evidential used an experimental approach, it yielded commendable outcomes. In contrast, the Ensemble method was slightly less effective in out-of-distribution (OOD) and error detection, likely due to the Autoencoder (AE) used to train the Recurrent Neural Network (RNN).

Additionally, the RNN exhibited sensitivity to the connected AE used during training, leading to significantly higher epistemic uncertainty variance in AE variations compared to the RNN, for a fixed single model, as shown in Figure 4.21.

5.2 Discussion

One major point to discuss is the sensitivity of the Recurrent Neural Network (RNN) to Autoencoder (AE) choice. Therefore the results for AE-RNN Ensemble should be repeated with the RNNs trained on the individual AEs for each RNN, which was already done, and the models will be also uploaded to the GitHub repository.

Furthermore a fundamental point is the potential for improved fine-tuning across all neural networks (NNs). There may be an overlooked local minimum in the loss landscape, and enhancing the model complexity can also have a strong impact on this (Li et al., 2018). Especially for Monte Carlo Dropout (MCDropout), the fine-tuning of the dropout rate regarding loss and also the epistemic separation needs to be further explored.

Due to the limited comparisons of flow prediction uncertainty quantification (UQ) methods in existing literature, distinguishing between fine-tuning effects and method limitations is difficult. Thus, here there was a main focus on the overall behavior of these methods with various out-of-distribution (OOD) datasets for this case, keeping evaluations specific to this architecture and datasets. Furthermore the epistemic results were explored quantitatively and qualitatively. But further research into the sensitivity of the methods with different datasets could be beneficial. In particular more in-depth analysis on the differences in OOD detection results between the ID-OOTR and Couette.

Additionally, it would be valuable to see how the methods perform when trained reversed on Couette data and tested on KVS as an OOD case. This could provide more insights into OOD behavior for the methods and datasets, but would require additional time. Finally if the experiments were redone, incorporating benchmarking sets for UQ, which were later found, could enhance assessments and improve result comparability.

5.3 Future Work

First and foremost, the scope of this master's thesis is limited, and there is much more to explore regarding uncertainty quantification, as briefly noted in the Discussion section.

For instance, a review on this topic spans over 100 pages (He et al., 2023). Further notable examples include other data distance algorithms, such as Gaussian Processes (GP) and distance-based methods, as well as alternative approximation techniques for Bayesian Neural Networks (BNNs), like variational methods. If there is a strong need for highly accurate model variance in neural network uncertainty quantification (UQ), it may be necessary to maintain a consistent model architecture.

Additionally, combined frameworks are under development, and it is likely that new methods will be published in the near future that offer more robust UQ predictions and improved inference speeds. These methods may also integrate model architectures with inherent data distance variations more effectively.

Throughout this research, numerous challenges and open questions emerged. There is a lack of literature on practical comparisons of different uncertainty quantification methods, and very few studies address their specific incorporation into flow prediction models. Moreover, there is no detailed existing research on the AE-RNN architecture for flow prediction, especially concerning UQ incorporation and the propagation of uncertainty between dependent models in this context.

While the primary goal was to evaluate the integration and effectiveness of error detection and out-of-distribution (OOD) detection within the specific architecture, further research in this area would be valuable. Many new questions have arisen, indicating the need for additional specific focus based on this work.

Beyond general retraining for improved fine-tuning algorithms, it would be beneficial to give the MCDropout approach another chance in retrospect due to its easier combination of uncertainties in dependent model chains. This includes the previous discussion on the dropout rate balance. It could be interesting to investigate the loss landscape resulting from the architecture and flow prediction dataset. This could help further determine whether the observed behaviors of epistemic uncertainty and their robustness in Deep Ensemble, MCDropout, and Laplace methods are merely intrinsic to the specific models and training data or if they are systematically derivable and correlated with the loss landscape. While research already exists on loss landscape complexity concerning architecture and the volume/flatness of loss minima with varying datasets, as briefly discussed in 2.1.5, this area is still in its early stages and could yield additional insights into the behaviors of epistemic uncertainty quantification methods.

Moreover, the in-depth analysis of the dependencies between the AE and RNN model uncertainties, as well as their co-influences, was beyond the scope of this thesis and was only briefly examined in 4.4. In the future, it would be interesting to conduct a more comprehensive analysis of their relationship based on the trained models and the generated epistemic data, including a deeper exploration of the latent space generated by the AE and the RNN sensitivity. Furthermore, as mentioned, the architecture, particularly the RNN, was highly sensitive to training and incorporated methods. In recent years, new architectures have been developed to better enforce flow physics, such as Physics-Informed Neural Networks (PINNs),

Continuous Normalizing Flows (CNFs), and direct generative models that learn distributions (GANs, Diffusion models). Other sequence-based models, such as Transformers and general attention mechanisms (e.g., "Attention is All You Need," ([Vaswani et al., 2017](#))), have also demonstrated capabilities in capturing complex interactions over multiple scales more effectively. ([Truong et al., 2025](#); [Gao et al., 2025](#))

Appendix

In the appendix, we provide additional material that, while not directly part of the main thesis, serves as a helpful resource for readers. This allows them to easily reference information without needing to look it up, thereby concluding the thesis effectively.

We begin with a brief introduction to machine learning, particularly focusing on neural networks in Section 5.3. This section aims to elucidate the foundational concepts behind neural networks, to better understand the uncertainty methods applied to neural networks, as well as to better understand the advantages, disadvantages, and uncertainty capabilities of these methods.

Next, we discuss additional theoretical details regarding sparse, Laplace, and Gaussian methods in Section 5.3. While this information could have lengthened the theory section, it includes insights that were directly in some cases utilized in the thesis.

Furthermore, we provide procedural and theoretical details about the simulation software MaMiCo, as well as more advanced tables on the simulation configuration settings in Section 5.3. Additionally tables are presented that outline the training hyperparameters for the AE and RNN in Sections 5.3 for the baseline model and 5.3 for the new methods retraining and methods hyperparameters. Furthermore a sketch of the overall workflow, the structure of included running and analysis scripts is included in Section 5.3.

Also included adding to the results section, is the further results comparing different Gaussian Process kernels in Section 5.3.

Lastly, the repository that contains the model weights, configuration files, and visualization scripts, is linked and shortly introduced in Section 5.4.

Introductory Machine Learning and Neural Networks

This section offers a brief introduction to fundamental machine learning concepts for readers who may be unfamiliar with the field. Machine learning is a branch of artificial intelligence in which computers learn patterns from data without being explicitly programmed for each specific task. Instead of writing rules to solve a problem, we provide the computer with examples, allowing it to discover the underlying patterns through an automated learning process.

For those in atmospheric physics and related fields, machine learning can be viewed as a part of data-driven approach often in atmospheric physics refereed to as *inverse modeling*. Inverse modeling is a broader term for methods inferring model parameters or relationships

from observational data, instead of the other way around. Machine learning, particularly deep learning, represents a powerful set of inverse modeling methods that can learn complex, nonlinear relationships from large datasets, like regression does for less complex data, without the need for explicit physical equations.

Supervised Learning: Machine learning methods are often further divided into supervised and unsupervised methods. Neural networks in general are most often supervised learning methods, in contrast to e.g. clustering.

In supervised learning, the most common paradigm as visualized in Figure 5.1 used in this work, we have a dataset consisting of input-output pairs $\{(x_i, y_i)\}_{i=1}^N$, where x_i represents the input features (e.g., flow field measurements) and y_i represents the desired output or target (e.g., future flow state). The goal is to learn a function f that maps inputs to outputs: $f(x_i) \approx y_i$. The most well-known supervised learning task is regression.

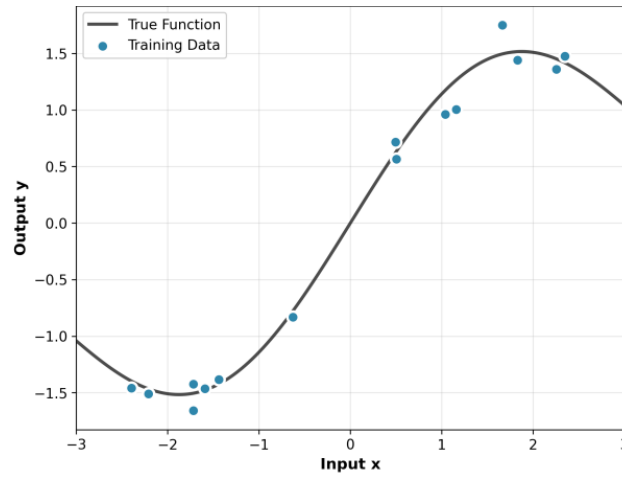


Figure 5.1: Sketch of a simple inverse modelling problem. We have observational data (blue) and want to construct a function or physical model approximating its behaviour (black).

During *training*, the model learns this mapping by examining example data, called "Training Data". During *inference* or *testing*, the model applies what it learned to make predictions on new, unseen inputs. The key challenge is *generalization*, ensuring that the model performs well on new data, not just the data it was trained on.

Neural Networks: Neural networks are computational models inspired by biological neurons, consisting of interconnected layers of simple processing units called *neurons* or *nodes*.

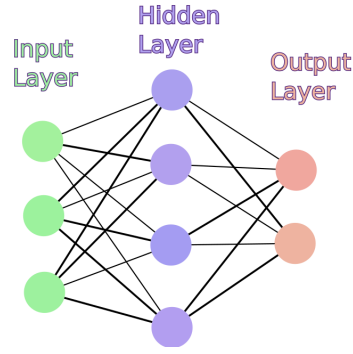


Figure 5.2: Sketch of a basic neural network, including an input layer, a hidden and a output layer. The content is propagating starting from the input layer through the neural network to the output layer, this is often called a forward pass or inference.

Each neuron receives inputs, performs a weighted sum, applies a nonlinear activation function, and produces an output. The network learns by adjusting the *weights and biases* (connection/edge strengths) between neurons to minimize prediction errors. A typical neural network has an *input layer* that receives the data, one or more *hidden layers* that process the information further, and an *output layer* that produces the final prediction. The depth (number of layers) and width (number of neurons per layer) determine the network's capacity to learn complex patterns.

Mathematically, a single layer computes:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where $\mathbf{x}$ is the input vector, $\mathbf{W}$ is a weight matrix, $\mathbf{b}$ is a bias vector, σ is a nonlinear activation function (such as ReLU: $\sigma(z) = \max(0, z)$), and $\mathbf{h}$ is the output. Deep neural networks usually stack multiple such layers, allowing them to learn hierarchical representations, with early layers detecting simple features (edges, patterns), while deeper layers combine these into more complex concepts.

Loss Functions and Optimization: To train a neural network, we need a way to measure how well it performs. A *loss function* (also called a cost function or objective function) quantifies the difference between the model's predictions and the true values e.g. observation. For regression tasks (predicting continuous values), a common loss function is the *mean squared error*:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2,$$

which penalizes larger errors more heavily. The goal of training is to find the network weights that minimize this loss function.

Gradient descent is one of the most commonly used optimization algorithm to minimize the loss. The idea is simple, if you want to find the bottom of a valley, you take steps in the direction of steepest descent. Mathematically, we compute the gradient (derivative) of the loss with respect to each weight, which tells us the direction of steepest increase. We then update the weights by moving in the opposite direction:

$$\theta_{new} = \theta_{old} - \alpha \nabla_{\theta} \mathcal{L},$$

where α is the *learning rate* (step size) and $\nabla_{\theta} \mathcal{L}$ is the gradient.

Figure 5.3 below shows basic illustrations of how a lower dimensional intersection for the higher-dimensional parameter loss landscape could look.

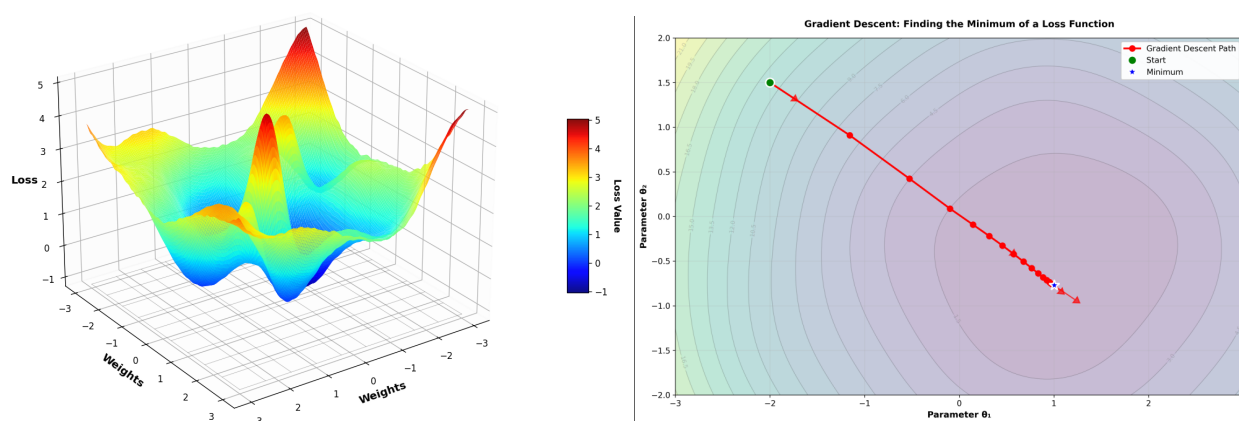


Figure 5.3: This is a sketch of an example intersection through a parameter loss landscape, as retrieved through backpropagation gradients (Left). Additionally, there is a sketch illustrating the basic behavior of an iterative convergence algorithm, such as the gradient descent algorithm, on a loss landscape as it converges to a loss minimum (Right).

In each iteration, the algorithm moves along the loss landscape in the direction of decreasing values or decreasing gradients, guided by a step size defined by the learning rate as visualized in 5.3 on the right. From this new position, the optimal weight values are updated. The goal is to reach the global minimum of the loss landscape, which corresponds to the weights yielding predictions that are as close as possible to the ground truth values.

The *backpropagation* algorithm efficiently computes the needed gradients for all weights in the network by applying the chain rule of calculus, propagating errors backward from the output layer to the input layer.

Training, Validation, and Testing: To properly evaluate a machine learning model, the data is typically divided into three sets.

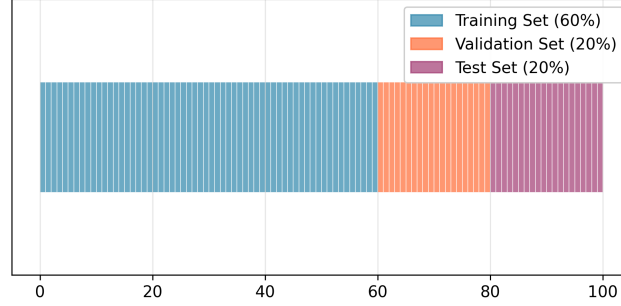


Figure 5.4: Most often split ratio for the total available dataset, for the whole training and testing procedure. The main part is used for the actual training, while Validation set is often used for further hyperparameter optimization and Test dataset for Inference test on not in training included data.

The *training set* is used to learn the model parameters. The *validation set* is used during training to monitor performance and adjust hyperparameters (settings like learning rate that are not learned from data). The *test set* is held out completely until the end and used only for final evaluation, providing an unbiased estimate of how well the model will perform on new data. This separation is crucial because a model might memorize the training data (*overfitting*) without actually learning generalizable patterns, leading to poor performance on new examples.

Further Theoretical Details for the Methods

Sparse Laplace Approximation:

To address the computational intractability of full Hessian computation and inversion, sparse Laplace approximations employ structured approximations of the Hessian matrix (Daxberger et al., 2021). The most common approaches include:

Diagonal Approximation: The simplest sparse approximation assumes independence between parameters, approximating the Hessian as a diagonal matrix (Daxberger et al., 2021):

$$\mathbf{H} \approx \text{diag}(\mathbf{H}) = \text{diag} \left(\frac{\partial^2}{\partial \theta_i^2} \log p(\mathbf{Y}|\mathbf{X}, \theta) \right).$$

This reduces storage from $\mathcal{O}(P^2)$ to $\mathcal{O}(P)$ and inversion from $\mathcal{O}(P^3)$ to $\mathcal{O}(P)$, where P is the number of parameters (Daxberger et al., 2021). The diagonal approximation is particularly efficient and can be computed using the diagonal of the Fisher information matrix or through second-order optimization methods. However, it ignores parameter

correlations, which can lead to overconfident uncertainty estimates, especially in networks where parameters are strongly coupled (Daxberger et al., 2021).

Sparse Gaussian Process Formulations

Full Gaussian Process inference scales cubically with the number of training samples n due to matrix inversion of the $n \times n$ covariance matrix K_n , making it computationally prohibitive for large datasets. Sparse Gaussian processes address this by introducing a smaller set of $m \ll n$ inducing variables $\{(\hat{x}_i, \hat{y}_i)\}_{i=1}^m$ to reduce complexity from $\mathcal{O}(n^3)$ to $\mathcal{O}(m^2n)$ (Snelson and Ghahramani, 2006; Titsias, 2009). These inducing points may lie anywhere in the input domain and are optimized jointly with GP hyperparameters to best approximate the full posterior (Titsias, 2009).

Sparse GP with Diagonal Approximation: For very large datasets, further computational savings are achieved through diagonal approximations of the covariance structure (Rasmussen and Williams, 2006). In the diagonal sparse GP formulation, the full covariance matrix K_n is approximated by its diagonal elements, reducing computational complexity from $\mathcal{O}(m^2n)$ to $\mathcal{O}(mn)$. The predictive mean and variance become (Rasmussen and Williams, 2006):

$$\mu^* = K_{x^*m}(K_{mm} + \Lambda)^{-1}K_{mx^*}y \quad (5.1)$$

$$\sigma^{*2} = K_{x^*x^*} - K_{x^*m}(K_{mm} + \Lambda)^{-1}K_{mx^*} + \sigma_n^2 \quad (5.2)$$

where μ^* is the predictive mean at test point x^* , σ^{*2} is the predictive variance, K_{mm} is the $m \times m$ covariance matrix between inducing points with entries $[K_{mm}]_{ij} = \kappa(\hat{x}_i, \hat{x}_j)$ where $\hat{x}_i$ are inducing points, K_{x^*m} is the $1 \times m$ row vector of covariances between the test point x^* and inducing points with entries $[K_{x^*m}]_j = \kappa(x^*, \hat{x}_j)$, K_{mx^*} is the transpose of K_{x^*m} (column vector), y is the vector of training targets at inducing points, $\Lambda = \text{diag}(K_{nm}K_{mm}^{-1}K_{mn})$ is a diagonal $m \times m$ matrix capturing the approximation error (difference between full and sparse GP covariances), $K_{x^*x^*} = \kappa(x^*, x^*)$ is the prior variance at the test point, and σ_n^2 is the observation noise variance. The diagonal approximation assumes independence between training points, which is reasonable when data points are sufficiently separated or when the inducing points capture the main structure of the data. While this approximation sacrifices some accuracy, it makes GP inference feasible on datasets that would otherwise be computationally prohibitive, which is valuable for high-dimensional latent spaces (See Section 2.3.1) where the number of training samples can be very large (Rasmussen and Williams, 2006). This computational efficiency is important for practical deployment of GP-based uncertainty quantification in flow prediction applications (See Section 4.2).

Simulation Software Details

Spatial Domain Decomposition and Macroscopic Cell Structure

The fundamental unit of the coupling approach is the *macroscopic cell*, which serves as the interface between the two simulation domains. The spatial domain is partitioned into a regular Cartesian grid of macroscopic cells, each with dimensions specified by the `cell-size` parameter (typically $2.5\sigma \times 2.5\sigma \times 2.5\sigma$ in Lennard-Jones units, where σ represents the molecular diameter). These cells span both the MD and CFD regions, providing a consistent spatial reference frame for data exchange.

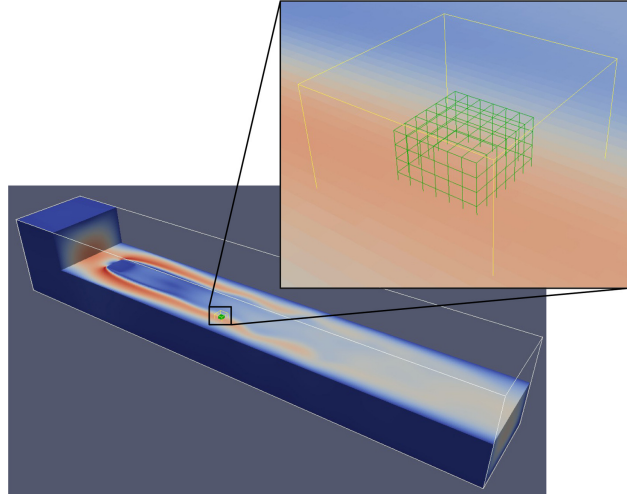


Figure 5.5: Example Illustration of a spatial alignment of a MD domain inside the macroscopic CFD channel domain for a KVS simulation configuration case.

The green box indicates the MD domain size and cell size. The yellow surrounding boxes visualise the expansion of the "coupling cells" not performing MD simulation, but used for transfer of mass and impuls between the solvers.

Within the MD domain, each macroscopic cell contains a specified number of linked cells (controlled by `linked-cells-per-macroscopic-cell`), which are used for efficient neighbor list construction in the MD solver. The coupling interface consists of several layers of macroscopic cells surrounding the core MD region.

These *overlap layers* facilitate the bidirectional transfer of information: the innermost overlap layers (specified by `innermost-overlap-layer`) extract microscopic quantities (mass, momentum) from MD to inform the CFD solver, while the outermost overlap layers (`outermost-overlap-layer`) impose macroscopic constraints from CFD onto the MD simulation.

Simulation Physics

Governing Equations and Scaling Procedures

The MaMiCo coupling framework (See Section 2.3) integrates molecular dynamics (MD) and lattice Boltzmann method (LBM) solvers, each governed by distinct equations that must be properly scaled to ensure physical consistency across scales.

Molecular Dynamics The MD solver integrates Newton’s equations of motion for N molecules with positions $\mathbf{x}_i$, velocities $\mathbf{v}_i$, and mass m (Allen and Tildesley, 2017; Rapaport, 2004):

$$\frac{d}{dt}\mathbf{x}_i = \mathbf{v}_i, \quad \frac{d}{dt}\mathbf{v}_i = \frac{1}{m}\mathbf{F}_i \quad (5.3)$$

where $\mathbf{F}_i$ are intermolecular forces computed from the Lennard-Jones potential (Allen and Tildesley, 2017):

$$U(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right] \quad (5.4)$$

where ϵ is the energy scale, σ is the length scale (molecular diameter), and r is the inter-particle distance. Forces are computed only for pairs within cutoff radius $r_c = 2.5\sigma$ to reduce computational cost (Neumann et al., 2016). The molecule configuration sets $\epsilon = 1.0$, $\sigma = 1.0$, and $m = 1.0$ in reduced Lennard-Jones units. Integration uses velocity-Verlet scheme with timestep $\Delta t_{MD} = 0.005$ (in LJ time units) (Allen and Tildesley, 2017; Rapaport, 2004).

Lattice Boltzmann Method The LBM solves the discrete Boltzmann equation on a regular lattice (Succi, 2001; Krüger et al., 2017):

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{1}{\tau} (f_i(\mathbf{x}, t) - f_i^{eq}(\mathbf{x}, t)) \quad (5.5)$$

where $f_i(\mathbf{x}, t)$ are distribution functions for discrete velocities $\mathbf{c}_i$, τ is the relaxation time related to kinematic viscosity via $\nu = c_s^2(\tau - 0.5)\Delta t$ with $c_s = 1/\sqrt{3}$ being the lattice speed of sound, and f_i^{eq} is the Maxwell-Boltzmann equilibrium distribution (Krüger et al., 2017). Macroscopic quantities are obtained as moments:

$$\rho = \sum_i f_i, \quad \rho \mathbf{u} = \sum_i f_i \mathbf{c}_i \quad (5.6)$$

where ρ is density and $\mathbf{u}$ is velocity. The implementation uses D3Q19 stencil (3 dimensions, 19 discrete velocities) with Two-Relaxation-Time (TRT) collision operator (Neumann et al., 2016).

Scaling Procedures Physical consistency requires matching dimensionless numbers (Reynolds number $Re = UL/\nu$) across scales. For characteristic physical length $L_{phys} = 0.1$ m, velocity $u_{phys} = 2.25$ m/s, and viscosity $\nu = 10^{-3}$ m²/s, the lattice relaxation parameter $\omega = 1/\tau$ is calibrated to maintain consistent Re behavior. This scaling ensures that MD fluctuations and LBM continuum fields represent the same physical flow regime, supporting reliable data generation for training the AE-RNN architecture (See Section 2.3.1).

Grid and time indexing

The MD solver integrates Newton’s equations of motion for N molecules using Verlet-type integration with timestep $\Delta t_{MD} = 0.005$ (LJ units) (Allen and Tildesley, 2017; Rapaport, 2004; Neumann et al., 2016). Intermolecular forces arise from the Lennard-Jones potential $U(r) = 4\epsilon[(\sigma/r)^{12} - (\sigma/r)^6]$ with $\epsilon = \sigma = m = 1.0$ (reduced LJ units), computed only for pairs within cutoff radius $r_c = 2.5\sigma$ to reduce computational cost. Linked-cell spatial decomposition ($\mathcal{O}(N)$ complexity) aligns with macroscopic cell dimensions for efficient coupling (Neumann et al., 2016).

The LBM solves the discrete Boltzmann equation on a regular lattice using distribution functions $f_i(\mathbf{x}, t)$ with D3Q19 stencil and TRT collision operator (Succi, 2001; Krüger et al., 2017; Neumann et al., 2016). Macroscopic quantities are obtained as moments: $\rho = \sum_i f_i$, $\rho \mathbf{u} = \sum_i f_i \mathbf{c}_i$. The relaxation time τ relates to kinematic viscosity via $\nu = c_s^2(\tau - 0.5)\Delta t$ with $c_s = 1/\sqrt{3}$.

Macro–Micro Coupling and Data Transfer

Time Stepping and Overlap Regions The coupling methodology employs nested time stepping where the particle system is advanced over n timesteps during every timestep of the continuum solver (Neumann et al., 2016; Jarmatz et al., 2023):

$$dt_C = n \cdot dt_P \quad (\text{nested time stepping}) \quad (5.7)$$

$$\Delta t_{coupling} = N_{MD} \Delta t_{MD} \quad (5.8)$$

$$N_{LB} = \text{round}\left(\frac{\Delta t_{coupling}}{\Delta t_{LB}}\right) \quad (5.9)$$

Thus the faster MD dynamics is able to equilibrate within each macroscopic timestep while maintaining temporal consistency across scales. Coupling is handled by macroscopic overlap layers: inner (MD→LBM) and outer (LBM→MD) (Neumann et al., 2016).

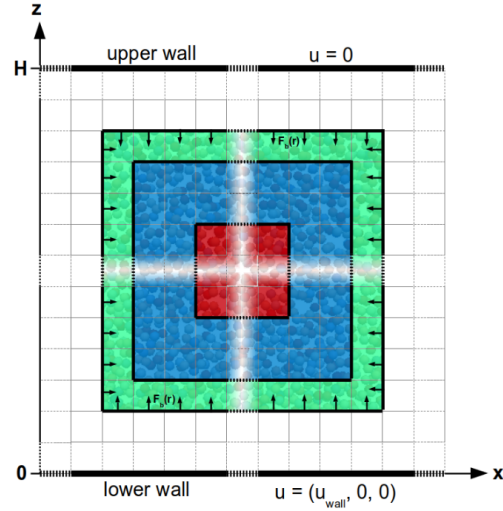


Figure 5.6: (Neumann et al., 2016). The figure shows an example sketch of a coupled transfer region for a couette setup from (Neumann et al., 2016), showing a 2D slice through 3D domain. H is the wall distance. Besides an additional ghost cell, the whole MD domain contains a mass transfer layer (green cells layer), velocity Macro->MD transfer layer (blue cells layer) and finally for a (arbitrarily large region), velocity and density values are sampled from the MD solver, post-processed, and sent back to the continuum solver (red cell layers).

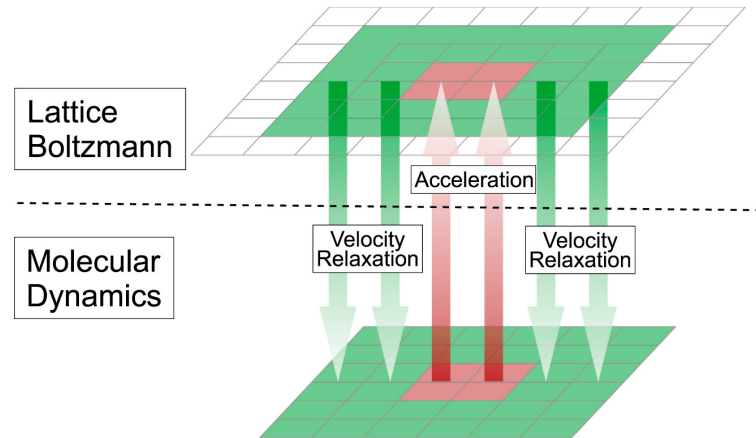


Figure 5.7: Sketch of the coupling and transfer algorithm between the MD and CFD domains if two-ways coupling is enabled, exchanging information between the scales/solvers both ways. (Neumann et al., 2016)

From the inner domain simulated by the MD solver velocity and mass information is overgiven to the LB grid, and the other way around LB simulation information is used as (boundary) forcing on the outer MD domain, here termed "velocity relaxation".

Coupling Equations and Transfer Mechanisms The MaMiCo framework (See Section 2.3) employs bidirectional coupling between MD and LBM domains through macroscopic cells, with nested time stepping where MD advances n timesteps per LBM timestep: $dt_C = n \cdot dt_P$ (Neumann et al., 2016; Jarmatz et al., 2023). This ensures faster MD dynamics equilibrate within each macroscopic timestep while maintaining temporal consistency.

Particle $\rightarrow$ Continuum (Inner Overlap): In the inner overlap region, microscopic quantities are sampled from particle data and transferred to the continuum solver. Continuum velocity is sampled cell-wise over time interval dt_C :

$$\mathbf{u} = \frac{dt_P}{N dt_C} \sum_{k=0}^{dt_C/dt_P-1} \sum_{i=1}^N \mathbf{v}_i(t_0 + k dt_P) \quad (5.10)$$

where N is the number of particles, $\mathbf{v}_i$ are particle velocities, dt_P is the particle timestep, and dt_C is the continuum timestep. Density for each macroscopic cell is computed as:

$$\rho(\mathbf{X}) = \frac{1}{V_{cell}} \frac{dt_P}{dt_C} \sum_{k=0}^{dt_C/dt_P-1} \sum_{i \in V(\mathbf{X})} m_i \quad (5.11)$$

where $V(\mathbf{X})$ denotes the volume of the macroscopic cell centered at $\mathbf{X}$, V_{cell} is the cell volume, and m_i are particle masses. Time-averaged quantities reduce thermal fluctuations inherent in MD simulations, providing smooth continuum fields for the LBM solver (Neumann et al., 2016).

Continuum $\rightarrow$ Particle (Outer Overlap): In the outer overlap region, continuum velocity and density fields from LBM are imposed onto MD particles. The *Nie velocity imposition* method adjusts particle velocities toward target macroscopic velocity while preserving temperature:

$$\mathbf{v}_i^{new} = \mathbf{v}_i^{old} + \alpha(\mathbf{u}^{CFD}(\mathbf{X}) - \langle \mathbf{v} \rangle_{\mathbf{X}}) \quad (5.12)$$

where α is a relaxation parameter, $\mathbf{u}^{CFD}$ is the velocity from the LBM solver, and $\langle \mathbf{v} \rangle_{\mathbf{X}}$ is the current mean velocity of particles in the macroscopic cell. Mass flux transfer is realized through particle deletion and insertion using the USHER algorithm, allowing selective mass flux across Cartesian boundaries (`mass-flux-west`, `mass-flux-east`, etc.) for physically consistent treatment of flow through the coupling interface (Neumann et al., 2016). This bidirectional transfer ensures that MD fluctuations inform continuum evolution while continuum constraints guide particle dynamics, creating the multi-scale data needed for training uncertainty-aware flow prediction models (See Section 2.3.1).

Boundary Conditions and Thermostat

Boundary conditions ensure physically realistic coupling: reflecting boundaries at the MD-continuum interface with boundary force $\mathbf{F}_b(r)$ compensating truncated interactions (Neumann et al., 2016). Temperature control uses selective thermostat application in outermost cell layers (typically 1-2 layers) to maintain isothermal coupling conditions while preserving adiabatic dynamics in the interior. The LBM solver employs standard wall-bounded flow boundary conditions: no-slip walls via bounce-back schemes, velocity Dirichlet conditions at inflow, and fixed density at outflow. Detailed boundary condition formulations are provided in (Appendix 5.3).

Boundary conditions in the MaMiCo framework (See Section 2.3.2) must ensure physical consistency at MD-continuum interfaces while maintaining appropriate thermodynamic conditions. The choice of boundary treatment directly affects the quality of generated training data for flow prediction models (See Section 2.3.1).

MD Boundary Conditions At the MD-continuum boundary, reflecting boundaries serve as the primary mechanism, preventing particle escape while allowing momentum transfer. Additionally, the boundary force $\mathbf{F}_b(r)$ proposed by Zhou *et al.* is applied at the outer MD domain boundary to estimate missing intermolecular force contributions from truncated interactions. This analytical fitting formula compensates for truncated molecular interactions at the domain edge, valid over a wide range of densities and temperatures, ensuring physically realistic boundary behavior that prevents artificial force artifacts from affecting the flow field (Neumann et al., 2016).

Temperature Control Temperature control in the MD domain is achieved through selective thermostat application. The `thermostat` configuration specifies where temperature rescaling is applied: `outerLayers` mode activates the thermostat only in a specified number of outermost cell layers (typically 1–2 layers), maintaining isothermal conditions at the coupling boundary while allowing adiabatic dynamics in the interior. This prevents artificial thermal perturbations in the region of interest while ensuring consistent thermodynamic conditions at the MD-CFD interface, which is critical for generating physically consistent training data (Neumann et al., 2016).

LBM Boundary Conditions The LBM solver employs standard boundary conditions appropriate for wall-bounded flows. No-slip walls are implemented using bounce-back schemes, where distribution functions are reflected at boundary nodes. Inflow boundaries use velocity Dirichlet conditions (UBB - Velocity Bounce Back), enforcing specified velocity profiles. Outflow boundaries apply fixed density conditions, allowing flow to exit the domain naturally. For the cylinder obstacle in KVS flow (See Section 2.2.2), a no-slip boundary is enforced on cells satisfying geometric criteria, creating the bluff body that generates vortex shedding.

These boundary conditions ensure that the generated flow data captures realistic physics essential for training reliable uncertainty quantification methods (See Section 4.3).

Simulation Settings and Data Preprocessing

Simulation Initialization

The simulation so of the KVS and Couette Flow Scenarios are done using the introduced MaMico Framework. Now I will shortly introduce the most relevant settings for the simulations in the starting Parameterfiles and their reason Table 5.1. Both scenarios use identical coupling strategies but differ in initialization requirements due to their flow characteristics.

Table 5.1: Running Parameters for KVS and Couette Simulations

Parameter	KVS (INI)	Couette (XML)
<i>Coupling Configuration</i>		
Coupling cycles	1000	1000
Two-way coupling	no	no
Send MD to Macro	yes	yes
Send Macro to MD	yes	yes
<i>Macroscopic Solver</i>		
Type	lbmpy	lb
Viscosity	2.14	2.14
Init timesteps/cycles	20000, 22000, 24000, 26000, 28000	0
Omega (relaxation)	1.9	N/A
<i>Microscopic Solver</i>		
Type	md	md
Temperature	1.1	1.1
Equilibration steps	10000	10000
Number MD simulations	1	1
Density	0.813037037	0.813037037

KVS simulations require extended initialization (20000–28000 timesteps) to establish periodic vortex shedding from an initially quiescent state. The multiple initialization timesteps (20000, 22000, 24000, 26000, 28000), following (Jarmatz et al., 2023), provide temporal diversity, capturing different phases of the shedding cycle, to than use all of them in the training. Couette flow reaches steady state rapidly due to its linear profile, requiring no special initialization. For Couette more diversification of the flow for the data is reached through different Upper-Velocity settings U_{wall} .

Data Preprocessing and Domain Merging

The table with the parameter file settings for the simulation, the description of the full data preprocessing steps from the raw simulation data, merging processes and flow convergence time needed are discussed in the appendix. Simulation output generates separate files for inner MD domains and outer boundary regions, requiring merging into unified 3D velocity fields for training. The preprocessing pipeline, with an merging algorithm based on the algorithm from my supervisor Olga Catalan, then processes multiple files in parallel, combining inner and outer domains for each coupling cycle and removing ghost cells to produce $24 \times 24 \times 24$ voxel grids per timestep. The full pipeline with all scripts will be uploaded to the GitHub Repository for easy reproducibility and continuations.

Additionally as can be seen for one point in one of the MD domains 5.8 the flow needs some time until it converge to its statistical stationary form, this first time steps are truncated, more precise the first $t=100$ time

Boundary Conditions and Coupling

Boundary conditions differ between scenarios: KVS uses periodic boundaries in all directions (appropriate for wake region), with mass flux transfer across all six boundaries. Couette uses reflecting boundaries (no-slip walls), with mass flux restricted to streamwise direction (west, east) matching channel geometry. Table 5.2 summarizes these differences.

Table 5.2: Boundary Conditions and Coupling Mechanisms

Parameter	KVS	Couette
MD boundaries	Periodic (all)	Reflecting (all)
Mass flux directions	All 6 directions	West, East only
Boundary force	None	zhou-boundary-force (all 6)
Transfer type	nie-transfer	nie-transfer
Particle insertion	usher	usher

Training Details and Hyperparameters

Reproduction of AE-RNN Results

The reproduction simulation follows the same configuration as (Jarmatz et al., 2023), with training hyperparameters summarized in Table 5.3. These parameters ensure consistency with the original work while enabling fair comparison of uncertainty quantification methods. The hybrid model consists of three components: a convolutional autoencoder (AE) for spatial feature extraction, a recurrent neural network (RNN) for temporal dynamics, and a triple model approach handling the three velocity components (x, y, z) separately.

Table 5.3: Reproduction Simulation Training Hyperparameters (from (Jarmatz et al., 2023))

Parameter	Value
<i>AE Architecture</i>	
Features	[4, 8, 16]
Input channels	1
Output channels	1
Activation	ReLU
<i>RNN Architecture</i>	
Input size	256
Hidden size	256
Sequence length	25
Num layers	1
<i>Training Configuration</i>	
Epochs	100/15
Batch size	32
Learning rate	1×10^{-4}
Optimizer	Adam
<i>Loss Function</i>	
AE loss	L1Loss
RNN loss	L1Loss
Triple model	Separate AE+RNN for u_x , u_y , u_z

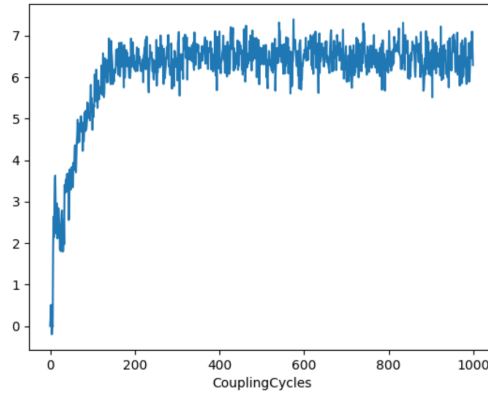


Figure 5.8: Raw simulation data showing the convergence period required before periodic KVS behavior emerges. The first 100 timesteps are discarded to ensure statistical stationarity.

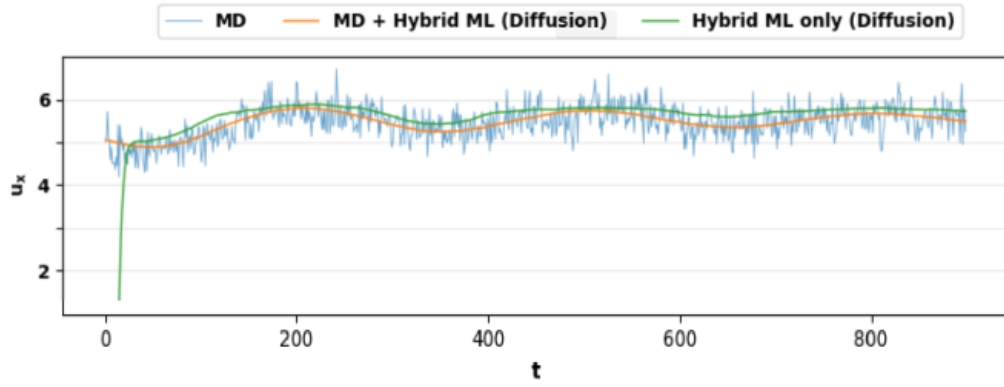


Figure 5.9: Reproduction simulation test for one inference file and one spatial position in the MD Domain, after removing the first 100 timesteps without any uncertainty quantification changes yet.

Hyperparameter Optimization and Training for Epistemic Methods

Table 5.4: Training Hyperparameters for Epistemic Uncertainty Methods

Parameter	Evidential	MC Dropout	NLL
<i>Architecture</i>			
Features	[4, 8, 16]	[4, 8, 16]	[4, 8, 16]
Output channels	4	2	2
<i>Training Configuration</i>			
Epochs	50	100	50–100
Batch size	2	32	32
Learning rate	1×10^{-4}	1×10^{-3} (varies)	1×10^{-3} (varies)
Optimizer	Adam	Adam	Adam
Weight decay	1×10^{-6}	–	–
<i>Loss Function</i>			
Loss type	EvidentialNIGLoss	NLLLoss	NLLLoss
Regularization coef.	1×10^{-4}	–	–
<i>Regularization</i>			
Gradient clipping	0.5	–	–
Dropout rate	–	0.001	–
Dropout layers	–	2perConv	–
MC samples (inference)	–	30	–
<i>Learning Rate Schedule</i>			
Scheduler	–	ReduceLROnPlateau	–
Scheduler factor	–	0.5	–
Scheduler patience	–	10	–

The evidential regression model uses a Normal-Inverse-Gamma (NIG) loss function with a regularization coefficient of 1×10^{-4} to balance predictive accuracy and uncertainty calibration. The Monte Carlo dropout approach employs dropout layers with rate 0.001 during training and performs 30 forward passes during inference to estimate epistemic uncertainty. Based on the final evaluation in 4.1, NLL is chosen for learning the aleatoric uncertainty, in cases, where there is only epistemic uncertainty (MCDropout, Laplace, GP, Ensemble). The NLL/in the grid search simple called aleatoric uncertainty model uses the NLLLoss.

The optimal learning rates and batch sizes were either chosen based on the paper by (Jarmatz et al., 2023) or searched in a grid search (especially evidential and mcdropout because of strong loss/architecture change).

The grid search parameter space is designed around the reference hyperparameters from the basic AE-RNN training.5.3. Evidential regression explored learning rates

$\{5 \times 10^{-5}, 1 \times 10^{-4}, 2 \times 10^{-4}, 5 \times 10^{-4}\}$, batch sizes $\{2, 4, 8, 16\}$, and regularization coefficients $\{1 \times 10^{-5}, 5 \times 10^{-5}, 1 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}\}$ (320 combinations). Monte Carlo dropout explores learning rates $\{5 \times 10^{-4}, 1 \times 10^{-3}, 2 \times 10^{-3}, 5 \times 10^{-3}\}$, batch sizes $\{16, 32, 64\}$, and dropout rates $\{0.0001, 0.001, 0.005, 0.01, 0.02\}$ (60 combinations). Aleatoric model explores learning rates $\{5 \times 10^{-4}, 1 \times 10^{-3}, 2 \times 10^{-3}, 5 \times 10^{-3}\}$, batch sizes $\{16, 32, 64\}$.

For mcdropout through the grid search the lowest loss was found for a dropout rate of 0.001. which is lower than the epistemic sensitivity finetuning tested range in 4.2.3 of 0.1. But because of the large variance in loss results for Dropout seen in boxplot Figure 5.10 showing the grid search loss result distribution.

Dropout hyperparameter variation in the choosen range influenced the prediction accuracy the most and also had the most convergence problems for dropout models to not have a bad performing model. Here it was decided to choose the lowest prediction accuracy influence, which influenced the good mcdropout results on low prediction accuracy negative influence, but probably also the less good error correlation.4.3 Based on the results for MCDropout further finetuning to find a better trade-off for dropout rate is needed. Overall evidential had a really low spread in loss during grid search, which correlates with the robustness of the converged architecture in 4.3, but also had the highest minimal achieved loss, probably because of the loss complexity.

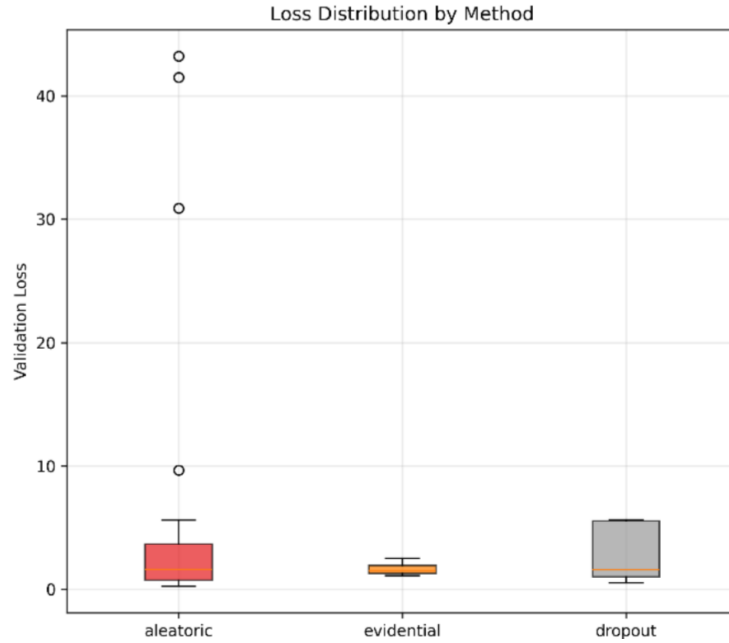


Figure 5.10: Grid search loss distributions for evidential regression, Monte Carlo dropout, and aleatoric uncertainty methods. The spread and best loss values reflect method-specific characteristics and optimization challenges.

Methodology Workflow Sketch

The main Python function files are grouped together in the brief sketch shown in Figure 5.11, illustrating the workflow. All models, training processes, and inference methods are organized into separate files. Additionally, there is a helper script that allows these methods to be executed sequentially or in parallel.

The saved results can be visualized using dedicated visualization scripts for comparing Aleatoric, EpistemicAE, EpistemicRNN, and AEtoRNN methods. There are also further scripts available for generating the plots included in this thesis, such as the latent space PCA shown in Figure 4.4.

It is important to note that the sketch does not fully represent the sequential nature of the workflow. In reality, all training, inference, and visualization scripts can be run in parallel, not just those from a single functionality group.

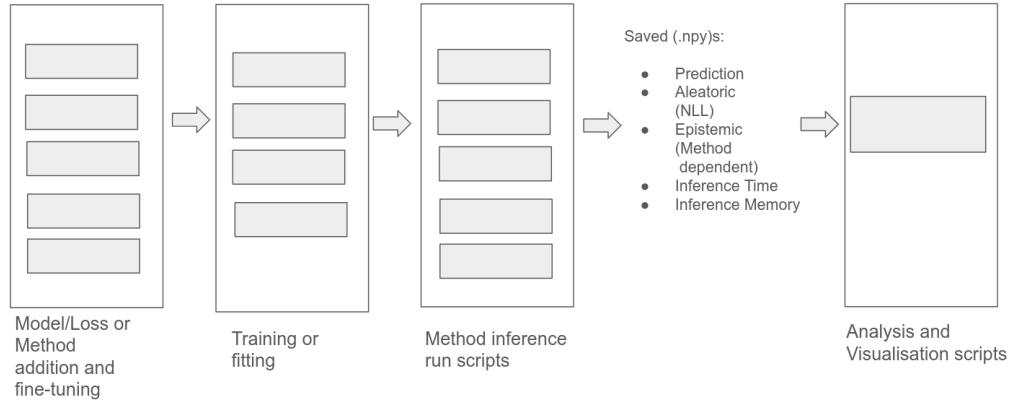


Figure 5.11: Sketch visualising the steps of retrieving all data used for the OOD detection comparison of the methods, specifically visualising that for each method each step can be run separately, in our case even having separate scripts, therefore giving rise to better parallelisation during the whole process until visualisation of the results.

The python files are uploaded in the GitHub repository linked below [5.4](#).

Gaussian Process Kernel Functions

The kernel function $\kappa(x_i, x_j)$ is the fundamental component of a Gaussian process, defining the covariance structure and determining the properties of functions that can be learned. The kernel must be positive definite, ensuring that the resulting covariance matrix is valid. This section provides detailed mathematical formulations of commonly used kernel functions

(Rasmussen and Williams, 2006; Duvenaud, 2014).

Radial Basis Function (RBF) Kernel:

The RBF kernel, also known as the squared exponential kernel, is the most widely used kernel for GPs (Rasmussen and Williams, 2006). It produces infinitely differentiable functions and is well-suited for smooth, stationary processes:

$$\kappa_{\text{RBF}}(x_i, x_j) = \sigma_f^2 \exp \left(-\frac{1}{2} \sum_{d=1}^D \frac{(x_{i,d} - x_{j,d})^2}{\ell_d^2} \right), \quad (5.13)$$

where σ_f^2 is the signal variance controlling the overall scale of the function, and ℓ_d are length-scale parameters for each dimension d that control how quickly the covariance decays with distance. Larger length scales produce smoother functions that vary slowly, while smaller length scales allow for more rapid variations. The RBF kernel assumes stationarity, meaning that the covariance depends only on the distance between points, not on their absolute locations (Duvenaud, 2014).

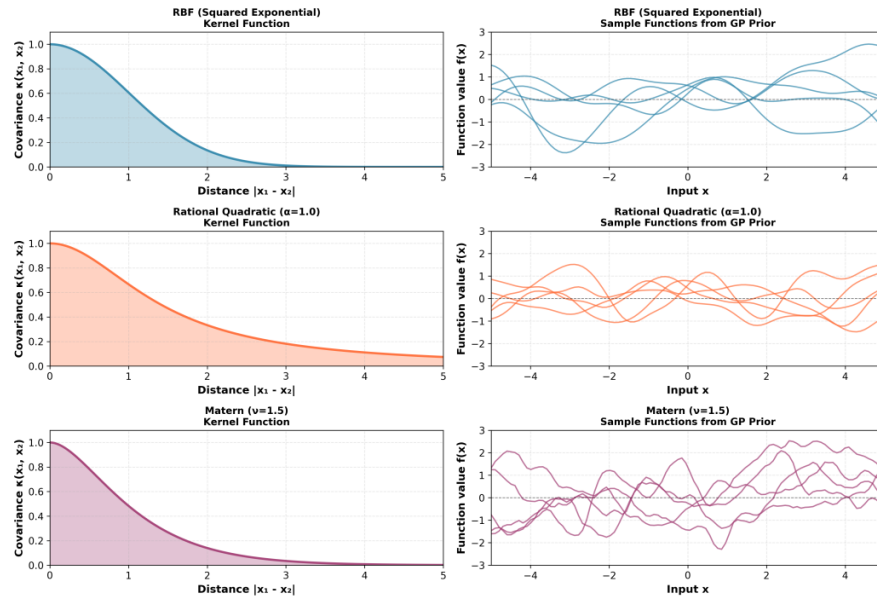


Figure 5.12: Plot of comparison of different often used kernel shapes and their prior samples for a toy case. The variance, sharpness and tail vary, impacting e.g. the smoothness/sharpness of the fit/distance calculation. Plot inspired by (Rasmussen and Williams, 2006)

Rational Quadratic (RQ) Kernel:

The Rational Quadratic kernel can be viewed as an infinite mixture of RBF kernels with

different length scales, providing more flexibility than a single RBF kernel (Rasmussen and Williams, 2006; Duvenaud, 2014):

$$\kappa_{\text{RQ}}(x_i, x_j) = \sigma_f^2 \left(1 + \frac{r^2}{2\alpha\ell^2} \right)^{-\alpha} \quad (5.14)$$

where $r = \|x_i - x_j\|$ is the Euclidean distance, ℓ is the characteristic length scale, and $\alpha > 0$ is the scale-mixture parameter that controls the relative weighting of different length scales. As $\alpha \rightarrow \infty$, the RQ kernel converges to the RBF kernel. The RQ kernel is particularly useful when the function exhibits variations at multiple scales, as it can capture both short-range and long-range dependencies simultaneously. The parameter α controls the tail behavior: smaller values of α produce heavier tails in the covariance function, allowing for more variation at larger distances (Duvenaud, 2014).

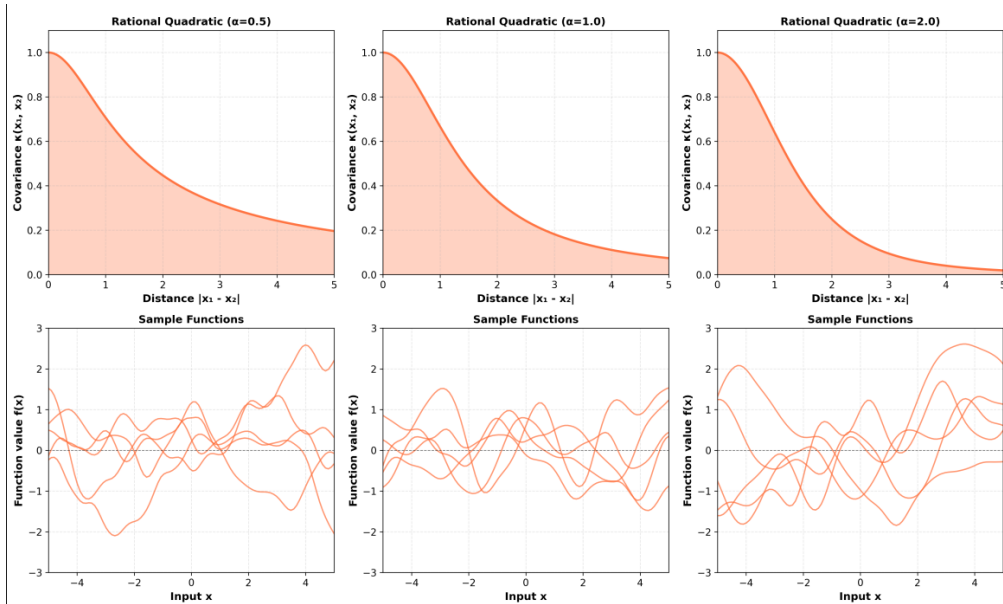


Figure 5.13: Effect of parameter α on the RQ kernel.

Matern Kernel:

The Matern kernel provides more flexibility in controlling function smoothness compared to the RBF kernel (Rasmussen and Williams, 2006; Duvenaud, 2014). It is defined as:

$$\kappa_{\nu}(x_i, x_j) = \sigma_f^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}r}{\ell} \right)^{\nu} K_{\nu} \left(\frac{\sqrt{2\nu}r}{\ell} \right) \quad (5.15)$$

where $r = \|x_i - x_j\|$ is the Euclidean distance, ν is a smoothness parameter (commonly $\nu = 1/2, 3/2, 5/2$, or ∞), ℓ is the length scale, $\Gamma(\cdot)$ is the gamma function, and $K_{\nu}(\cdot)$ is

the modified Bessel function of the second kind. The parameter ν controls the degree of differentiability: $\nu = 1/2$ produces functions that are once differentiable (equivalent to the exponential kernel), $\nu = 3/2$ produces functions that are once differentiable, and $\nu = 5/2$ produces functions that are twice differentiable. As $\nu \rightarrow \infty$, the Matern kernel converges to the RBF kernel. The Matern kernel is particularly useful when the underlying function is expected to have rougher behavior than what the RBF kernel can capture (Duvenaud, 2014).

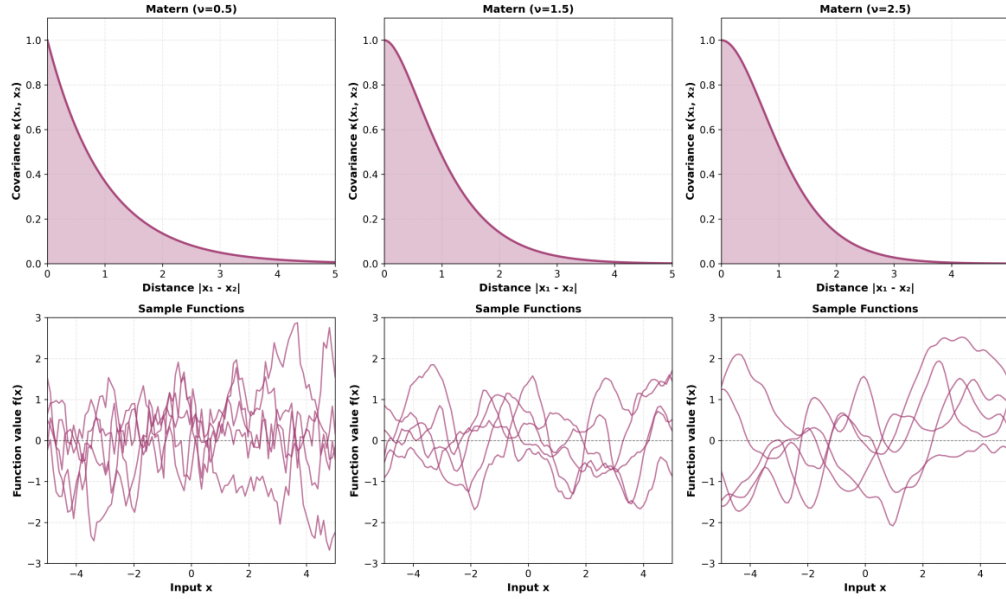


Figure 5.14: Effect of smoothing parameter ν on the Matern kernel. Differentiability and therefore smoothness increases with increasing ν , converging to the RBF kernel shape.

Linear Kernel:

The linear kernel produces linear functions and is equivalent to Bayesian linear regression (Rasmussen and Williams, 2006):

$$\kappa_{\text{linear}}(x_i, x_j) = \sigma_b^2 + \sigma_f^2 x_i^\top x_j, \quad (5.16)$$

where σ_b^2 is a bias term and σ_f^2 scales the linear relationship. This kernel is useful when the relationship between inputs and outputs is known or expected to be approximately linear, or as a component in composite kernels that combine multiple kernel types (Duvenaud, 2014).

Dot Product Kernel:

The dot product kernel, also known as the polynomial kernel of degree one, computes the inner product between inputs (Duvenaud, 2014):

$$\kappa_{\text{dot}}(x_i, x_j) = \sigma_f^2 x_i^\top x_j + \sigma_b^2, \quad (5.17)$$

where σ_f^2 scales the dot product and σ_b^2 is an optional bias term. Unlike stationary kernels (RBF, Matern, RQ) that depend only on the distance between points, the dot product kernel depends on the absolute positions of the inputs, making it non-stationary. This property makes it particularly useful when the magnitude or direction of the input vectors is meaningful, such as in high-dimensional feature spaces or when inputs have been normalized. The dot product kernel is equivalent to the linear kernel when $\sigma_b^2 = 0$, but the bias term allows for more flexibility. In practice, the dot product kernel is often used in combination with other kernels or in deep kernel learning settings where the inputs are learned neural network representations, where the inner product structure can capture meaningful similarities in the learned feature space (Duvenaud, 2014).

Kernel Hyperparameter Optimization:

The kernel hyperparameters (length scales ℓ_d , signal variance σ_f^2 , and noise variance σ_n^2) are typically optimized via maximum likelihood estimation (Rasmussen and Williams, 2006). The log marginal likelihood is:

$$\log p(\mathbf{y}|\mathbf{X}) = -\frac{1}{2}\mathbf{y}^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K} + \sigma_n^2 \mathbf{I}| - \frac{n}{2} \log(2\pi) \quad (5.18)$$

where $\mathbf{K}$ is the covariance matrix computed using the kernel function. The first term encourages data fit, the second term penalizes model complexity (preferring simpler models), and the third term is a normalization constant. Optimization is typically performed using gradient-based methods, where the gradients with respect to hyperparameters can be computed analytically. For deep kernel learning, where the kernel operates on learned neural network representations, the kernel hyperparameters and network weights are optimized jointly, enabling the network to learn representations that are well-suited for GP uncertainty quantification.

Gaussian Process Kernel Comparison Results

The kernel function shows can influence the resulting distance calculation as illustrated in the theoretical section on the kernels also in the appendix 5.3 Therefore, selecting an appropriate kernel can be important in improving the OOD-Detection through sufficient sensitivity and interpolation between distances.

The heatmaps for length scale (LS) and signal variance (SV) hyperparameter calibration, shown in Figure (Left) (See Figure(Left) 5.15), give an overview over the sensibility in epistemic uncertainty on the length schale and signal variance for three different kernels

influenced by this hyperparameter (RBF, Matern and RationalQuadratic, as introduced in 5.3, while on the right an overview of the best calibration scores and epistemic separations are highlight to show a first comparison of the best GP kernel results.

Furthermore in Figure (See Figure 5.16) below, we can observe the mean epistemic results for the GP across the different datasets, directly comparing the resulting dataset sensibility for the in this hyperparameter range finetuned kernels.

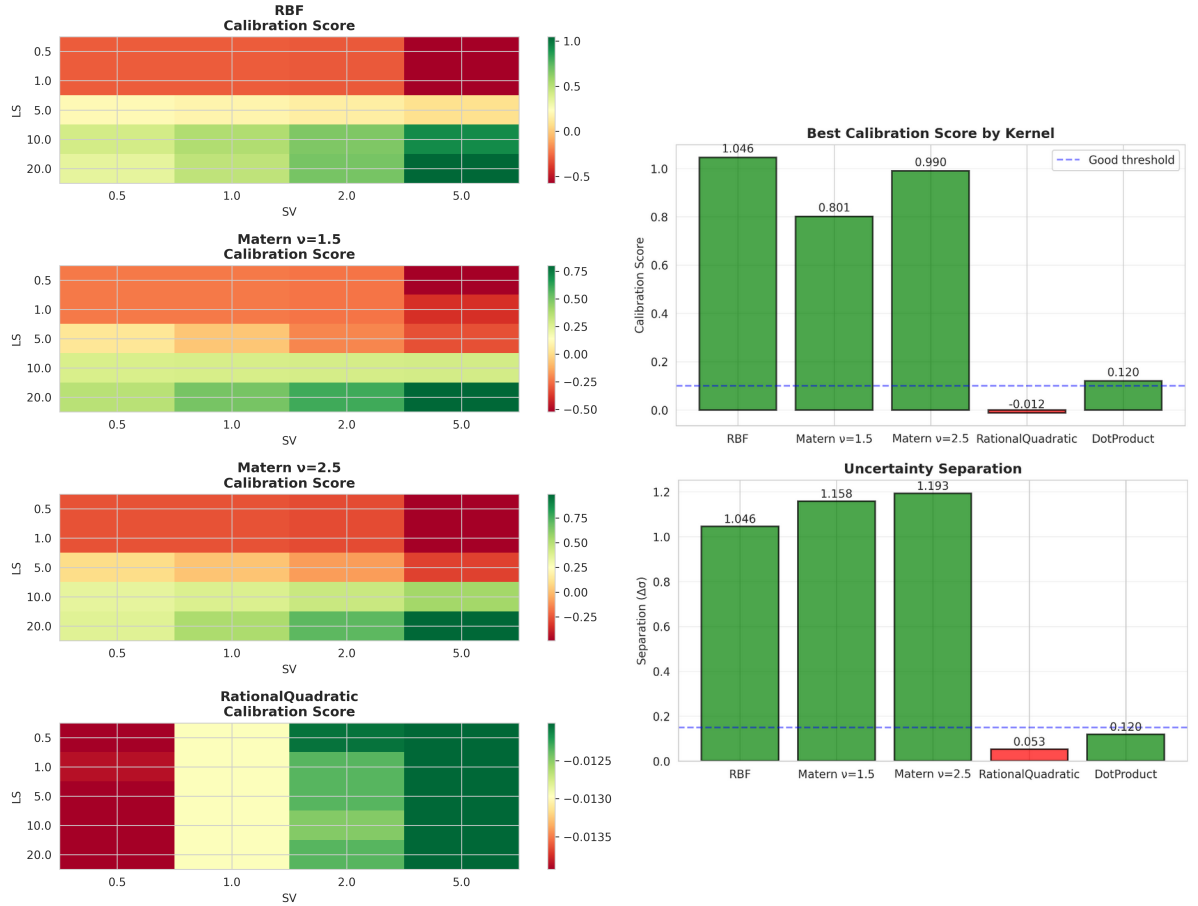


Figure 5.15: Kernel calibration heatmaps showing scores across different kernel types and hyperparameter combinations.

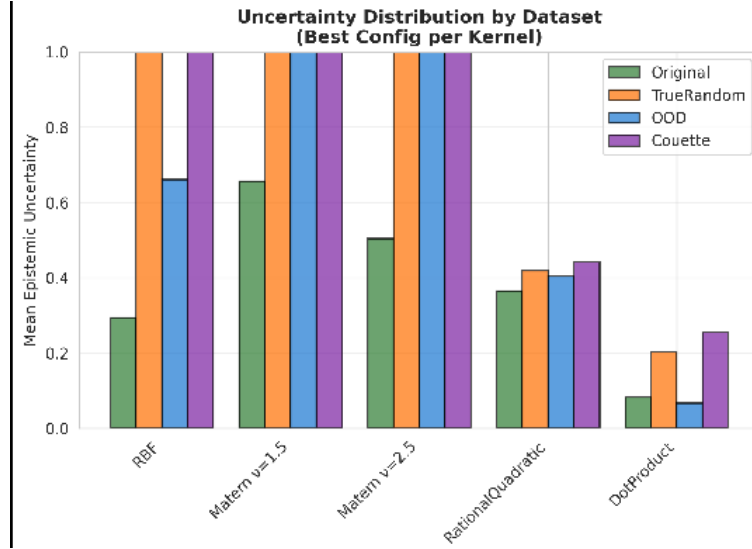


Figure 5.16: Uncertainty distribution by dataset for different kernel types.

The RBF kernel slightly outperformed the others in terms of calibration score, coming in just behind the Matern kernels. In contrast, the Rational Quadratic and Dot Product kernels consistently produced lower scores and exhibited poor separation across the range of hyperparameters tested.

Although the Matern kernel achieved the highest separation score, as shown in Figure 5.15, the RBF kernel demonstrated the best balance between sensitivity across the combined datasets 5.16 and a strong separation score within the tested hyperparameter range. Therefore, the RBF kernel has been selected for the implementation of the GP method.

Separation Metrics Results

Tables 5.5, 5.6, 5.7 below quantify distribution differences using more normalized distance metrics (Wasserstein distance, KL divergence, overlap coefficient, and normalized mean, median, and IQR differences) between Original and OOD uncertainty distributions. The quantitative results reinforce the qualitative observations: Ensemble and GP show strong separation across metrics, while MCDropout consistently shows poor separation with high overlap and low distance values, confirming the limitations visible in the boxplots and CDFs.

Table 5.5: Distribution Distance Metrics: TrueRandom Dataset

Method	W $\uparrow$	KL $\uparrow$	O $\downarrow$	μ $\uparrow$	Med $\uparrow$	IQR $\uparrow$
Ensemble	0.932	22.628	0.000	111.066	111.304	3.165
Evidential	0.198	5.101	0.412	1.096	1.267	-0.214
GP_LS10	0.681	22.458	0.000	50.363	51.172	15.724
Laplace2_3	0.052	0.870	0.739	-0.109	0.088	-0.362
MCDropout	0.026	0.210	0.751	0.262	0.279	0.433

Table 5.6: Distribution Distance Metrics: ID-OOTR Dataset

Method	W $\uparrow$	KL $\uparrow$	O $\downarrow$	μ $\uparrow$	Med $\uparrow$	IQR $\uparrow$
Ensemble	0.274	18.359	0.123	6.346	3.147	10.184
Evidential	0.092	3.858	0.603	0.425	0.576	-0.297
GP_LS10	0.489	21.946	0.000	20.954	18.563	8.518
Laplace2_3	0.091	2.406	0.662	-0.568	-0.409	-0.315
MCDropout	0.010	0.315	0.895	-0.093	-0.051	-0.110

Table 5.7: Distribution Distance Metrics: Couette Dataset

Method	W $\uparrow$	KL $\uparrow$	O $\downarrow$	μ $\uparrow$	Med $\uparrow$	IQR $\uparrow$
Ensemble	0.849	22.226	0.000	60.005	60.641	15.572
Evidential	0.190	11.279	0.345	0.986	1.272	-0.299
GP_LS10	0.966	22.496	0.000	76.989	77.376	1.695
Laplace2_3	0.294	18.921	0.018	-2.003	-1.884	-0.755
MCDropout	0.040	2.005	0.399	-0.368	-0.376	-0.506

Table 5.8: Distribution Distance Metric Abbreviations and Interpretations

Abbr.	Metric	Interpretation
W	Normalised Wasserstein distance	Distribution Separation
KL	Normalised KL divergence	Distribution Difference
O	Normalised Overlap coefficient	Distribution Overlap
μ	Normalised Mean difference	Mean Difference
Med	Normalised Median difference	Median Difference
IQR	Normalised IQR difference	IQR Difference

Evidential Propagation Bounds

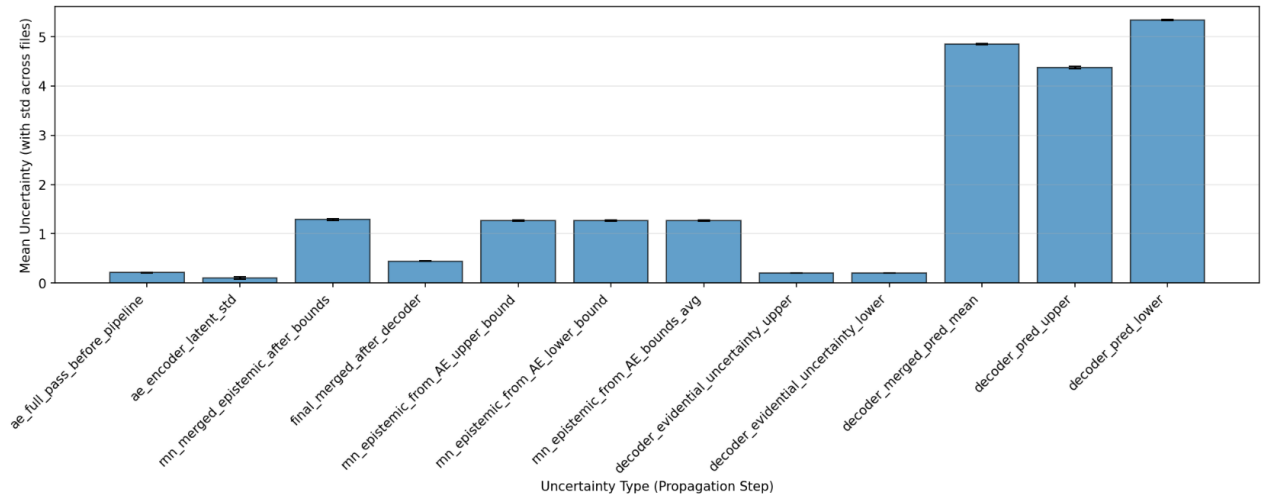


Figure 5.17: Barplots of the resulting mean magnitudes of epistemic uncertainty bounds, for multiple steps through the pipeline and after the merging processes, as well as the end prediction uncertainty including the epistemic uncertainty.

Histogram of Random OOD Dataset: AE vs AE-RN

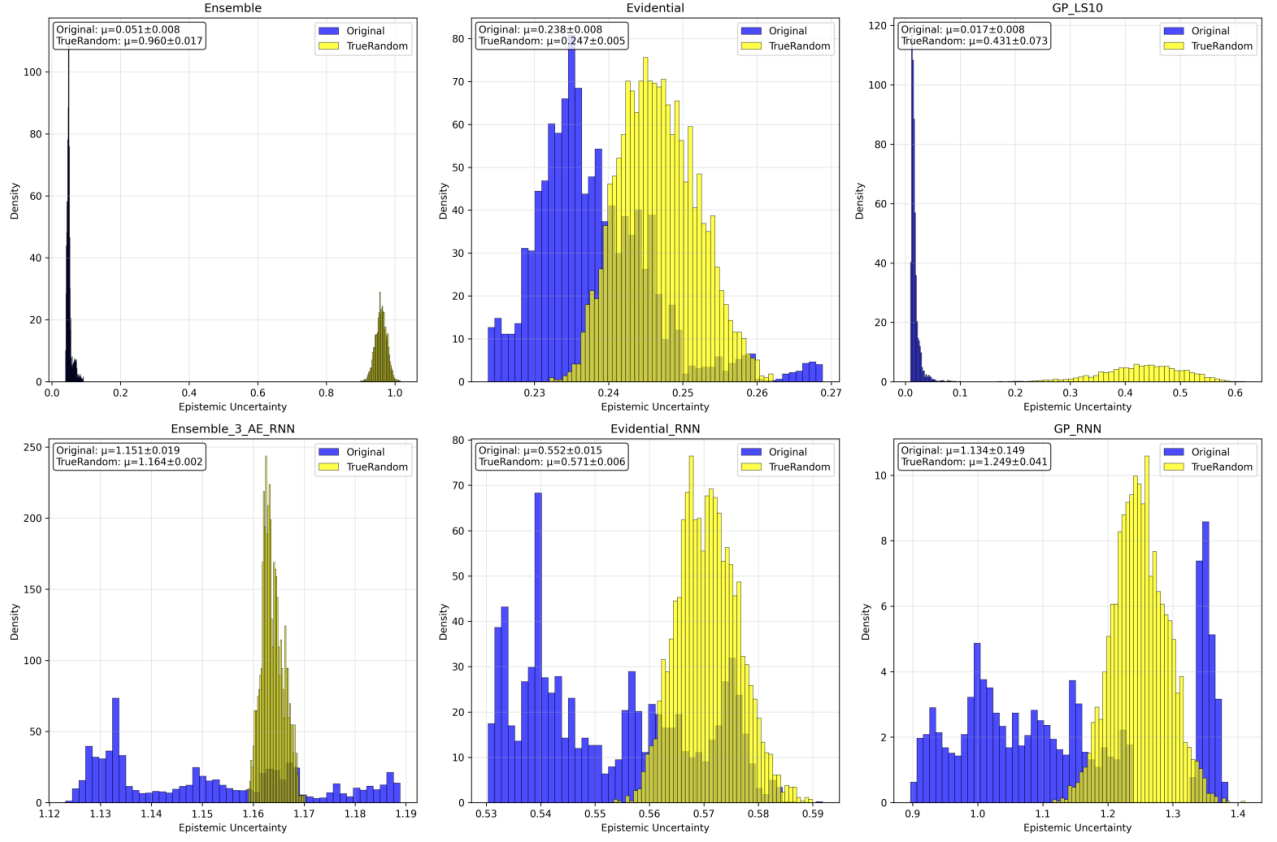


Figure 5.18: Histograms of AE vs AE+RNN uncertainty quantification methods

5.4 Code Availability: GitHub Repository

The model weights, config files and visualization scripts are uploaded on GitHub for reproduction and expandability.

The repository can be found under: <https://github.com/se04ber/flow-uq-thesis>

Bibliography

- Michael P Allen and Dominic J Tildesley. *Computer simulation of liquids*. Oxford university press, 2017.
- Alexander Amini, Wilko Schwarting, Ava Soleimany, and Daniela Rus. Deep evidential regression. *Advances in Neural Information Processing Systems*, 33:14927–14937, 2020.
- Anastasios N Angelopoulos, Stephen Bates, Jitendra Malik, and Michael I Jordan. Uncertainty sets for image classifiers using conformal prediction. *arXiv preprint arXiv:2109.14119*, 2021.
- George Keith Batchelor. *An introduction to fluid dynamics*. Cambridge university press, 2000.
- Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. Pangu-weather: A 3d high-resolution model for fast and accurate global weather forecast. *arXiv preprint arXiv:2211.02556*, 2023.
- Christopher M Bishop. *Pattern recognition and machine learning*. Springer, 2006.
- B. Blocken et al. Cfd simulation of pedestrian-level wind conditions around buildings: Past achievements and prospects. *Journal of Wind Engineering and Industrial Aerodynamics*, 104–106:103–130, 2012.
- Glenn W Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- George Casella and Roger L Berger. *Statistical inference*. Cengage Learning, 2nd edition, 2002.
- Junhwa Chi and Hyun-cheol Kim. Machine learning for sea ice prediction. *Remote Sensing*, 13(16):3204, 2021.
- K. P. Chopra and L. F. Hubert. Mesoscale eddies in wake of islands. *Journal of Atmospheric Sciences*, 22(6):652–657, 1965.
- Chu and Qian. Physics-guided machine learning for uncertainty and error estimation in turbulence-model predictions. In *Proceedings of the NeurIPS 2025 Workshop on Machine Learning and the Physical Sciences*, 2025.

-
- Maurice Couette. Études sur le frottement des liquides. *Annales de Chimie et de Physique*, 21:433–510, 1890.
- Andreas Damianou and Neil D Lawrence. Deep gaussian processes. *Artificial Intelligence and Statistics*, pages 207–215, 2013.
- Erik Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux-effortless bayesian deep learning. *Advances in Neural Information Processing Systems*, 34:20089–20103, 2021.
- John S Denker, Daniel Schwartz, Ben Wittner, Sara Solla, Richard Howard, Lawrence Jackel, and John Hopfield. Large automatic learning, rule extraction, and generalization. *Complex systems*, 1(5):877–922, 1987.
- Cameron Douglas et al. Uncertainty quantification in cascaded deep learning models for scientific computing. *Journal of Computational Physics*, 498:112678, 2024.
- David Duvenaud. Automatic model construction with gaussian processes. 2014.
- Andre Esteva, Brett Kuprel, Roberto A Novoa, Justin Ko, Susan M Swetter, Helen M Blau, and Sebastian Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639):115–118, 2017.
- European Commission. Proposal for a regulation laying down harmonised rules on artificial intelligence (artificial intelligence act). <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52021PC0206>, 2021.
- Lukas Feiner et al. Uncertainty quantification in cascaded deep learning models for medical imaging. *Medical Image Analysis*, 89:102915, 2023.
- Pablo Flores et al. Improved uncertainty quantification in physics-informed neural networks via two-step bayesian training. In *Proceedings of Machine Learning Research*, volume 286, 2025.
- Stanislav Fort, Gintare Karolina Dziugaite, Daniel M Roy, and Surya Ganguli. Deep ensembles: A loss landscape perspective. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Yarin Gal. Uncertainty in deep learning. *PhD thesis, University of Cambridge*, 2016.
- Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. *international conference on machine learning*, pages 1050–1059, 2016.

-
- Guoqiang Gao, Wei Zhang, Xiaofeng Li, and Yifan Wang. A deep neural network with attention mechanism for flow prediction. *Scientific Reports*, 15(1):99688, 2025. ISSN 2045-2322. doi: 10.1038/s41598-025-99688-0.
- Jakob Gawlikowski, Cedrique Rushi Nassi Tassi, Mohsin Ali, Jongseok Lee, Matthias Humt, Jianxiang Feng, Alexander Kruspe, Rudolph Triebel, Peter Jung, Ribana Roscher, et al. A survey of uncertainty in deep neural networks. *Artificial Intelligence Review*, 56(Suppl 1):1513–1589, 2023.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.
- Alex Graves. Practical variational inference for neural networks. *Advances in neural information processing systems*, 24, 2011.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. *International conference on machine learning*, pages 1321–1330, 2017.
- J. Halcrow et al. Turbulent-laminar patterns in plane couette flow. *Physical Review E*, 78: 016301, 2008. doi: 10.1103/PhysRevE.78.016301.
- James A Hanley and Barbara J McNeil. The meaning and use of the area under a receiver operating characteristic (roc) curve. *Radiology*, 143(1):29–36, 1982.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2nd edition, 2009.
- Xinran He, Kaiyong Zhao, and Xiaowen Chu. A survey of uncertainty in deep neural networks. *Artificial Intelligence Review*, 56(Suppl 1):1513–1589, 2023.
- Geoffrey E Hinton and Drew Van Camp. Keeping the neural networks simple by minimizing the description length of the weights. *Proceedings of the sixth annual conference on Computational learning theory*, pages 5–13, 1993.
- Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

-
- Ákos Horváth and Hartwig Deneke. Evolution of an atmospheric kármán vortex street from high-resolution satellite winds: Guadalupe island case study. *Monthly Weather Review*, 148(8):3205–3224, 2020. doi: 10.1175/MWR-D-19-0392.1.
- R. Iskhakov, S. Taghizadeh, J. Ling, E. Parish, et al. Quantifying out-of-training uncertainty of neural-network turbulence closures. *arXiv preprint arXiv:2508.16891*, 2025. URL <https://arxiv.org/abs/2508.16891>.
- Piet Jarmatz, Sebastian Lerdo, and Philipp Neumann. Convolutional recurrent autoencoder for molecular-continuum coupling. In *International Conference on Computational Science (ICCS 2023)*, page 140760520, 2023. URL <https://www.iccs-meeting.org/archive/iccs2023/papers/140760520.pdf>.
- Christopher Kadow, David M Hall, and Uwe Ulbrich. Artificial intelligence reconstructs missing climate information. *Nature Geoscience*, 13(6):408–413, 2020.
- Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In Isabelle Guyon, Ulrike V. Luxburg, Samy Bengio, Hanna Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30, pages 5574–5584. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/6cbbc9e0ba78df492284685b5a2658dd-Paper.pdf.
- Abbas Khosravi, Saeid Nahavandi, Doug Creighton, and Amir F Atiya. Lower upper bound estimation method for construction of neural network-based prediction intervals. *IEEE transactions on neural networks*, 22(3):337–346, 2011a.
- Abbas Khosravi, Saeid Nahavandi, Doug Creighton, and Amir F Atiya. Comprehensive review of neural network-based prediction intervals and new advances. *IEEE Transactions on neural networks*, 22(9):1341–1356, 2011b.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2014.
- Benjamin Kompa, Jasper Snoek, and Andrew L Beam. Second opinion needed: communicating uncertainty in medical machine learning. *NPJ Digital Medicine*, 4(1):4, 2021.
- Timm Krüger, Halim Kusumaatmaja, Alexandr Kuzmin, Orest Shardt, Goncalo Silva, and Erlend Magnus Viggen. *The lattice Boltzmann method: principles and practice*. Springer, 2017.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.

-
- Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirsberger, Meire Fortunato, Alexander Pritzel, Suman Ravuri, Timo Ewalds, Ferran Alet, Zach Eaton-Rosen, et al. Graphcast: Learning skillful medium-range global weather forecasting. *Science*, 382(6677): 1416–1421, 2023.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Hao Li, Zhengxu Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018. URL <https://arxiv.org/abs/1712.09913>.
- X. Li et al. Urban canyon vortex shedding and its impact on pollutant dispersion: A review. *Atmospheric Environment*, 223:117–135, 2020.
- Jeremiah Z. Liu, Ziqi Lin, Shuying Chua, Michael Lu, Jason Y. Tamir, Andrew Matuschak, Charles D. Freeman, Guy Bresler, and Jiequn Tang. Simple and principled uncertainty estimation with deterministic deep learning via distance awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 17561–17573, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c6d4d5b3c5e9c1b0d0f4d5b3c5e9c1b0-Paper-Conference.pdf.
- David JC MacKay. A practical bayesian framework for backpropagation networks. *Neural computation*, 4(3):448–472, 1992.
- L. Mahrt et al. Nocturnal boundary-layer regimes. *Boundary-Layer Meteorology*, 88:255–288, 1998.
- P. L. McDermott and C. K. Wikle. Bayesian recurrent neural network models for forecasting and quantifying uncertainty in spatio-temporal data. *Entropy*, 21(9):905, 2019.
- John Moody and Christian J Darken. Fast learning in multi-resolution hierarchies. *Advances in neural information processing systems*, 1, 1989.
- Radford M Neal. Bayesian learning via stochastic dynamics. *Advances in neural information processing systems*, 6, 1993.
- Philipp Neumann, Holger Flohr, Ravi Arora, Hans-Joachim Bungartz, and Wolfgang Eckhardt. Mamico: Software design for parallel molecular-continuum flow simulations. *Computer Physics Communications*, 200:1–11, 2016. URL <https://www.sciencedirect.com/science/article/abs/pii/S0010465515004129>.
- T. R. Oke. Street design and urban canopy layer climate. *Energy and Buildings*, 11(1–3): 103–113, 1988.

-
- T. R. Oke, G. Mills, A. Christen, and J. A. Voogt. *Urban Climates*. Cambridge University Press, Cambridge, 2017. ISBN 9781107429512.
- Ian Osband, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy. Deep exploration via bootstrapped dqn. *Advances in neural information processing systems*, 29, 2016.
- Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D Sculley, Sebastian Nowozin, Joshua V Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. *Advances in Neural Information Processing Systems*, 32, 2019.
- Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ankur Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, et al. Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. *arXiv preprint arXiv:2202.11214*, 2022.
- Guillaume Péron et al. Uncertainty propagation in particle tracking with deep learning. *Journal of Computational Physics*, 510:113045, 2025.
- Daniel Pollithy, Marcel Reith-Braun, Patrick Pfaff, and Uwe D. Hanebeck. Estimating uncertainties of recurrent neural networks in multi-target tracking. In *Proceedings of the 18th International Conference on Informatics in Control, Automation and Robotics (ICINCO) Workshop on Multisensor Fusion and Integration for Intelligent Systems*, 2020. URL https://isas.iar.kit.edu/pdf/MFI20_Pollithy.pdf.
- Ilan Price, Alvaro Sanchez-Gonzalez, Ferran Alet, Tom R. Andersson, Andrew El-Kadi, Dominic Masters, Timo Ewalds, Jacklynn Stott, Shakir Mohamed, Peter Battaglia, Remi Lam, and Matthew Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637:84–90, 2025. doi: 10.1038/s41586-024-08252-9.
- Alvin Rajkomar, Eyal Oren, Kai Chen, Andrew M Dai, Nissan Hajaj, Moritz Hardt, Peter J Liu, Xiaobing Liu, Jake Marcus, Mimi Sun, et al. Scalable and accurate deep learning with electronic health records. *NPJ digital medicine*, 1(1):18, 2018.
- D. C. Rapaport. *The art of molecular dynamics simulation*. Cambridge university press, 2004.
- Edward N Rappaport. Fatalities in the united states from atlantic tropical cyclones: New data and interpretation. *Bulletin of the American Meteorological Society*, 95(3):341–346, 2014.
- Carl Edward Rasmussen and Christopher KI Williams. *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006.

-
- Stephan Rasp, Peter D Dueben, Sebastian Scher, Jonathan A Weyn, Soukayna Mouatadid, and Nils Thuerey. Weatherbench: A benchmark data set for data-driven weather forecasting. *Journal of Advances in Modeling Earth Systems*, 12(11):e2020MS002203, 2020.
- Anatol Roshko. On the drag and shedding frequency of two-dimensional bluff bodies. Technical Note 3169, NACA, 1954.
- David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- Robin Senge, Stefan Bösner, Krzysztof Dembczyński, Jürgen Haasenritter, Oliver Hirsch, Norbert Donner-Banzhoff, and Eyke Hüllermeier. Reliable classification: Learning classifiers that distinguish aleatoric and epistemic uncertainty. *Information Sciences*, 255:16–29, 2014. doi: 10.1016/j.ins.2013.07.045.
- Murat Sensoy, Lance Kaplan, and Melih Kandemir. Evidential deep learning to quantify classification uncertainty. *Advances in neural information processing systems*, 31, 2018.
- Frank Shih, Zhenghao Jiang, and Faming Liang. Uncertainty quantification for physics-informed neural networks with extended fiducial inference. *arXiv preprint arXiv:2505.19136*, 2025.
- Edward Snelson and Zoubin Ghahramani. Sparse gaussian processes using pseudo-inputs. In *Advances in neural information processing systems*, volume 18, 2006.
- Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.
- Gabriel Sperrer. *Uncertainty Quantification of Deep Learning Reduced-Order Models for Fluid Dynamics*. PhD thesis, TU Wien, 2025. URL <https://repositum.tuwien.at/handle/20.500.12708/221761>.
- T. Stathopoulos. Pedestrian level winds and outdoor human comfort. *Journal of Wind Engineering and Industrial Aerodynamics*, 94(9):769–780, 2006.
- Sauro Succi. *The lattice Boltzmann equation: for fluid dynamics and beyond*. Oxford university press, 2001.
- L. Sun, J.-X. Wang, and Z. Gao. Physics-constrained bayesian neural network for fluid flow reconstruction with sparse and noisy data. *Journal of Computational Physics*, 416:109545, 2020. doi: 10.1016/j.jcp.2020.109545.
- Jingfeng Tan, Bo Zhang, Erran Li, and Hao Chen. Single forward pass uncertainty estimation with neural networks. *arXiv preprint arXiv:2303.02265*, 2023.

-
- N. Tillmark and P. H. Alfredsson. Experiments on transition in plane couette flow. *Journal of Fluid Mechanics*, 313:243–258, 1996. doi: 10.1017/S0022112096002209.
- Michalis K Titsias. Variational learning of inducing variables in sparse gaussian processes. In *Artificial intelligence and statistics*, pages 567–574. PMLR, 2009.
- Eric J Topol. High-performance medicine: the convergence of human and artificial intelligence. *Nature medicine*, 25(1):44–56, 2019.
- Thanh-Dat Truong, Christophe Bobda, Nitin Agarwal, and Khoa Luu. Mango: Multimodal attention-based normalizing flow approach to fusion learning, 2025. URL <https://arxiv.org/abs/2508.10133>.
- Miguel M. Valero and Matteo Meldi. Improved state estimation for turbulent flows combining ensemble data assimilation and machine learning. *Computers & Fluids*, page 106757, 2025. ISSN 0045-7930. doi: 10.1016/j.compfluid.2025.106757.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017. URL <https://arxiv.org/abs/1706.03762>.
- von Kármán, Theodor. Über den mechanismus des widerstandes, den ein bewegter körper in einer flüssigkeit erfährt. *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse*, pages 509–517, 1911.
- von Kármán, Theodor. Über den mechanismus des widerstandes, den ein bewegter körper in einer flüssigkeit erfährt. zweite mitteilung. *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse*, pages 547–556, 1912.
- Ke Wang, Chongqiang Shen, Xingcan Li, and Jianbo Lu. Uncertainty quantification for safe and reliable autonomous vehicles: A review of methods and applications. *IEEE Transactions on Intelligent Transportation Systems*, 26(3):2880–2896, 2025. doi: 10.1109/TITS.2025.3532803.
- Frank M White. *Fluid mechanics*. McGraw-Hill Education, 2011.
- C. H. K. Williamson. Oblique and parallel modes of vortex shedding in the wake of a circular cylinder at low reynolds numbers. *Journal of Fluid Mechanics*, 206:579–627, 1989.
- C. H. K. Williamson. Vortex dynamics in the cylinder wake. *Annual Review of Fluid Mechanics*, 28:477–539, 1996.
- WMO. Guidelines on ensemble prediction systems and forecasting. Wmo-no. 1091, WMO, 2021.

Y. Zhang et al. Atmospheric flow parameters and vortex dynamics from high-resolution observations. *Journal of Geophysical Research: Atmospheres*, 124:e2019JD032121, 2019. doi: 10.1029/2019JD032121.